SunSpot Protocol

Proposal A

Author	retc3, SunSpot maintainer
Document	SSP-PROP-A
Version	1.0.0
Date	1 August 2026
Protocol version	SSP/1

Abstract

The SunSpot Protocol (SSP) carries private messages between people without a central operator. It gives end-to-end confidentiality, forward secrecy, post-compromise security, and strong resistance to metadata collection.

SSP has no directory, no phone numbers, and no accounts. An identity is a self-authenticating, hash-linked, append-only log that holds device keys, rotations, revocations, and recovery rules. The identifier of an identity is the hash of the first entry of that log. It is stable for the life of the identity, and it survives full key replacement.

The identity of a user and the address that the network uses are separate. A sender and a recipient derive an opaque 32-byte mailbox address for their pair, and that address changes every day. A relay stores ciphertext under mailbox addresses. It does not learn an identity, a display name, a social graph, a group identifier, or a membership list. It does learn traffic patterns, and 3.11.6 states what those reveal.

Relays are store-and-forward nodes. They are interchangeable and untrusted. A relay accepts a deposit only when the recipient has registered a deposit key for that mailbox. That one rule removes open relaying, removes the need for a reputation system, and confines proof of work to first contact.

Transport uses QUIC with raw public keys, or TCP with a Noise IK handshake where a client must cross a censored or onion network. Session key agreement combines X25519 with ML-KEM-768. Message keys come from a double ratchet. Groups use TreeKEM epochs with a deterministic rule for concurrent commits, and need no ordering server. Hashing, key derivation, and content addressing use BLAKE3, and attachments use its tree structure for verified streaming.

Normative statements carry an identifier, a rationale, and a verification method. The verification methods describe work that is planned, not work that has been carried out: no implementation, test corpus, or proof exists at the time of writing. Section 4 gives the plan and Appendix A the traceability matrix. Section 1.3 lists the decisions still open.

Prior art

SSP reuses published work and names it where it does.

- The Noise Protocol Framework supplies the handshake patterns and the naming discipline.
- WireGuard shows that a small, opinionated cryptographic core with no agility is a strength.
- The Signal protocol supplies the double ratchet and asynchronous key agreement.
- RFC 9420 supplies TreeKEM.
- SimpleX shows that queue-based addressing removes long-lived network identifiers.
- Briar shows what a protocol looks like when the network is assumed hostile.
- Cabal and Secure Scuttlebutt show the strengths and the costs of append-only logs.
- Tor supplies Equi-X and a decade of measurement against funded denial-of-service attacks.
- Monero research sets the standard for honest statements about the limits of a privacy system.

Where SSP differs from these systems, Section 3 and Appendix F state the difference and defend it.

Contents

1 Introduction

1.1 Purpose

This specification defines the SunSpot Protocol: the rules that let independent programs exchange private messages with no central operator.

The specification is the source of truth. A team that has this text and the referenced cryptographic standards can write a client that interoperates with every other client, and does not need to read anyone else's source code.

1.2 Scope

In scope:

- Creation, rotation, revocation, and recovery of cryptographic identities.
- Enrolment and removal of the devices that belong to one identity.
- Authenticated, confidential links between a client and a relay.
- Asynchronous delivery between two devices, inside a closed group, and on a one-to-many channel.
- Transfer of attachments with incremental integrity checks.
- Synchronisation of state between the devices of one identity.
- Storage and expiry of ciphertext at a relay.
- Prevention of spam and denial of service with no identity registry.
- Reduction of the metadata that a relay or a network observer can collect.
- Negotiation of versions and of optional capabilities.

Out of scope:

- The user interface, the notification style, and the local database schema.
- Real-time voice and video media transport. Section 3.17 reserves the code points that a later media specification needs.
- Moderation policy, keyword filters, and reporting workflows.
- Interoperation with XMPP, Matrix, SMTP, or SMS.
- Payment, tokens, ledgers, and any mechanism that needs global state.

1.3 Status of this document

SSP/1 is a working draft written for contributor review before implementation begins. The architecture is complete enough to build against, and the decisions below are unresolved. Each one changes the wire format or the security argument, so each needs maintainer agreement before the format is frozen.

Table 1. Open decisions.

Ref	Decision	Alternatives and trade-off
O-01	Group fan-out cost (3.9.6)	Pairwise mailboxes cost one deposit per recipient device and hide the membership set from the relay. A shared group mailbox costs one deposit and exposes that set. No third option is known that keeps both properties.

Ref	Decision	Alternatives and trade-off
O-02	The plaintext `ETYPE` field (3.11.3)	The relay needs an object class for quota and expiry. Keeping five values in the clear leaks the class; folding the class into the mailbox registration moves the leak from the object to the mailbox; removing it collapses the abuse model.
O-03	Ed25519 with hybrid key agreement (3.7.12)	Keeping a classical signature holds the identity log small and leaves authenticity classical. Adopting ML-DSA-65 now adds roughly 200 KiB to a log with fifty entries and eight devices.
O-04	Recovery thresholds m and r (3.6.11)	A low claim threshold makes recovery easy for the user and easy for a thief. A high rejection threshold makes a stolen key harder to use and makes a legitimate rejection harder to assemble. The two are not independent: $r > m$ means a claim is easier to make than to stop. Analysis is needed against a stated key-storage model before either is fixed.
O-05	Offline pre-registration window (3.8.7)	The number of future mailbox epochs a recipient registers in advance sets the maximum offline period that still receives mail. Longer windows increase relay storage, widen the set of addresses one relay can link to one client, and raise the cost of a mailbox rotation.
O-06	Concurrent policy update during a pending recovery claim (3.6.8)	This document uses snapshot semantics: a rejection is judged under the policy at the claim's `head_ref`. The alternative is to honour a policy update ordered before the rejection, which lets a fast legitimate holder respond to a stolen key, and equally lets an attacker holding a device key install a friendly policy mid-decision.

SSP/1 is not compatible with any SunSpot build that came before this document. 3.17.3 states why the break happens once.

1.4 Product overview

1.4.1 Position in the system

Today the SunSpot application defines behaviour and the wire format is a side effect of the source code. That order is wrong, and it blocks a second implementation. The target relationship is strict, and it is shown in Figure 1.

SSP-GEN-001 An implementation MUST NOT depend on behaviour that this specification does not state.

Rationale. Undocumented behaviour becomes a de-facto standard and turns one client into the specification.
Verification. Inspection, plus the interoperability suite of 4.5: a second implementation built only from this document completes the mandatory function set.

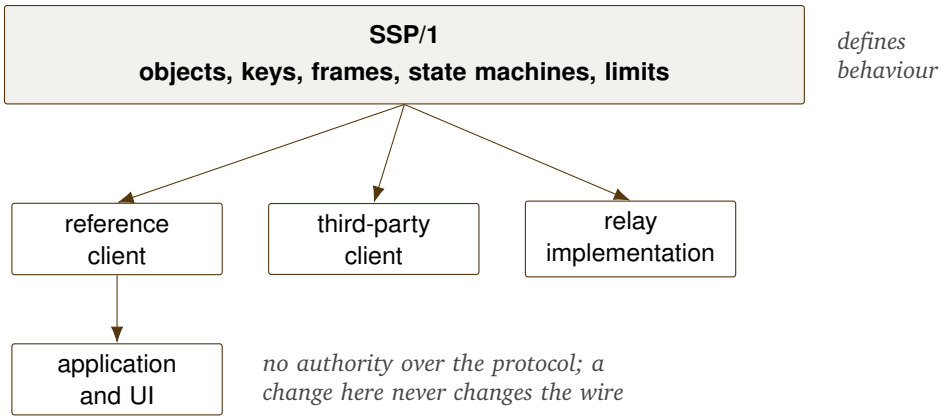

Figure 1. Authority runs one way. The specification constrains every implementation, and no implementation constrains the specification.

SSP-GEN-002 The reference client MUST NOT hold any protocol privilege, and MUST NOT use a code point outside the public registry of Appendix C.

Rationale. A privileged client is a central authority in a decentralised network.
Verification. Inspection of the reference client against the registry: it uses no code point outside the published ranges and no operation this document does not define.

1.4.2 Actors and objects

Actor	Definition
Identity	One long-lived cryptographic principal. Not a person.
Device	One installation that holds private keys for one identity.
Client	The program on a device that speaks SSP.
Relay	A store-and-forward node holding ciphertext under mailbox addresses.
Observer	Any party that watches the network. Not an actor of the system.

Table 2. Actors.

Object	Purpose
Identity log	The append-only, hash-linked record of an identity.
Contact code	The out-of-band token that starts a first contact.
Mailbox	An opaque, rotating destination at a relay.
Envelope	The unit that a relay stores and forwards.
Message	The end-to-end object inside an envelope.
Blob	A content-addressed attachment body.

Table 3. Object classes.

1.4.3 User classes

Ordinary users accept defaults, do not read this document, and scan a code to add a contact. They must get strong security with no configuration. *Operators* run relays; they have a server and modest operational skill, and they must not be able to break confidentiality even if they try. *High-risk users* face a state adversary and will accept extra cost: onion routing, constant-rate cover traffic, and manual key checks.

SSP-USE-001 The protocol MUST give ordinary users end-to-end encryption, forward secrecy, and post-compromise security with no user action.

Rationale. Security that needs configuration is security that most users do not have.
Verification. Analysis of the default parameter set in Appendix B, plus a demonstration that a client installed with no configuration establishes a session with forward secrecy and performs a ratchet step on first reply.

1.4.4 Limitations

The protocol cannot remove these facts. Section 3.11 gives the detail.

- A relay learns the IP address of the client that connects to it, unless the client uses an onion network or a VPN.
- An observer that watches every link can correlate traffic by timing and by volume. SSP raises the cost of this attack; it does not stop it.
- A device that an adversary controls can copy every plaintext that it reads.
- A member of a group can copy and republish the content of that group.
- Availability depends on relays. An adversary that blocks every relay of a user blocks that user.

1.4.5 Assumptions and dependencies

Ref	Assumption
A-01	Each device has a cryptographically secure random number generator.
A-02	Each device has a monotonic clock, and a wall clock accurate to five minutes for most of the time.
A-03	The user can compare a short code out of band for at least one contact.
A-04	At least one reachable relay is honest about availability.
A-05	X25519, Ed25519, ChaCha20-Poly1305, BLAKE3, and Argon2id resist a classical adversary for the life of SSP/1.
A-06	ML-KEM-768 resists a quantum adversary, or the hybrid of 3.7.12 retains classical security if ML-KEM-768 fails.
A-07	The operating system offers a place to store a wrapped key, or the user supplies a passphrase.

Table 4. Assumptions.

SSP-GEN-003 An implementation MUST fail closed if assumption A-01 does not hold, and MUST NOT fall back to a weaker source of randomness.

Rationale. A silent fallback to a predictable generator destroys every other guarantee.
Verification. Test with a blocked entropy source: key generation fails and reports, and no key is

produced from any fallback source.

1.5 Motivation

Four problems drive the design.

The application is the protocol. A second implementation cannot exist while the truth lives in the source of one client. Every fix to that client silently changes the network. This blocks review, blocks embedded clients, and ties the network to one language and one repository.

Encryption is not privacy. Signal encrypts content well and holds a phone number for every user. Matrix encrypts content and then replicates room membership, timestamps, and the social graph to every server in the room. Both show that metadata is the harder problem, and that it leaks through the addressing layer rather than through the cipher.

Identity is either fragile or unstable. A system that uses a raw public key as the identifier cannot rotate that key. A system that uses a server account can rotate keys and needs the server. Both models fail a protocol intended to last a decade, because keys leak and servers close.

Abuse prevention pulls the design toward the centre. Reputation needs a global view. Phone numbers need a registry. Payment needs a ledger. Each solves spam and creates a point that an adversary can attack, buy, or subpoena. A decentralised protocol needs an abuse defence that one untrusted node can enforce from local state alone.

SSP answers these in order: a written protocol, an addressing layer that carries no identity, a log-based identity that rotates keys under a stable name, and a recipient-issued capability that gates every deposit.

1.6 Design philosophy

Protocol first, implementation second, application third. The protocol defines behaviour, the implementation realises it, the application presents it. A layer never defines the behaviour of a layer above it in that list. The practical test: if a change needs an edit to this document it is a protocol change and takes the version rules of 3.17; if it does not, it is a product decision and never touches the wire.

One way to do each thing. Each function has exactly one mandatory mechanism. There is no cipher suite negotiation at the message layer, no choice of serialisation, and no optional integrity mode. WireGuard shows the value of this rule and TLS 1.0 through 1.2 shows the cost of breaking it. A protocol with N optional mechanisms has more than N test paths, and the rare paths hold the defects. Algorithm agility lives at the version boundary, never inside a session.

A relay is furniture. A relay is not a server that a community trusts. It is a disk with a network port: a public locker bank, not a post office. Everything must hold when every relay in the world is hostile, apart from the availability of the messages passing through it.

Refuse to be clever. Any mechanism whose complexity outweighs its gain is rejected, and the rejection is recorded next to the decision it lost to.

Say what you cannot do. Sections 3.2.5, 3.4, and 3.11.6 list the attacks that SSP does not stop. A privacy claim without a stated limit is marketing, and a user who believes a false claim takes a risk they did not choose.

Canonical bytes. Every object that a party signs or hashes has exactly one valid encoding, and a parser rejects anything else. This removes signature malleability, removes hash confusion, and makes content addressing sound.

1.7 Terminology

1.7.1 Technical verbs

These verbs carry these meanings throughout. An implementation description MUST NOT substitute a synonym.

Verb	Meaning
encrypt, decrypt	apply, or remove, an authenticated cipher
sign, verify	produce, or check, a digital signature
hash	compute a fixed-length digest of a byte string
derive	compute a key from other key material with a KDF
authenticate	prove the origin and the integrity of data
negotiate	agree a version or a capability set
serialise, parse	write, or read, the wire encoding
publish, subscribe	send to, or register interest at, a relay
rotate	replace a key and keep the identifier
revoke	declare a key invalid from a stated point
delegate	give a capability to another key
ratchet	advance a key chain in one direction only
commit	apply a group state change at an epoch boundary
zeroize	overwrite key material in memory with zero bytes

Table 5. Technical verbs.

1.7.2 Technical names

Name	Definition
Identity	The principal that an identity log defines.
SunSpot identifier (SID)	The 32-byte BLAKE3 hash of the genesis entry of an identity log.
Identity log	The signed, hash-linked, append-only history of an identity.
Genesis entry	Entry 0 of an identity log.
Root key	The Ed25519 key pair that signs identity log entries.
Recovery key	An offline Ed25519 key pair that can replace a lost root key.
Device	One installation that holds keys for one identity.
Device key	The Ed25519 signature key of a device.
Link key	The X25519 static key of a device.
Prekey	A one-time X25519 key and a one-time ML-KEM-768 key, used once.

Name	Definition
Contact code	A token carrying a SID, a relay set, and an introduction mailbox.
Mailbox	A destination at a relay, with an address and a deposit key.
Mailbox address	A 32-byte opaque identifier of a mailbox.
Deposit key	The Ed25519 key pair that authorises a deposit into a mailbox.
Envelope	The outer object that a relay stores. It holds ciphertext.
Message	The end-to-end object inside an envelope.
Blob	A content-addressed attachment body, split into 64 KiB chunks.
Relay	A store-and-forward node.
Relay set	The ordered list of relays that serve one device.
Session	The double ratchet state between two devices.
Group	A closed set of identities that share a TreeKEM tree.
Epoch	The interval between two group commits.
Channel	A one-to-many stream with one publisher.
Wire encoding (SWE)	The canonical binary encoding of 3.5.4.

Table 6. Technical names.

1.7.3 Security terms

Forward secrecy: an adversary holding current keys cannot read past messages. *Post-compromise security*: once an adversary loses access, the parties heal the session and the adversary loses the ability to read new messages. *Deniability*: a recipient cannot prove to a third party that a given sender wrote a given message. *Unlinkability*: an observer cannot tell that two events relate to the same principal. *Fork*: two valid identity log entries that share a parent; always a security event.

1.8 Conventions

1.8.1 Normative keywords

MUST, MUST NOT, REQUIRED, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL carry the meanings of RFC 2119 and RFC 8174, and are normative only where they appear in small capitals, as here.

1.8.2 Requirement identifiers

A requirement identifier has the form `SSP-CAT-NNN`: the protocol prefix, a three-letter category, and a serial number within that category. Identifiers are permanent. A withdrawn requirement keeps its identifier and gains the text *Withdrawn*, so that the traceability matrix stays stable across revisions. Each requirement holds exactly one testable statement; a requirement holding two is a defect and a reviewer should report it.

1.8.3 Verification methods

Inspection: a reviewer reads the code, the document, or the object. *Analysis*: a reviewer uses a model, a proof, or a calculation. *Demonstration*: an operator runs the system and observes the behaviour. *Test*: an automated case gives a pass or a fail against a known vector.

Code	Category	Code	Category
GEN	General and governance	MDR	Metadata resistance
IDN	Identity	SEC	Security attribute
DEV	Device	PRV	Privacy attribute
KEY	Key management	PRF	Performance
CRY	Cryptography	USE	Usability
NET	Networking and transport	IFC	Interface
RLY	Relay	CON	Design constraint
MSG	Message transport	EXT	Extensibility and version
GRP	Group and channel	ATT	Attachment
SYN	Synchronisation	STO	Storage
ABU	Abuse prevention		

Table 7. Requirement categories.

1.8.4 Notation

All integers on the wire are unsigned and big-endian; there are no variable-length integers. `u8` to `u64` denote unsigned integers of that width, and $[N]$ denotes a fixed field of N bytes. Concatenation is written $\|$. $H(x)$ is BLAKE3-256 of x with a 32-byte output, $\mathrm{KH}(k, x)$ is BLAKE3 in keyed mode with the 32-byte key k, and $\mathrm{KDF}(k, \texttt{label}, n)$ is BLAKE3 in keyed mode with k over the label bytes with an n-byte extendable output. Time values are `u64` milliseconds since the UNIX epoch, in UTC. A byte is eight bits; KiB is 1024 bytes and MiB is 1 048 576 bytes.

2 References

2.1 Normative references

An implementation conforms to every document in Table 8.

Table 8. Normative references.

Tag	Reference
RFC 2119	Bradner, S. *Key words for use in RFCs to indicate requirement levels*. BCP 14, 1997.
RFC 8174	Leiba, B. *Ambiguity of uppercase vs lowercase in RFC 2119 key words*. 2017.
RFC 7748	Langley, A., Hamburg, M., Turner, S. *Elliptic curves for security*. 2016. X25519.
RFC 8032	Josefsson, S., Liusvaara, I. *Edwards-curve digital signature algorithm*. 2017. Ed25519.
RFC 8439	Nir, Y., Langley, A. *ChaCha20 and Poly1305 for IETF protocols*. 2018.
RFC 9106	Biryukov, A., et al. *Argon2 memory-hard function*. 2021.
RFC 9000	Iyengar, J., Thomson, M. *QUIC: a UDP-based multiplexed and secure transport*. 2021.
RFC 9001	Thomson, M., Turner, S. *Using TLS to secure QUIC*. 2021.
RFC 9221	Pauly, T., Kinnear, E., Schinazi, D. *An unreliable datagram extension to QUIC*. 2022.
RFC 7250	Wouters, P., et al. *Using raw public keys in TLS and DTLS*. 2014.

Tag	Reference
RFC 9420	Barnes, R., et al. *The messaging layer security protocol*. 2023. TreeKEM only.
FIPS 203	NIST. *Module-lattice-based key-encapsulation mechanism standard*. 2024. ML-KEM-768.
BLAKE3	O'Connor, J., Aumasson, J.-P., Neves, S., Wilcox-O'Hearn, Z. *BLAKE3: one function, fast everywhere*. 2020.
Noise	Perrin, T. *The Noise protocol framework*, revision 34. 2018.
BIP-350	*Bech32m format for v1+ witness addresses*. 2021.
Equi-X	The Tor Project. *Equi-X proof-of-work scheme*, Tor proposal 327. 2023.
Bao	O'Connor, J. *Bao: verified streaming for BLAKE3*. 2020.

2.2 Informative references

Table 9. Informative references.

Tag	Reference
Double Ratchet	Marlinspike, M., Perrin, T. *The double ratchet algorithm*. Signal, 2016.
X3DH	Marlinspike, M., Perrin, T. *The X3DH key agreement protocol*. Signal, 2016.
PQXDH	Kret, E., Schmidt, R. *The PQXDH key agreement protocol*. Signal, 2023.
Sealed sender	Signal. *Sealed sender*. 2018.
WireGuard	Donenfeld, J. *WireGuard: next generation kernel network tunnel*. NDSS, 2017.
SimpleX	SimpleX Chat. *SimpleX messaging protocol*. 2023.
Briar	Briar Project. *Bramble transport protocol and design documents*. 2020.
Cabal	Cabal Club. *Cabal protocol specification*. 2020.
SSB	Tarr, D., et al. *Secure Scuttlebutt protocol guide*. 2019.
Session	Oxen. *Session protocol whitepaper*. 2020.
Matrix	The Matrix.org Foundation. *Matrix specification*. 2024.
Tor	Dingledine, R., Mathewson, N., Syverson, P. *Tor: the second-generation onion router*. 2004.
Monero	Monero Research Lab. *MRL bulletins and technical notes*. 2014–2024.
Loopix	Piotrowska, A., et al. *The Loopix anonymity system*. USENIX Security, 2017.
ZIP 215	Hopwood, D. *Explicitly defining and modifying Ed25519 validation rules*. 2020.
RFC 9180	Barnes, R., et al. *Hybrid public key encryption*. 2022.

3 Requirements

3.1 System context and stakeholders

Figure 2 shows the parties and the one deliberate absence: there is no relay-to-relay protocol. Section 3.8.10 defends that.

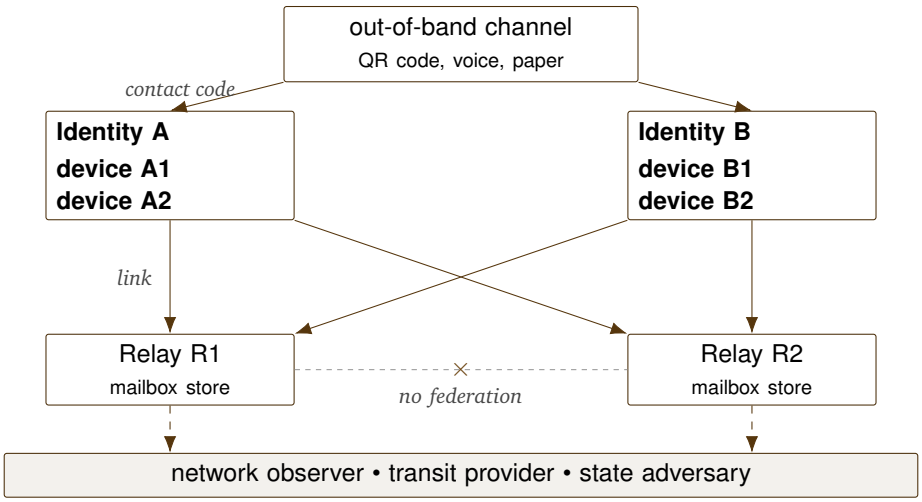

Figure 2. System context.

Stakeholder	Primary concern	Answered in
User at risk	Confidentiality, metadata, deniability	3.7, 3.11
Ordinary user	Reliable delivery, multi-device, low battery cost	3.9, 3.12
Relay operator	Bounded cost, no liability, simple operation	3.8.8, 3.10
Implementer	Complete rules and test vectors	3.5, 4
Maintainer	Long-term stability, safe extension	3.17
Reviewer	Explicit threat model and stated limits	3.2, 3.11.6

Table 10. Stakeholder concerns.

3.2 Threat model

3.2.1 Assets

The protocol protects these assets in this order of priority:

1. Message content.
2. The social graph: the set of pairs that communicate.
3. Group membership.
4. The activity pattern of a user over time.
5. The long-term private keys of an identity.
6. The availability of delivery.

The order is operative. Where a design choice trades availability against content or the social graph, content and the social graph win and availability is allowed to suffer. Section 3.10 applies this rule to spam defence.

3.2.2 Adversary classes

Table 11. Adversary classes.

Ref	Adversary	Capability
T1	Passive local observer	Reads packets on one link: addresses, times, sizes.
T2	Active local observer	Drops, delays, injects, and replays on one link.
T3	Honest-but-curious relay	Reads all state it stores. Follows the protocol.
T4	Malicious relay	Drops, reorders, replays, lies, serves a forked view.
T5	Relay coalition	Shares logs across every relay it runs.
T6	Global passive adversary	Observes every link at the same time.
T7	Compromised device	Holds current keys and plaintext of one device.
T8	Malicious contact	An authorised peer that copies and republishes.
T9	Malicious group member	Inside a group; sees membership and content.
T10	Sybil operator	Creates unlimited identities and relays at low cost.
T11	Botnet	10^5 to 10^7 hosts with distinct addresses.
T12	State adversary	T2, T5, T6, plus legal power and border filtering.
T13	Supply-chain adversary	Ships a modified client to a subset of users.
T14	Future quantum adversary	Records traffic now, breaks X25519 later.

3.2.3 What each adversary obtains

Asset	T1	T3	T4	T5	T6	T7	T9	T12
Message content	–	–	–	–	–	yes	yes	–
Social graph	partial	–	–	partial	partial	yes	–	partial
Group membership	–	–	–	–	–	yes	yes	–
Activity pattern	yes	yes	yes	yes	yes	yes	partial	yes
Long-term keys	–	–	–	–	–	yes	–	–
Recovery denial	–	–	–	–	–	partial	–	–
Availability	delay	yes	yes	yes	–	yes	–	yes

Table 12. Adversary capability against SSP/1 as specified. T7 and T9 obtain content only for the device or the group they are inside. A compromised device can delay recovery once, by a single contest, and cannot deny it: see 3.6.11.

The partial results need explanation. T1 sees that a client talks to a relay, and not which mailboxes it reads, because the mailbox list travels inside the encrypted link. T5 sees that one client subscribes to a set of mailbox addresses across the relays it runs; 3.11.4 bounds that with a sharding rule. T6 correlates a send at one client with a fetch at another, which is a real attack that SSP does not stop. T12 can block SSP at a border, and 3.8.3 raises the cost without guaranteeing reachability.

3.2.4 Threats in scope

Table 13. Threats the protocol defends against.

Ref	Threat	Defence
TH-01	Read of content by a relay or an observer	3.7, 3.9.1
TH-02	Retrospective decryption after a key leak	3.9.2
TH-03	Retrospective decryption by a quantum adversary	3.7.12
TH-04	Impersonation of an identity	3.6.2
TH-05	Silent addition of a device by an attacker	3.6.6
TH-06	Silent recovery takeover with a stolen recovery key	3.6.11
TH-07	Replay of an envelope	3.5.6
TH-08	Reorder or drop by a relay	3.9.4
TH-09	A relay serves a forked identity log	3.6.12
TH-10	Enumeration of users	3.8.9
TH-11	Correlation of a mailbox address to an identity	3.8.6
TH-12	Size and count analysis at a relay	3.11.2
TH-13	Spam to an unknown recipient	3.10.2
TH-14	Flood of a known mailbox	3.10.1
TH-15	Resource exhaustion of a relay	3.10.3
TH-16	Sybil creation of identities to gain trust	3.10.4
TH-17	Group membership leak to a relay	3.9.6
TH-18	A removed member reads new messages	3.9.6
TH-19	A parsing defect that a hostile object triggers	3.5.4
TH-20	Downgrade of the negotiated version set	3.17.2
TH-21	Denial of legitimate recovery by a compromised device	3.6.11

Ref	Threat	Reason

3.2.5 Threats out of scope

This list is normative. An implementation MUST NOT claim a defence that it excludes.

Table 14. Threats the protocol does not address.

Ref	Threat	Reason
TX-01	Traffic confirmation by a global passive adversary	A defence needs constant-rate cover traffic on every device at all times. The battery and bandwidth cost make that unusable, and a partial defence gives false confidence. 3.11.5 offers an opt-in mode.
TX-02	Plaintext capture on a device an adversary controls	No protocol can stop an endpoint reading its own plaintext.
TX-03	Republication by an authorised recipient	The recipient must be able to read the content.
TX-04	Coercion of a user to unlock a device	The protocol offers crypto-erase of old keys and nothing more. Duress modes make a promise they cannot keep.
TX-05	A modified client the user installed	Reproducible builds and signing are a distribution problem.
TX-06	A state that blocks all internet access	Beyond the reach of any protocol.
TX-07	Fingerprinting by local clock, fonts, or CPU timing	A client fingerprint problem. 3.11.4 covers the protocol-level rules only.
TX-08	Analysis of the size of a very large attachment	Chunks are padded, and the chunk count still leaks an approximate size.
TX-09	Anonymity of the sender towards the recipient	SSP authenticates senders to recipients deliberately. Anonymous messages are a spam vector.
TX-10	Proof that a message was not delivered	Non-repudiation of delivery needs a trusted third party.

3.2.6 Trust assumptions

SSP-SEC-001 A client MUST treat every relay as a member of adversary class T4.

Rationale. A design that assumes an honest relay fails the moment an operator changes or a host provider seizes a disk.
Verification. Analysis of every relay operation in 3.8.4 against class T4, plus the hostile-relay cases of 4.5.

Party	Trusted for	Not trusted for
Relay	Availability of the data it holds	Confidentiality, integrity, ordering, honesty, existence
Contact device	Content it sends, its own key state	The keys of any other identity
Out-of-band channel	Integrity of a contact code at first contact	Confidentiality of that code
Operating system	Process isolation and the key store	Confidentiality of past traffic
Group member	Nothing beyond current-epoch content	Membership honesty, forward silence

Table 15. Trust assumptions.

3.3 Design goals

Table 16. Design goals.

Ref	Goal	Measure of success
G-01	Implementation independence	Two clients written from this text, by teams with no contact, interoperate.
G-02	Confidentiality of content	Content is readable only by the intended devices.
G-03	Forward secrecy and post-compromise security	A key leak exposes a bounded window and the session heals.
G-04	Metadata minimisation at relays	A relay holds no identity, no name, no graph.
G-05	Stable identity with rotatable keys	One identifier survives full key replacement.
G-06	Multi-device with no primary device	Any device can act alone and can add or remove another.
G-07	Decentralisation with no privileged node	Anyone can run a relay; no node has a special role.
G-08	Abuse defence from local state	One relay enforces quotas with no global view and no identity.
G-09	Ten-year stability	Core objects and frames do not change for the life of SSP/1.
G-10	Post-quantum confidentiality	Recorded traffic stays private against a later quantum adversary.
G-11	Bounded relay cost	Storage per mailbox and per epoch has a hard limit.
G-12	Small implementable core	The protocol core builds without transport, storage, or user interface code; each state machine is testable in isolation; the parsing path allocates nothing; dependency boundaries are explicit. Size and throughput are measured once prototypes exist, not asserted here.
G-13	Honest documentation of limits	Every privacy claim carries a stated boundary.

SSP-GEN-004 The protocol MUST reach G-01 without a shared library.

Rationale. A shared library that every client must use is the reference client under another name, and it hides protocol rules in code.
Verification. Demonstration, once a second implementation exists: two teams working only from this document and the vector corpus of Appendix E complete the mandatory function set against each other.

3.4 Non-goals

Recorded so that a later revision cannot add them by accident.

Table 17. Non-goals.

Ref	Non-goal	Reason
N-01	Anonymity of the sender to the recipient	Contradicts the spam model of 3.10.
N-02	Global message search across relays	Needs plaintext or a searchable index at the relay.
N-03	Public discoverability of users	Every discovery mechanism is an enumeration mechanism.
N-04	Server-side history for new devices	Needs long-term ciphertext under a long-term key, which breaks forward secrecy.
N-05	Federation between relays	Adds a trust relationship, a routing table, and an amplification vector.
N-06	Consensus or a ledger	Needs global state, and global state is a central point.
N-07	Moderation of content by a relay	A relay cannot read content by design.
N-08	Compatibility with pre-SSP/1 builds	3.17.3.
N-09	Rooms above 1024 members with hidden membership	3.9.8 offers channels for the one-to-many case.
N-10	Cipher suite negotiation inside a session	3.7.1.

3.5 Protocol architecture

3.5.1 Layer model

SSP has six layers plus an identity substrate. Each layer has one duty, and no layer reads the payload of the layer above it.

SSP-GEN-005 Each layer SHOULD be implementable and testable with a stub for the layer below it.

Rationale. A layer that needs a live socket to exercise its state machine cannot be tested against vectors, and becomes the place defects accumulate. This constrains implementation structure

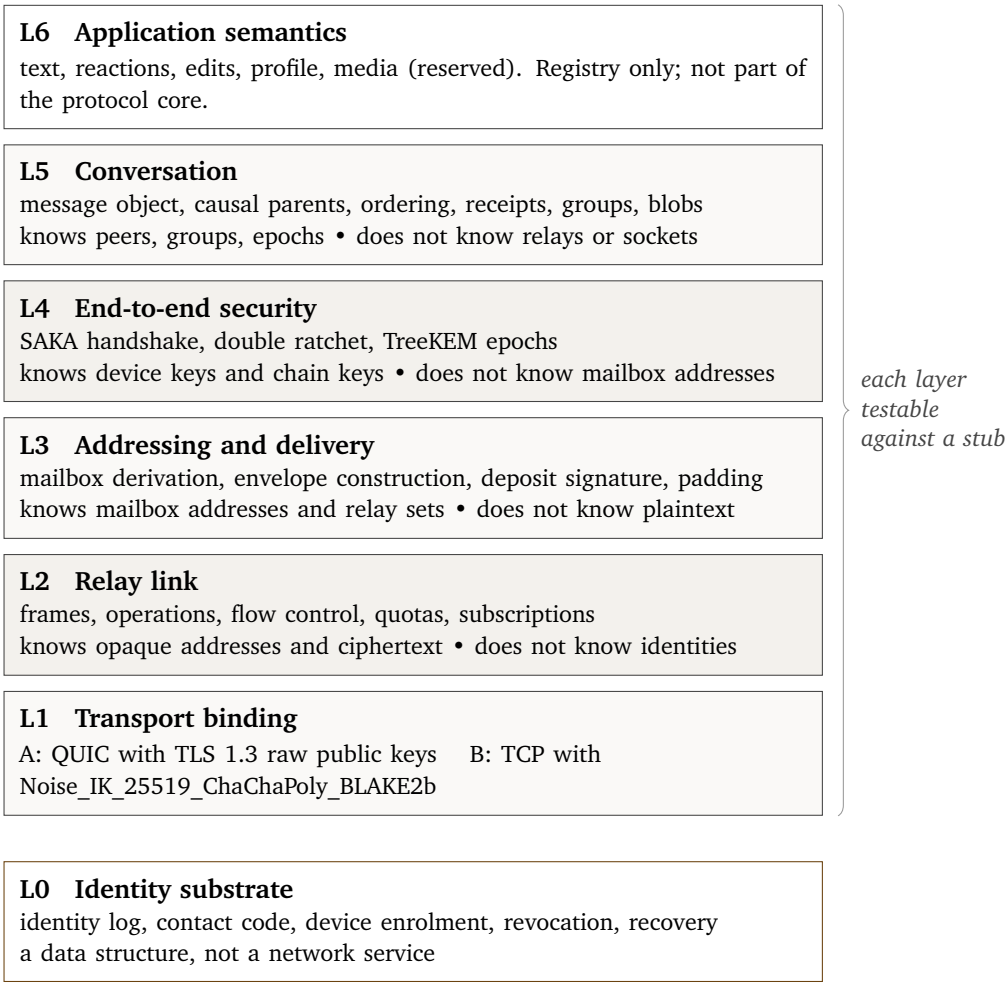

Figure 3. Layer model. L0 sits beneath the stack and beside it: an identity log carries its own proof and can arrive over any channel.

rather than the wire.
Verification. Inspection of the crate boundaries in 5.2, plus a test run of the session and group state machines against recorded byte sequences.

SSP-GEN-006 L2 MUST NOT parse, interpret, or act on any byte of the L3 payload beyond the fields that 3.5.5 defines.
Rationale. The mechanical form of "a relay is furniture".
Verification. Inspection of the relay code path, plus a fuzz run feeding random L3 payloads: the relay's behaviour depends only on the fields 3.5.5 defines.

3.5.2 Delivery of one message

Figure 4 traces one text message from device A1 to identity B. Only steps 5 and 9 are visible to the relay, which sees an opaque address, a padded ciphertext, a signature, and a time.

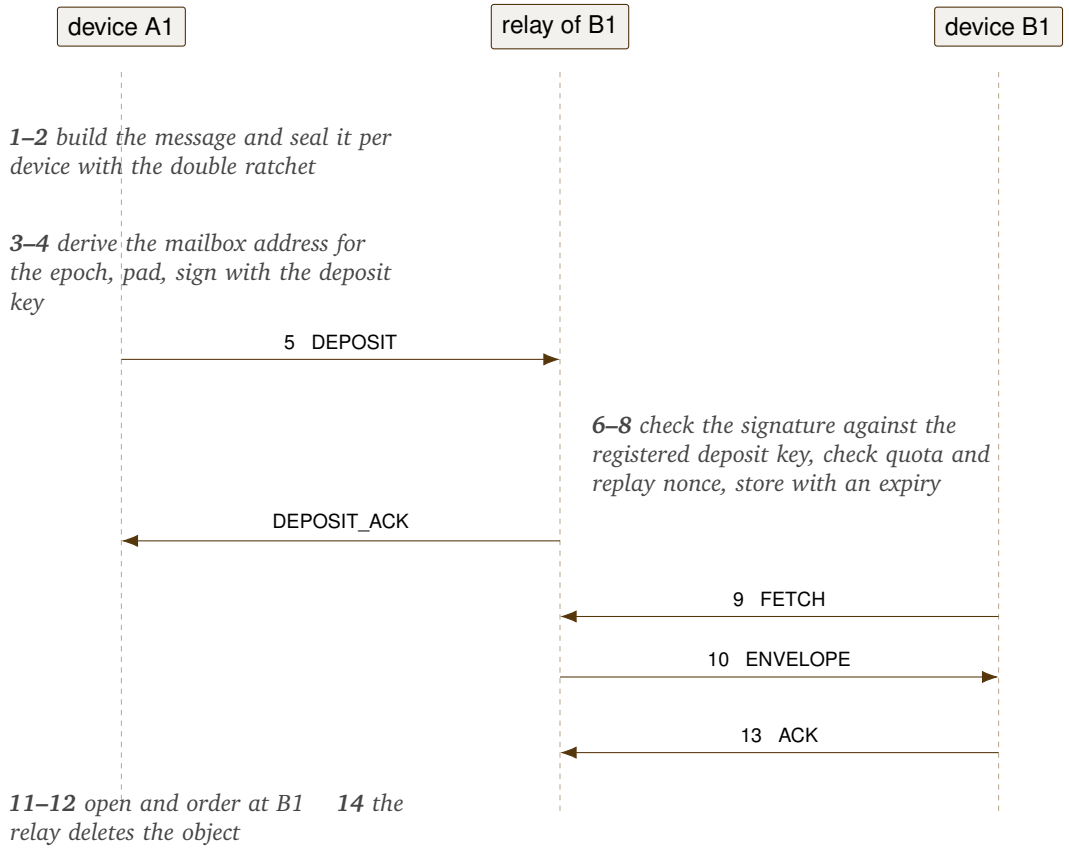

Figure 4. Delivery path for one message.

Delivery to the second device of B repeats steps 3 to 14 with a different mailbox and the relay set of that device; the two mailboxes are unlinkable. Delivery to the second device of the sender takes the same path, through the self group of 3.9.10. There is no special case for a sender that syncs to itself.

3.5.3 Object model

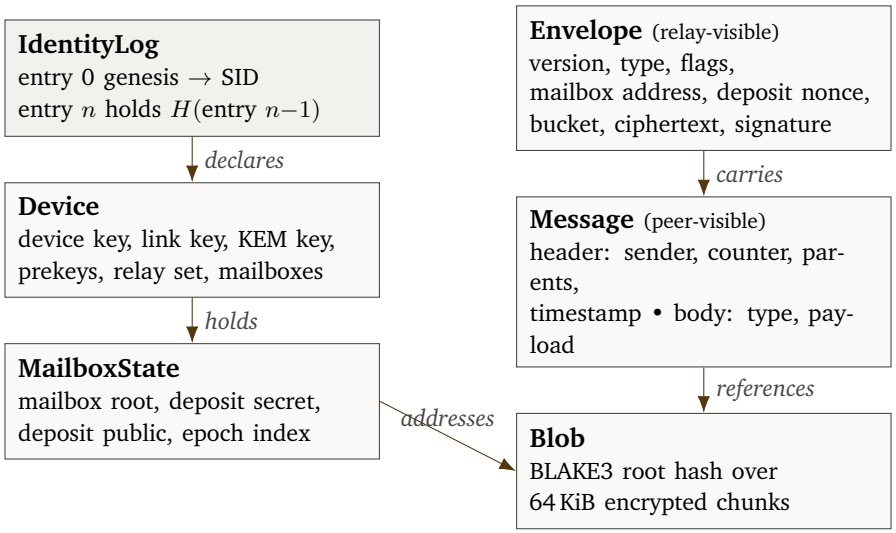

Figure 5. Object model.

SSP-GEN-007 Every object that a party signs MUST begin with a domain separation label, the SSP version, and a context identifier, before any other field.

Rationale. Prevents a signature made in one context from verifying in another. This class of defect has broken several deployed protocols.

Verification. Test: a signature produced over an identity log entry does not verify when the same bytes are presented as a message, and the reverse.

3.5.4 Wire encoding

The SunSpot wire encoding is canonical, binary, and not self-describing. The schema lives in this document; the bytes carry no field names and no type tags except at a declared extension point.

1. Integers are unsigned and big-endian. There are no variable-length integers.
2. A fixed-length field carries no length prefix.
3. A variable-length field carries a `u32` byte-count prefix.
4. A list carries a `u16` count, then the elements in order.
5. There is no padding between fields and no alignment rule.
6. An optional field is present according to a bit in a flags field. Present optional fields appear in the bit order of that flags field, from the least significant bit upward.
7. An extension is a `(u16 type, u32 length, bytes)` triple inside an extension list. Bit 15 of the type is the critical bit.
8. A parser rejects any encoding that breaks a rule above, and rejects any trailing byte after the last field.

SSP-CON-001 An implementation MUST use this encoding for every byte that crosses the network or that enters a hash or a signature.

Rationale. One encoding for one purpose. A second encoding means two parsers and two attack

surfaces.
Verification. Inspection of every encode and decode call site, plus a test that no other serialisation library is reachable from the core.

SSP-CON-002 A parser MUST reject a non-canonical encoding and MUST NOT normalise it.
Rationale. A parser that repairs input makes the hash of an object depend on the parser, which breaks content addressing.
Verification. Test against the planned malformed corpus: each non-canonical encoding is rejected with its expected error code and no normalised value is produced.

SSP-CON-003 A parser MUST check every length field against the hard limit in Appendix B before it allocates memory.
Rationale. A length field is the classic remote allocation attack.
Verification. Test: a frame declaring a length above the limit for its operation is refused before allocation, verified by an allocator counter that stays flat.

Rejected encodings.

Table 18. Serialisation formats considered and rejected.

Candidate	Reason for rejection
Protocol Buffers	Not canonical. Varints have several valid forms. Unknown fields survive a round trip and change the bytes a signature covers. Presence rules changed between proto2 and proto3, and the ambiguity has produced real security defects elsewhere.
CBOR	The base format is permissive. Canonical CBOR exists, but a parser must reject a great deal of valid CBOR to reach it, and most libraries do not. The self-describing tags add a type-confusion surface SSP does not need.
JSON	Large, no byte type, no canonical number form, costly parser. Text formats invite Unicode normalisation attacks inside signed data.
MessagePack	The same non-canonical problem as CBOR with fewer strict parsers.
bincode, postcard, serde-derived formats	The wire format follows the Rust type layout and the derive macro, which ties the protocol to one language and one library version. A field reorder in the source silently changes the wire.
ASN.1 DER	Canonical, and battle-tested in the negative sense. Parser complexity is very high and the defect history is long.
Cap'n Proto, FlatBuffers	Good zero-copy properties, no canonical form. The pointer layout hands an attacker aliasing tricks inside one buffer.

The cost is a hand-written parser. The return is one valid encoding per object, zero-copy parsing over a borrowed slice, no code generation, and no dependency that can change the wire format in a minor release.

3.5.5 Relay link frame

Every transport binding carries the same frame.

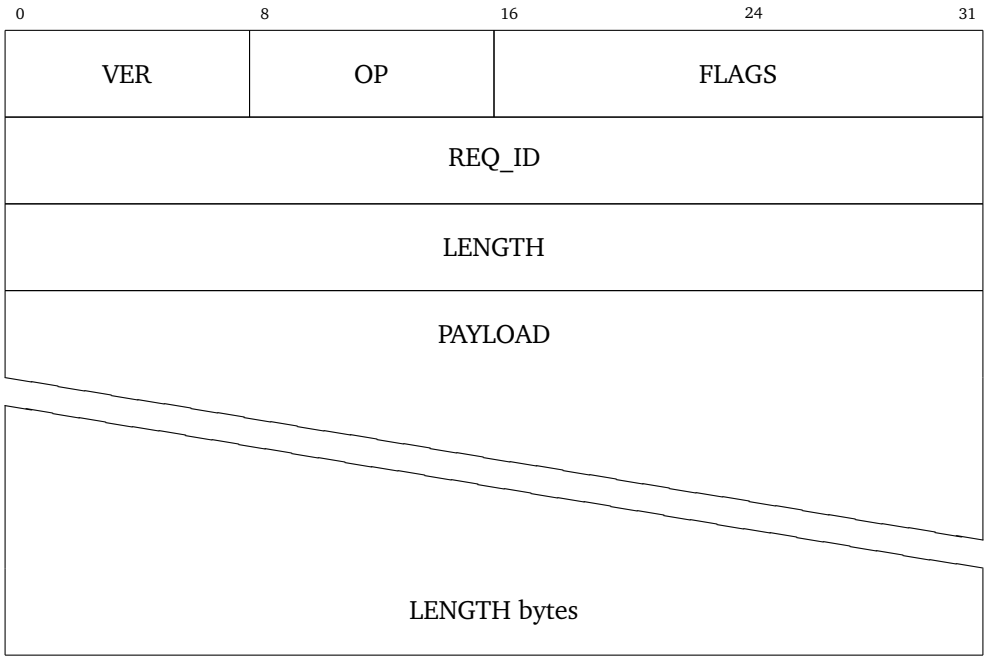

Figure 6. L2 frame.

Field	Width	Rule
VER	u8	Frame version, `0x01` for SSP/1. A relay closes the link on an unknown value.
OP	u8	Operation code; registry in Appendix C.
FLAGS	u16	Bit 0 response. Bit 1 more frames follow for this REQ_ID. Bit 2 padding frame, which the receiver discards. Bits 3–15 reserved and zero.
REQ_ID	u32	Chosen by the client and repeated in the response. Zero means no response wanted.
LENGTH	u32	Payload length. At most 1 048 576 bytes for a frame carrying an envelope, 65 536 otherwise.

Table 19. Frame header fields.

SSP-NET-001 A relay MUST close the link when a frame declares a LENGTH above the limit for its OP value.

Rationale. The limit has to apply before the relay reads the body, so that one frame cannot exhaust memory.
Verification. Test: a frame declaring a length above its operation limit closes the link, and the relay's resident memory does not rise during the attempt.

SSP-NET-002 A client and a relay MUST accept frames for one REQ_ID out of order with respect to other REQ_ID values.

Rationale. QUIC streams complete in any order, and a strict global order would force the head-of-line blocking that QUIC exists to remove.
Verification. Test: two requests are issued and answered in reverse order; both complete and neither blocks the other.

Table 20 summarises the operations; Appendix C gives the payload layouts.

Table 20. Relay link operations.

Code	Name	Direction	Purpose
0x01	HELLO	client → relay	Version and capability offer.
0x02	HELLO_ACK	relay → client	Selection, policy, limits.
0x10	MB_REGISTER	client → relay	Create a mailbox with a deposit key and a quota.
0x11	MB_RENEW	client → relay	Extend the life of a mailbox.
0x12	MB_DELETE	client → relay	Delete a mailbox and its content.
0x13	MB_STATUS	client → relay	Report count, bytes, quota state.
0x20	DEPOSIT	sender → relay	Store one envelope.
0x21	DEPOSIT_ACK	relay → sender	Accept, with the assigned sequence.
0x22	DEPOSIT_NAK	relay → sender	Reject, with a reason and any required effort.
0x30	SUBSCRIBE	client → relay	Watch a set of mailboxes.
0x31	UNSUBSCRIBE	client → relay	Stop watching.
0x32	FETCH	client → relay	Read objects after a cursor.
0x33	ENVELOPE	relay → client	Deliver one stored envelope.
0x34	ACK	client → relay	Confirm receipt so the relay can delete.
0x40	BLOB_PUT	client → relay	Store one chunk group.
0x41	BLOB_GET	client → relay	Read one chunk group.
0x50	POW_CHALLENGE	relay → client	Issue an Equi-X challenge.
0x51	POW_SOLUTION	client → relay	Supply the solution.
0x60	PAD	either	A frame carrying only padding.
0x7F	ERROR	either	A terminal error with a code from Appendix C.

3.5.6 Envelope

Field	Rule
EVER	`0x01` for SSP/1.
ETYPE	`0x01` message, `0x02` introduction request, `0x03` group commit, `0x04` blob notice, `0x05` receipt only.
EFLAGS	Bit 0 the sender wants an acknowledgement. Bit 1 urgent, and the relay should push it to a subscriber first. Bits 2–15 reserved and zero.
MAILBOX_ADDRESS	The output of 3.8.6.
DEPOSIT_NONCE	Sixteen random bytes, held in the replay cache for the life of the object.
BUCKET	The padding bucket index of 3.11.2.
CIPHERTEXT	The L4 output, padded to the bucket size.
DEPOSIT_SIGNATURE	Ed25519 over `SSP1/deposit` ‖ `0x00` ‖ EVER ‖ ETYPE ‖ MAILBOX_ADDRESS ‖ DEPOSIT_NONCE ‖ BUCKET ‖ H(ciphertext).

Table 21. Envelope fields.

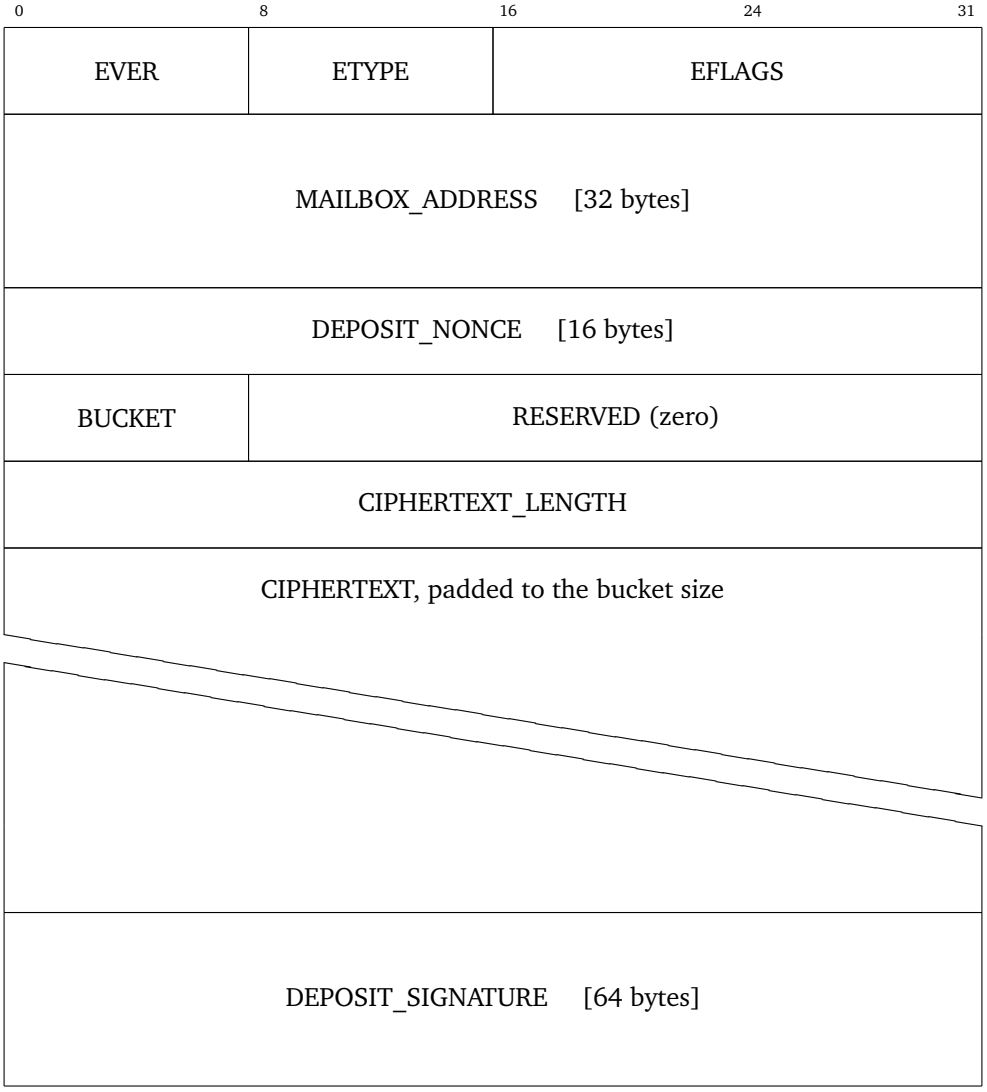

Figure 7. Envelope, the unit a relay stores.

ETYPE is visible to the relay: a deliberate leak of five values, needed for the quota class of 3.10.1 and the expiry class of 3.8.8. Section 3.11.3 states the cost.

SSP-MSG-001 A relay MUST reject a deposit whose signature does not verify against the registered deposit key of the mailbox.

Rationale. This single check carries the whole abuse model of 3.10.
Verification. Test: deposits signed by an unrelated key, by a key valid for a different mailbox, and with a corrupted signature are each refused with `BAD_SIGNATURE`.

SSP-MSG-002 A relay MUST reject a deposit whose nonce is already in the replay cache of that mailbox.

Rationale. Stops replay of ciphertext by an observer and stops quota waste.
Verification. Test: a captured deposit replayed within the object lifetime is refused with `REPLAY`, and the original object is unaffected.

SSP-MSG-003 The ciphertext length MUST equal the size of the bucket that BUCKET names.

Rationale. A mismatch would let a sender signal information in the difference between the declared and the real length.
Verification. Test: an envelope whose declared bucket does not match its ciphertext length is refused, for each of the six bucket values.

3.6 Identity, devices, and key management

3.6.1 Identity model

An identity is not a name and not a public key. It is an append-only, hash-linked, self-authenticating log. Its identifier is the hash of the first entry of that log:

$$\mathrm{SID} \;=\; \mathsf{BLAKE3\text{-}256}\big(\mathsf{SWE}(\mathrm{Entry}_0)\big) \qquad 32 \text{ bytes.}$$

The log holds everything another party needs in order to authenticate the identity: the current root key, the device set, revocations, the recovery set, and relay hints.

Model	Example	Failure
Identity is a raw public key	Session, Briar, SSB	The identifier cannot survive key rotation. A leaked key forces a new identity, and every contact must authenticate again out of band.
Identity is a server account	Signal, Matrix, XMPP	Rotation works and the server becomes the root of trust. The server can add a device.
Identity is a log	**SSP**	The identifier is stable, rotation works, no server is trusted. The cost: a client must fetch and check a log.

Table 22. Identity models.

The cost of the log model is real: a client stores a log per contact, checks every signature in it, and must detect a fork (3.6.12). It is bounded, because a normal log holds fewer than fifty entries in ten years.

SSP-IDN-001 A SunSpot identifier MUST be the BLAKE3-256 hash of the canonical encoding of the genesis entry of its identity log.

Rationale. The identifier binds to the initial state, and the hash chain binds every later state to that identifier.
Verification. Test against the planned identity vectors: the identifier equals the hash of the canonical genesis encoding, and a one-byte change to any genesis field changes it.

SSP-IDN-002 A SunSpot identifier MUST NOT change for any reason, including full rotation of every key of the identity.

Rationale. G-05. A stable name is what lets a user keep a contact list after a compromise.
Verification. Test: a log with a full root rotation and a complete device replacement yields the same identifier as its genesis.

SSP-IDN-003 The protocol MUST NOT carry a human-readable name, a phone number, an email address, or any other external identifier inside an identity log.

Rationale. The log is public to every contact and to every relay that stores a copy. A name in the log is a permanent, non-revocable leak. Display names are per-contact application values, synchronised end-to-end by 3.9.10.
Verification. Inspection of the entry schema: no entry body carries a free-text or externally issued identifier field.

3.6.2 Log structure

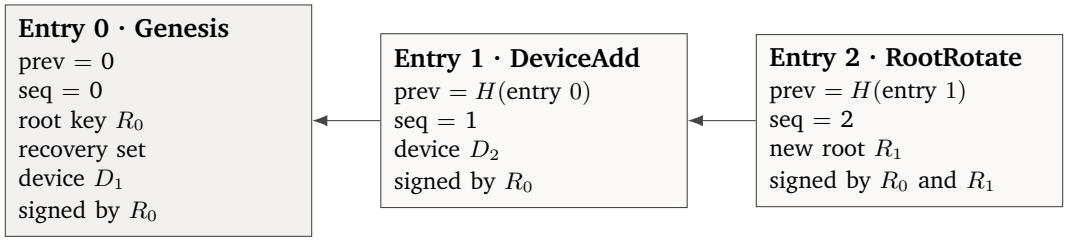

Figure 8. Identity log. Each entry names the hash of its predecessor, so the SID commits to the whole history.

Every entry carries the header of Table 23.

SSP-IDN-004 A verifier MUST check entries in order from sequence 0, and MUST reject the whole log when any entry fails a check.

Rationale. Partial acceptance would let an attacker graft a valid tail onto an invalid trunk.
Verification. Test: a log whose seventh entry has a broken hash link is rejected in whole, and no prefix of it is accepted.

SSP-IDN-005 A verifier MUST reject an entry whose time value is more than 86 400 000 ms after the time value of the entry that follows it.

Rationale. The log is not a clock and the writer is not trusted. The rule catches gross reordering

Field	Type	Rule
label	[12]	The ASCII bytes `SSP1/idlog` with two zero bytes.
version	u8	`0x01`.
entry_type	u8	See Table 25.
seq	u32	Zero for genesis, then plus one per entry.
prev	[32]	Zero for genesis, then the hash of the previous entry.
time	u64	Milliseconds since the UNIX epoch. Advisory.
not_before	u64	When the entry takes effect; used by 3.6.11.
body	var	Type-specific.
signatures	list	One or more `(key_id [8], sig [64])` pairs.

Table 23. Entry header.

without a timestamp service.
Verification. Test: an entry timestamped more than 24 hours after its successor is rejected, and one inside that tolerance is accepted.

SSP-IDN-006 A verifier MUST reject an entry whose sequence is not exactly one more than its predecessor.

Rationale. A gap or a repeat in the sequence means an entry is missing or duplicated, and a verifier that tolerates either cannot say which log it has validated.
Verification. Test: logs with sequences 0,1,3 and 0,1,1 are both rejected; 0,1,2 is accepted.

Entry type	Required signatures
Genesis	The root key R_0.
DeviceAdd	The current root key, or a device key holding the enrol capability.
DeviceRemove	The current root key, or any current device key.
RootRotate	The old root key and the new root key.
RecoverySetUpdate	The current root key and a claim-threshold quorum of the current recovery set.
RecoveryClaim	A claim-threshold quorum (m-of-n) of the recovery policy at `head_ref`.
RecoveryContest	Any device key active at `head_ref`, or the root key in force there. Contests, does not reject.
RecoveryReject	A rejection-threshold quorum (r-of-n) of the recovery policy at `head_ref`.
DeviceRemoveReject	Any current device key other than the signer of the pending removal.
RelaySetUpdate	Any current device key.
Extension	As the extension registry states.

Table 24. Signature requirements per entry type.

Signature rules. DeviceRemove ordinarily needs only one device key: a user whose device is stolen must be able to remove it from any other device without the root key, which may be offline. The risk is that a compromised device removes its siblings and becomes the only active device. 3.6.7 bounds that with a pending state for any removal that would leave fewer than two active devices, and 3.6.11 guarantees that a sole compromised device still cannot block recovery.

3.6.3 Entry types

Code	Name	Body
0x01	Genesis	root key, recovery policy, recovery set, first device record
0x02	DeviceAdd	device record
0x03	DeviceRemove	device id, reason, revoke_from
0x04	RootRotate	new root key, reason
0x05	RecoverySetUpdate	recovery policy, recovery set
0x06	RecoveryClaim	head_ref, new root key, new device record
0x07	RecoveryContest	claim hash, reason
0x08	RelaySetUpdate	device id, relay records
0x09	PrekeyPointer	device id, relay records for the prekey mailbox
0x0A	Close	reason; the identity accepts no further entries
0x0B	RecoveryReject	claim hash
0x0C	DeviceRemoveReject	removal entry hash
0x0D–0x7F	Reserved	
0x80–0xFF	Extension range, critical bit at bit 7	

Table 25. Identity log entry types.

A device record holds the device id (the first eight bytes of $H(\text{device key})$), the Ed25519 device key, the X25519 link key, the 1184-byte ML-KEM-768 key, a capability word, and a timestamp: about 1270 bytes. A log with eight devices is therefore about 10 KiB, which a contact fetches once and afterwards updates by delta.

SSP-IDN-007 A client MUST support an identity log of at least 1024 entries and 256 KiB.

Rationale. A smaller limit would reject a valid long-lived identity and split the network.
Verification. Test: a log of 1024 entries totalling just under 256 KiB verifies; the implementation reports a bounded, non-fatal error beyond those limits rather than truncating.

3.6.4 Address format

A user sees a SID as Bech32m text with the human-readable part `sun`:

```
sun1qw508d6qejxtdg4y5r3zarvary0c5xw7kv8f3t4...
```

SSP-IDN-008 A client MUST use Bech32m with the human-readable part `sun` for the text form of a SID.

Rationale. Bech32m detects every error of up to four characters, is case-insensitive, and encodes into the alphanumeric mode of a QR code, which yields a smaller code than base64. Base58 has no checksum design and no error location; hexadecimal is 64 characters with no error detection at all.
Verification. Test against the planned encoding vectors: the text form round-trips, and the human-readable part is rejected when altered.

SSP-IDN-009 A client MUST reject a SID string with a bad checksum and MUST NOT attempt to correct it.

Rationale. Error correction on a security identifier lets an attacker pick a string that corrects into the identifier of the victim.
Verification. Test: single-character substitutions across 1000 valid strings are all rejected, and no input produces a corrected identifier.

3.6.5 Device model

An identity has one to eight devices. Every device is equal; none is primary and none must be online.

SSP-DEV-001 The protocol MUST NOT define a primary device, or a device on which another device depends for its operation.

Rationale. Signal ties a linked device to a phone, so losing the phone breaks the linked devices. A protocol meant to last a decade cannot carry that dependency.
Verification. Inspection of the enrolment, group, and synchronisation state machines: none has a transition that requires a specific device to be reachable.

SSP-DEV-002 An identity MUST NOT hold more than eight active devices.

Rationale. A sender encrypts one copy per recipient device, so cost is linear in the device count. In a group of 1024 members, eight devices give a worst case of 8192 copies, the largest fan-out 3.12 accepts. A higher limit would let one member impose unbounded cost on every other member.
Verification. Test: a log declaring a ninth active device is rejected; removal of one device permits a subsequent addition.

SSP-DEV-003 Every device MUST hold its own device key, link key, and KEM key pair, and MUST NOT copy a private key to another device.

Rationale. A shared private key removes the ability to revoke one device, and removes the value of the ratchet, because two devices would share one state.
Verification. Demonstration during enrolment: the new device generates its own key pairs and no private key crosses the pairing channel.

3.6.6 Device enrolment

The new device generates its own keys and the existing device signs a DeviceAdd entry. A short secret travels out of band, so an active network attacker cannot insert a device.

SSP-DEV-004 Enrolment MUST use `Noise_IKpsk2_25519_ChaChaPoly_BLAKE2b` with the responder static key and the pre-shared key taken from the out-of-band channel.

Rationale. IK hides the initiator identity from a passive observer and needs one round trip. The psk2 placement binds the transport keys to the physical scan without changing the security properties of the pattern.
Verification. Test against the Noise vectors for the pattern, plus a case where the pre-shared key differs, which fails the handshake.

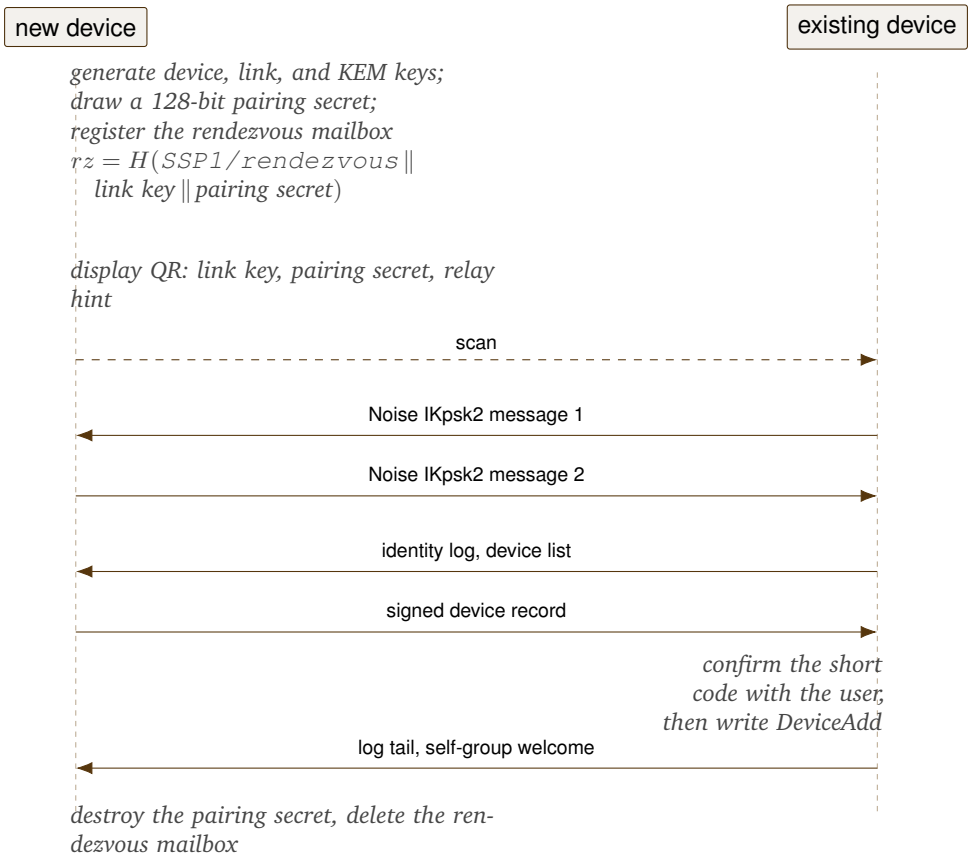

Figure 9. Device enrolment. The initiator is the existing device, which already knows the responder static key because the QR code carries it.

SSP-DEV-005 A client MUST show a five-word or eight-digit confirmation code derived from the handshake hash, and MUST NOT write the DeviceAdd entry before the user confirms it.

Rationale. The psk defends against a network attacker; the code defends against a user who scanned the wrong thing, and makes the event visible.
Verification. Demonstration: the words shown on both devices match for a genuine pairing and differ when a third party substitutes its own static key.

SSP-DEV-006 A client MUST alert the user on every device of the identity when a DeviceAdd entry appears in the log.

Rationale. TH-05. A silent device addition is the classic key-server attack; the log makes it detectable, and detection is worthless if the client hides it.
Verification. Demonstration: a device addition raises an alert on every other device, including one that was offline when the entry was written.

SSP-DEV-007 The pairing secret MUST be at least 128 bits, MUST be used once, and MUST be zeroized after the handshake.

Rationale. The pairing secret is the only value binding the handshake to the physical act of scanning. Reuse would let a party who saw one code join a later pairing, and a short secret is guessable inside the handshake window.
Verification. Test: a handshake reusing a prior secret is refused, and inspection confirms the secret is overwritten after the handshake completes.

Matrix mediates verification through the homeserver and Signal relays linked device traffic through its service. SSP keeps the QR model and removes the server from the trust path with the pre-shared key: the rendezvous relay cannot join the handshake, because it does not hold that secret.

3.6.7 Device removal

SSP-DEV-008 A DeviceRemove entry MUST carry a `revoke_from` time, and a verifier MUST treat every signature by that device at or after that time as invalid.

Rationale. A removal effective from the moment of writing would invalidate the whole history of that device and break the causal record of past messages.
Verification. Test: signatures made by the removed device before `revoke_from` still verify and their messages remain readable; signatures at or after it are refused.

SSP-DEV-009 On a DeviceRemove entry, every other device of the identity MUST start a commit in the self group and in every group where the identity is a member, within 60 seconds of learning of the entry.

Rationale. Post-compromise security. A removed device holds the current group secret until a commit replaces it.
Verification. Test: on removal, each remaining device issues a commit within 60 seconds in the self group and in every group of that identity.

SSP-DEV-010 A peer holding a session with the removed device MUST stop that session, delete its state, and reject any message from that device timed at or after `revoke_from`.

Rationale. A session left open to a removed device continues to accept its traffic and continues to advance a ratchet the attacker holds, which defeats the removal.
Verification. Test: after removal, a message from that device timestamped after `revoke_from` is refused and the session state is absent from the store.

SSP-DEV-011 A client MUST delete the mailbox it holds for a removed peer device, at every relay in the relay set of that device.

Rationale. An orphan mailbox is a place an attacker can leave data, and it consumes a quota.
Verification. Test: after removal, the mailbox for that peer device is absent at every relay in its relay set, verified by a status query.

Removal to a single device. Immediate removal is correct while several devices remain: the user is holding one device and disowning another. It is not correct when it would leave one device in sole control, because that is exactly what a compromised device does first.

SSP-DEV-012 A DeviceRemove entry that would leave fewer than two active devices MUST either carry a signature by the root key in force at its `prev`, or enter a pending state that ends 259 200 000 ms (72 hours) after its time value.

Rationale. A compromised device holds a device key and, usually, not the root key. The pending state converts a silent takeover into a visible, contestable event, and the root-key path keeps deliberate consolidation to one device immediate for a user who holds that key.
Verification. Test: with three active devices, removal of one applies immediately; the next removal, which would leave one device, is pending unless root-signed. With a root signature it applies immediately.

SSP-DEV-013 A client MUST raise a high-priority, persistent alert on every current device when a pending DeviceRemove appears in the log.

Rationale. The pending window is only useful if a surviving device sees it.
Verification. Demonstration: a pending removal is published while a second device is offline; that device alerts on its next synchronisation, and the alert persists across a restart.

SSP-DEV-014 A DeviceRemoveReject entry naming a pending DeviceRemove by its exact hash, signed by a current device key other than the one that signed the removal, MUST void that removal.

Rationale. The surviving device is the party the removal targets, and it is the party best placed to know whether the removal is legitimate.
Verification. Test: a rejection signed by the removing device itself is refused; a rejection signed by the targeted device is accepted and the removal never applies.

SSP-DEV-015 A further DeviceRemove naming the same target MUST NOT restart or extend the deadline of a pending removal already recorded for that target.

Rationale. Otherwise a compromised device stalls indefinitely by republishing, which is the pattern 3.6.11 removes from the recovery path.
Verification. Test: five successive removals for one target are published across the window; the effective deadline is that of the first, in every resulting state.

SSP-DEV-016 A verifier MUST evaluate concurrent DeviceRemove entries in the total order of 3.6.8, and MUST refuse any removal that would leave zero active devices.

Rationale. Two removals each valid against the same predecessor state could otherwise both apply and leave an identity with no device able to sign. Serialising them and re-checking the active-device count after each removal makes that state unreachable.
Verification. Test: two devices remain, and each signs a removal of the other against the same `prev`. After ordering, the first applies and the second is refused as a pending removal that would leave zero devices. One active device remains in every interleaving.

3.6.8 Concurrent identity log entries

Entries that name the same `prev` are concurrent. 3.6.12 treats a fork in a signed log as a security event, and the same total order is needed inside a single honest client that receives entries out of order, and by any verifier reconciling two partial views.

SSP-IDN-018 A verifier MUST order concurrent identity log entries by ascending sequence number, then by ascending entry hash compared as a byte string, and MUST apply the state transitions of 3.6.7 and 3.6.11 in that order.

Rationale. A deterministic order makes the resulting state a function of the entry set rather than of arrival timing, which is what lets two verifiers agree.
Verification. Test: 1000 random arrival permutations of one entry set, including concurrent removals, claims, contests, and rejections, all yield the same final state on every verifier.

Ordering alone does not decide every combination. Where a RecoveryClaim, a RecoveryReject, and a RecoverySetUpdate are concurrent, the snapshot rule of SSP-KEY-016 fixes which policy applies, and the order above fixes which entry is evaluated first. The remaining question is whether a RecoverySetUpdate ordered between a claim and its rejection should be visible to that rejection. This document takes the snapshot answer: it should not. 1.3 records the alternative for maintainer review.

3.6.9 Key hierarchy

SSP-KEY-001 A private key MUST serve exactly one purpose in this hierarchy.

Rationale. Key reuse across a signature scheme and a key agreement scheme has produced real breaks, and separation lets one key rotate without disturbing the others.
Verification. Inspection of the key hierarchy against every call site: no key is passed to both a signature and a key-agreement operation.

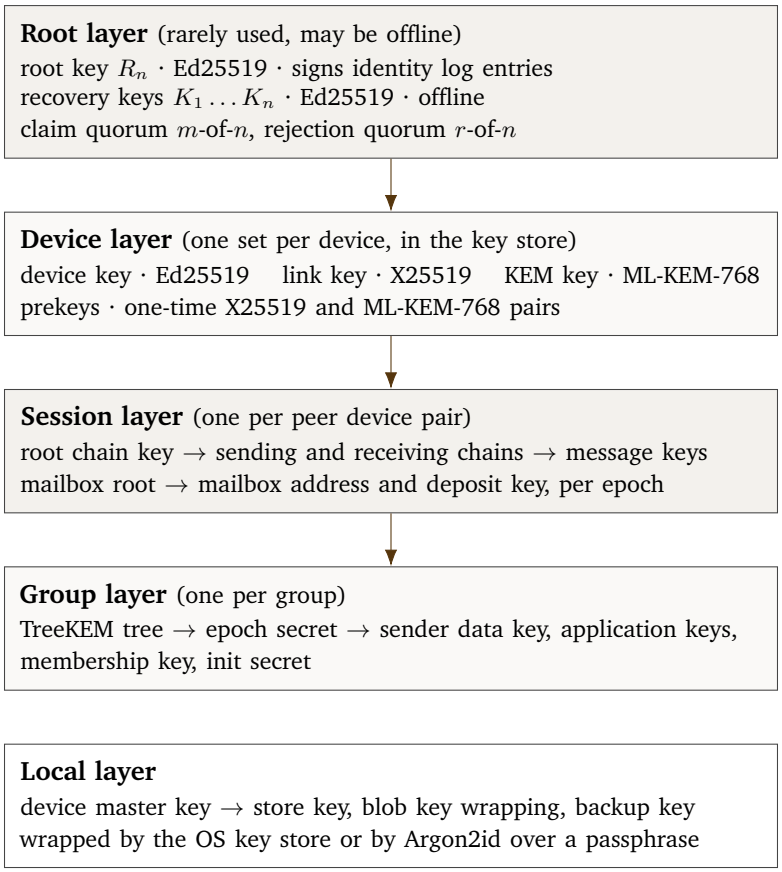

Figure 10. Key hierarchy. Each private key serves exactly one purpose.

SSP-KEY-002 A client MUST derive every symmetric key as KDF(parent, `SSP1/...`, length), where the label names the exact purpose.

Rationale. Domain separation. Appendix B holds the complete label registry, and an implementation never invents a label.
Verification. Test against the planned label vectors: each derivation reproduces its published output, and no two labels yield the same key from one parent.

SSP-KEY-003 A client MUST zeroize a key when it leaves the active window, and MUST NOT allow key material to reach a swap file, a core dump, or a log.

Rationale. Forward secrecy at the protocol level is worthless if plaintext keys stay in memory for the life of the process.
Verification. Inspection of the secret types, plus a test that a core dump taken after a session closes contains none of the message keys used.

3.6.10 Rotation

SSP-KEY-004 A client MUST rotate the root key at least once every 365 days while the identity is active, and MUST sign the RootRotate entry with both the old and the new root key.

Rationale. The double signature proves that the holder of the old key intended the change and that the new key exists. A single signature would let an attacker who steals the old key hand control to

Key	Trigger	Method
Message key	Every message	Symmetric ratchet
Chain key	Every DH ratchet step	Double ratchet
Session DH key	Every reply, and at most every 100 messages	Double ratchet
Prekey	Every use	One-time; the client uploads a new batch
Link and KEM key	180 days, or suspicion	DeviceAdd of a new record, DeviceRemove of the old
Deposit key	30 days, or abuse	Register a new mailbox, delete the old
Mailbox address	24 hours	Derived from the epoch index; no message needed
Device key	Not rotated	A new device replaces the old one
Root key	365 days, or suspicion	RootRotate entry
Recovery set	User action	RecoverySetUpdate entry

Table 26. Rotation schedule.

a key the attacker cannot use.
Verification. Test: a rotation entry signed by only one of the two keys is rejected; one signed by both is accepted and the identifier is unchanged.

SSP-KEY-005 A client MUST keep at least 32 unused one-time prekeys at each relay of its relay set, and MUST upload more when the count falls below eight.

Rationale. An empty prekey store forces a fallback to the static link key, which weakens forward secrecy for the first message. An attacker who drains the store on purpose gains that weakness.
Verification. Test: the client uploads a fresh batch when the remaining count reaches eight, and the store never empties under a sustained draw of one key per minute.

SSP-KEY-006 When the prekey store of a peer is empty, a sender MUST use the static link and KEM keys, MUST mark the session as a weak start, and MUST run a DH ratchet step at the first reply.

Rationale. Refusing to send would turn prekey exhaustion into a denial-of-service attack. The mark and the fast ratchet bound the damage.
Verification. Test: with the prekey store empty, the session is established, marked as a weak start, and performs a ratchet step on the first reply.

3.6.11 Recovery

Recovery answers one question: the user lost every device and must keep the SID.

Recovery policy. The recovery policy is carried in the genesis entry and replaced by a RecoverySetUpdate entry. It holds one to eight Ed25519 recovery public keys and two thresholds:

- the *claim threshold* m: the number of recovery keys that must sign a RecoveryClaim for that claim to be valid;
- the *rejection threshold* r: the number of recovery keys that must sign a RecoveryReject for a claim to be permanently rejected.

Values for m and r are not fixed by this document. They trade the risk of a stolen recovery key against the risk of an unrecoverable identity, and the trade-off depends on how the recovery keys are held. The alternatives are listed as an open decision in 1.3.

SSP-KEY-007 A genesis entry MUST carry a recovery policy with at least one recovery key and with both thresholds set.

Rationale. An identity with no recovery path is destroyed by a lost phone, and the user then rebuilds every contact by hand. A policy with a claim threshold and no rejection threshold cannot terminate a contested claim.
Verification. Test: a genesis entry with an empty recovery key list, with $m = 0$, with $r = 0$, or with m or r greater than the key count, is rejected by the verifier.

SSP-KEY-008 A client MUST derive a passphrase recovery key with Argon2id, with the parameters of Appendix B, over the salt $H(\texttt{SSP1/recovery-salt} \,\|\, \text{normalised passphrase})$ truncated to 16 bytes.

Rationale. A SID-independent salt lets the user recover from the passphrase alone. Argon2id at these parameters puts an offline attack on a 24-word passphrase far beyond the value of the target.
Verification. Test against the planned derivation vectors of Appendix E: the same normalised passphrase yields the same key on two implementations, and a passphrase differing by one Unicode normalisation form yields the same key.

Authority. Device keys and the root key can *detect* and *contest* a recovery claim. Neither can reject one. Only the recovery policy, through a rejection quorum, can end a claim permanently, and a fixed deadline ends it otherwise. This split exists because of a specific attack: a single compromised device can remove its siblings (3.6.7) and then, if a device key alone could veto, refuse every recovery claim for ever. The legitimate holder of the recovery keys must always be able to reach a decision in bounded time.

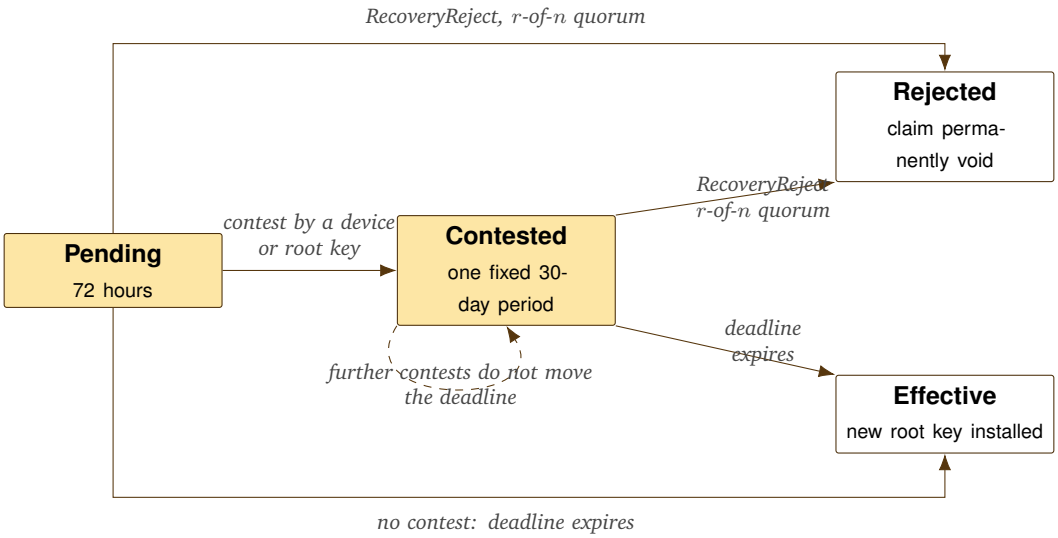

Figure 11. Recovery claim state machine. A contest buys time for the recovery authority to decide; it cannot decide by itself. Every path terminates.

Initial claim. A RecoveryClaim names the new root key, the first device record of the recovering user, and `head_ref`: the hash of an identity log entry that the claimant accepts as the current head.

SSP-KEY-009 A RecoveryClaim MUST carry a `head_ref` naming an entry of the identity log, and a verifier MUST validate the claim against the recovery policy in force at that entry.

Rationale. Binding the claim to a log position fixes which policy governs it. Without the binding, an attacker holding a device key could publish a RecoverySetUpdate after seeing a claim and change the rules the claim is judged by.
Verification. Test: a claim whose `head_ref` names an unknown entry is rejected; a claim signed by a quorum valid at `head_ref` but not under a later policy is still accepted; a claim signed only by a quorum valid under a later policy is rejected.

SSP-KEY-018 A RecoveryClaim MUST be signed by at least m distinct recovery keys of the policy in force at `head_ref`, where m is the claim threshold of that policy.

Rationale. The claim threshold is what separates a recovery from an unauthenticated request to replace the root key.
Verification. Test: claims signed by $m-1$, m, and $m+1$ distinct keys, and a claim carrying m signatures from a single key repeated, exercising the distinctness check.

SSP-KEY-010 A RecoveryClaim MUST enter a pending period of at least 259 200 000 ms (72 hours), measured from its own time value, before it can take effect.

Rationale. The window must be long enough for a user with one offline weekend to notice a claim they did not make. Shorter favours an attacker holding a recovery key; longer makes legitimate recovery painful.
Verification. Test: a claim with a 71-hour window is rejected as malformed; a claim with a 72-hour window that reaches its deadline with no contest and no rejection takes effect at the deadline and not before.

Contest. A contest is an assertion that the claim is suspect. It is not a decision.

SSP-KEY-019 A RecoveryContest entry MUST be signed by a device key that is active at `head_ref` of the contested claim, or by the root key in force at that entry, and MUST name the claim by its exact hash.

Rationale. Restricting the signer set to keys that existed before the claim stops a claimant from contesting its own claim, and stops a third party from injecting contests.
Verification. Test: contests signed by a device active at `head_ref`, by a device removed before `head_ref`, by the root key, and by an unrelated key; only the first two of these four are accepted.

SSP-KEY-020 A device record introduced by a RecoveryClaim MUST NOT be accepted as a signer of a RecoveryContest against that same claim.

Rationale. Otherwise a claim could suppress competing claims through the device it introduces, and an attacker could pair a claim with a self-contest to stall.
Verification. Test: the device in the claim signs a contest against its own claim; the contest is rejected while the claim remains pending.

SSP-KEY-011 The first valid RecoveryContest against a claim MUST move that claim into a contested period ending exactly 2 592 000 000 ms (30 days) after the time value of the claim. A later valid contest against the same claim MUST NOT change that deadline.

Rationale. One fixed deadline computed from the claim, not from the contest, is what guarantees termination. A deadline extended by each contest would let a single compromised device stall recovery indefinitely, which is the failure this design exists to remove.
Verification. Test: one contest, then five further contests spread across the period; the computed deadline is identical in all six states, and the claim takes effect at that deadline when no rejection arrives.

SSP-KEY-012 A client MUST raise a high-priority, persistent alert on every current device when it observes a RecoveryClaim or a RecoveryContest for its own identity.

Rationale. Both the pending period and the contested period are only useful if the legitimate holder learns that a decision is running.
Verification. Demonstration: a claim is published while two devices are offline; both raise the alert on their next synchronisation, and the alert persists across a restart until the claim resolves.

Final decision. Rejection is the only way to end a claim early, and it belongs to the recovery authority alone.

SSP-KEY-013 A RecoveryReject entry MUST name the claim by its exact hash and MUST be signed by at least r distinct recovery keys of the policy in force at the `head_ref` of that claim, where r is the rejection threshold of that policy.

Rationale. Permanent rejection is a decision about who controls the identity, so it needs the same class of authority as the claim itself.
Verification. Test: rejections signed by $r - 1$ and by r distinct recovery keys; a rejection naming a different claim hash; a rejection evaluated under a policy installed after `head_ref`. Only the second case voids the claim.

SSP-KEY-014 A device key alone and the root key alone MUST NOT reject a RecoveryClaim.

Rationale. This is the denial-of-recovery attack TH-21. A compromised sole device would otherwise reject every claim for ever and the identity could never be recovered.
Verification. Test: a compromised device removes every sibling, then signs a rejection for each of three successive claims; each rejection is refused and each claim reaches its deadline and takes effect.

SSP-KEY-015 A RecoveryClaim MUST take effect when its deadline passes with no valid RecoveryReject recorded against it, where the deadline is the end of the pending period if the claim was never contested, and the end of the contested period otherwise.

Rationale. A guaranteed terminating outcome. Without it, the absence of a decision would be indistinguishable from a rejection, which is the state an attacker wants.
Verification. Test: an uncontested claim takes effect at 72 hours; a contested claim with no rejection takes effect at 30 days; a contested claim with a valid rejection at day 29 does not take effect.

SSP-KEY-016 A verifier MUST evaluate a RecoveryClaim, every RecoveryContest against it, and every RecoveryReject against it under the recovery policy in force at the `head_ref` of that claim, whatever later RecoverySetUpdate entries the log holds.

Rationale. Snapshot semantics. Freezing the whole recovery set during a pending claim would let one claim block all policy maintenance for a month; evaluating a pending claim under a later policy would let an attacker rewrite the rules mid-decision. Binding to the snapshot avoids both.
Verification. Test: a claim is published, then a RecoverySetUpdate replaces every recovery key, then a rejection is signed by the new keys. The rejection is refused. A rejection by the old keys, valid at `head_ref`, is accepted.

SSP-KEY-017 An identity MUST NOT accept a RecoverySetUpdate signed under the authority introduced by a RecoveryClaim before that claim takes effect.

Rationale. Otherwise a claimant could install its own recovery set during the pending period and reject any competing claim from the legitimate holder.
Verification. Test: the new root key from a pending claim signs a RecoverySetUpdate at day one; the entry is refused until the claim takes effect.

A later RecoverySetUpdate governs every claim whose `head_ref` names an entry at or after it. Where a claim, a rejection and a policy update are concurrent, the tie-break of 3.6.8 applies.

Residual limitation. If an attacker controls a full recovery quorum, SSP cannot distinguish that attacker from the legitimate recovery authority. The attacker can claim the identity and can reject the claims of the legitimate holder. No mechanism in this document changes that: the recovery keys are the root of authority for recovery, and control of the root of authority is control of the identity. The contest path narrows the window in which such an attack goes unnoticed, and SSP-KEY-012 makes it visible on every device, but neither prevents it. This is why the choice of m, r, and the storage of the recovery keys is a security decision for the user, not a default this document can set.

Alternative	Reason it is not the default
m-of-n social recovery	Leaks the social graph to the participants and needs them online and competent. Available as a policy, not as the default.
Server-side backup with a PIN	Works, and needs trust in an enclave vendor and an operator. SSP has no operator.
Shamir shares across relays	Relays are untrusted and interchangeable; a share at a relay is a share at an attacker.
Veto by a single device key	The design this section replaces. One compromised device can deny recovery for ever.
No recovery at all	A lost device destroys the identity and the user rebuilds every contact by hand.

Table 27. Recovery mechanisms considered.

3.6.12 Fork detection and compromise

Two valid entries sharing a parent form a fork. An honest client never writes one, so a fork means either that an attacker holds a signing key or that a relay serves an inconsistent view.

SSP-IDN-010 A client MUST store the hash of the highest entry seen for each contact and compare a new tail against it.

Rationale. The client-side transparency check, at a cost of 32 bytes per contact.
Verification. Test: a stored head is compared against a served tail; a tail that does not extend the stored head is reported rather than adopted.

SSP-IDN-011 A client MUST fetch a changed identity log from at least two relays in the relay set of that contact, and MUST report a mismatch as a fork.

Rationale. One relay can serve a split view; two relays must collude, which raises the cost.
Verification. Test with a lying relay in the harness: the client fetches from two relays, detects the mismatch, and reports a fork rather than adopting either tail.

SSP-IDN-012 On a fork, a client MUST stop all sends to that identity, show both heads to the user, and MUST NOT choose a head by itself.

Rationale. Automatic resolution is what the attacker wants. A rule that picks the longer chain lets an attacker win by writing more entries.
Verification. Demonstration: on a fork the client stops sending to that identity, shows both heads, and offers no automatic resolution.

SSP-IDN-013 A client MUST include the known head hash of the identity log of a peer inside its encrypted messages to that peer, at least once every 24 hours.

Rationale. This turns every conversation into a transparency gossip channel at almost no cost. An attacker who forks a log must then keep the fork consistent across every conversation of the victim, which is not achievable in practice.
Verification. Test: over a 48-hour session, at least one message per 24 hours to each peer carries a log head; a peer serving a forked log to one conversation is detected by the other.

Situation	Response
One device lost, others available	Any other device writes DeviceRemove. Every group commits. Sessions restart.
Every device lost, recovery quorum available	Recovery claim, 72-hour pending period, new root key and new device at the deadline.
Root key leaked, a device available	Rotate the root key now, alert every contact.
One recovery key leaked, below the claim threshold	Update the recovery policy now, signed by the root key and a current quorum.
The attacker removed the honest devices	Removal to a single device is itself delayed (3.6.7). The honest user files a recovery claim. The attacker can contest, which extends the decision to 30 days once, and cannot reject: rejection needs a recovery quorum. The claim takes effect at the deadline.
A recovery quorum is in hostile hands	Not recoverable. The attacker is indistinguishable from the legitimate recovery authority (3.6.11).
Every key lost, no recovery	The identity is dead. Write a Close entry if any device still works.

Table 28. Response to compromise.

3.6.13 Contact codes

There is no directory. A user reaches a new contact only with a contact code, which carries the SID, the introduction mailbox address, the private seed of the deposit key for that mailbox, up to four relay hints, the log head at the time of issue, and optional single-use and expiry flags. The text form uses Bech32m with the human-readable part `suncode`; the QR form carries the same bytes.

SSP-IDN-014 A contact code MUST carry the private seed of the deposit key of the introduction mailbox.

Rationale. The holder needs the ability to deposit there. The key is not a shared secret between two parties; it is a token meaning "the bearer met the owner". Anyone who copies the code can send one introduction, and 3.10.2 bounds that with proof of work and a per-mailbox quota.
Verification. Test: a party holding only the contact code can deposit exactly one introduction into the named mailbox, and cannot deposit into any other mailbox of that identity.

SSP-IDN-015 A client MUST support single-use contact codes, and MUST rotate the introduction mailbox when the user requests a new code.

Rationale. A code posted in public becomes a spam target; a single-use code shown to one person does not.
Verification. Test: a single-use code is accepted once and refused on the second use; requesting a new code changes the introduction mailbox address.

SSP-IDN-016 A client MUST NOT publish, upload, or index a contact code anywhere in the protocol.

Rationale. N-03. A published set of codes is a user directory.
Verification. Inspection of every outbound operation: no request carries a contact code to a relay or a directory.

SSP-IDN-017 A client MUST show that an incoming introduction came from a named existing contact, and MUST NOT accept it automatically.

Rationale. An introduction is a transfer of trust, and the user makes it.
Verification. Demonstration: an introduction arriving inside an existing conversation names that contact and adds nothing until the user accepts.

3.7 Cryptographic design

3.7.1 Selection criteria

Every primitive was judged against these criteria, in order:

1. **Misuse resistance.** How does it fail when an implementer makes an ordinary mistake? A primitive that fails catastrophically on nonce reuse ranks below one that degrades.
2. **Constant-time software implementation.** It must be safe on a device with no hardware acceleration.
3. **Public analysis.** Years of open review.
4. **Availability.** At least two independent Rust implementations and one audited C implementation.
5. **Portability.** It must run on a 32-bit target with no allocator.
6. **Performance.** Ranked below everything above it.
7. **Size on the wire.** Small output helps padding and battery cost.

SSP-CRY-001 SSP/1 MUST define exactly one algorithm for each cryptographic role, and MUST NOT negotiate an algorithm inside a session.

Rationale. Agility inside a session creates a downgrade surface and doubles the test matrix. It has never saved a deployed protocol from a real break in less time than a version bump would have taken.

Verification. Inspection of the primitive table against every call site: no code path selects an algorithm from a negotiated value.

3.7.2 Primitive set

Role	Algorithm	Size
Hash, content address	BLAKE3-256	32 B
Keyed hash and MAC	BLAKE3 keyed mode	32 B key and tag
Key derivation	BLAKE3 keyed mode with XOF output	any
Signature	Ed25519, strict rules of 3.7.4	32 B key, 64 B sig
Key agreement, classical	X25519	32 B
Key agreement, post-quantum	ML-KEM-768	1184 B key, 1088 B ct
AEAD, counter nonce	ChaCha20-Poly1305	32 B key, 12 B nonce
AEAD, random nonce	XChaCha20-Poly1305	32 B key, 24 B nonce
Password hash	Argon2id	Appendix B
Handshake framework	`Noise_IK_25519_ChaChaPoly_BLAKE2b`	
Transport, binding A	TLS 1.3 in QUIC, raw public keys	
Proof of work	Equi-X	16 B solution
Group key schedule	TreeKEM with the suite of 3.9.6	

Table 29. Mandatory primitives.

3.7.3 BLAKE3

BLAKE3 is the most consequential choice here, because the protocol uses it in four roles.

- **The tree structure gives verified streaming.** A recipient checks a 64 KiB chunk of a 100 MiB attachment against the root hash with a log-size proof, and rejects a bad chunk immediately. Attachments (3.9.9) depend on this. SHA-256 cannot do it, and a Merkle tree over SHA-256 would need a separate specification.
- **One function gives three modes.** Plain, keyed, and derive-key. SSP uses the keyed mode as both MAC and KDF, which removes HMAC and HKDF from the implementation, along with two more places to pick the wrong construction.
- **Speed on a weak device.** Several times faster than SHA-256 in software with no SHA extension. Battery cost matters when a client checks logs, hashes attachments, and derives mailbox addresses every day.
- **Extendable output.** SSP needs 32 bytes in one place and 64 in another; the XOF removes the counter loop HKDF-Expand needs.

The costs: BLAKE3 is younger than SHA-256 and less deployed, though its core is BLAKE2s, which has a long analysis record, and the tree mode has a published proof. It carries no NIST approval, so SSP cannot be used in an environment that requires one. Compliance is not a goal.

> **SSP-CRY-002** A client MUST use BLAKE3-256 for every hash in this specification, except inside the Noise handshake and inside TLS.
>
> *Rationale.* One hash function for the protocol. 3.7.8 explains the exception.
> *Verification.* Test: every hash output in the encoding and derivation vectors matches the BLAKE3 reference; inspection confirms the only other hash functions reachable are inside the Noise handshake and TLS.

Candidate	Reason for rejection
SHA-256	No tree mode, no keyed mode, slower in software, and needs HMAC and HKDF as separate constructions. Its only advantage is compliance status.
SHA-512, SHA-512/256	Faster on 64-bit hardware and slower on 32-bit hardware. The same missing modes.
SHA-3, SHAKE	A good XOF and much slower in software. The sponge offers no tree mode for verified streaming.
BLAKE2b	The direct ancestor. No tree mode in the common libraries and no XOF in the standard form.

Table 30. Hash functions considered.

3.7.4 Ed25519

Small keys and signatures, deterministic nonce generation, fast verification, no dependence on a random source at signing time, and a large set of audited implementations. Determinism removes the failure class that broke ECDSA in several deployed systems.

ECDSA over P-256 is rejected for its random nonce requirement and its point validation history; RSA for key size and parameter choice; Ed448 because it is slower and 128-bit classical security is sufficient under a plan that already covers the quantum threat.

Known problem	Answer
Malleability, where a verifier accepts a non-canonical S	The strict rules below.
Libraries accept different signature sets, breaking agreement	One fully specified rule set.
No strong unforgeability under key reuse across contexts	A domain label in every signed object (SSP-GEN-007).
Not post-quantum	3.7.12 states the plan and the reason for the delay.

Table 31. Ed25519 problems and answers.

SSP-CRY-007 An Ed25519 verifier MUST apply all of the following:

(a) reject a signature whose scalar S is not fully reduced modulo the group order;

(b) accept a public key A and a signature point R in a small-order subgroup, using the cofactored equation $[8][S]B = [8]R + [8][k]A$;

(c) reject a public key or signature point that is not a valid, canonically encoded curve point;

(d) reject a signature over an object whose encoding is not canonical.

Rationale. Rules (a) to (c) give one deterministic answer across every library implementing the ZIP 215 rule set, and remove the malleability that lets an attacker produce a second valid signature over one object. Rule (d) ties the signature rules to 3.5.4.
Verification. Test with the full ZIP 215 vector set: the accept and reject sets match exactly, including the small-order and non-canonical scalar cases.

SSP-CRY-008 An implementation MUST NOT use batch verification for an identity log entry unless that mode yields the same accept set as rules (a) to (c) of SSP-CRY-007 for every input.

Rationale. Batch verification with a different accept set turns a log into a fork generator across implementations.
Verification. Test: the batch path and the single path are run over the full ZIP 215 vector set and over 10^5 random signatures; the accept sets are identical.

3.7.5 X25519

A fixed 32-byte encoding, no point format confusion, a safe curve whose cofactor the X25519 function handles, and universal library support.

SSP-CRY-003 A client MUST check that an X25519 output is not the all-zero value and MUST abort the operation when it is.

Rationale. A peer sending a small-order point would otherwise force a predictable shared secret. The check is one comparison, and its absence has broken deployed systems.
Verification. Test: each of the known small-order inputs produces an abort, and no session key is derived in any of those cases.

3.7.6 ChaCha20-Poly1305

A software implementation is naturally constant-time, where AES needs table-free code to reach the same property and most libraries do not provide it. Performance on a device without AES-NI or the ARMv8 crypto extension is far better. The construction is simple, and nonce reuse costs the confidentiality of two messages and the authentication key, which is no worse than GCM, where reuse also gives forgery of every later message. The 256-bit key leaves a comfortable margin against a Grover-style attack.

> **SSP-CRY-004** A client MUST derive a fresh key for each AEAD invocation in the message path, and MUST use the nonce that 3.9.2 specifies.
>
> *Rationale.* A fresh key per message makes the nonce question trivial and removes the counter management defects that cause reuse.
> *Verification.* Test: over 10^5 messages in one session, no AEAD key is used twice and every nonce equals the chain index of its message.

> **SSP-CRY-005** Where a nonce cannot come from a counter, a client MUST use XChaCha20-Poly1305 with a 24-byte random nonce.
>
> *Rationale.* A 24-byte random nonce keeps the collision probability negligible for any realistic message count. A 12-byte random nonce does not.
> *Verification.* Inspection of each AEAD call site: every site with a non-counter nonce uses the extended-nonce construction.

AES-256-GCM is not in the mandatory set. A client MAY use it for the local store on a platform with hardware acceleration, where the key never crosses the network and the platform controls the nonce, and MUST NOT use it on the wire.

3.7.7 Argon2id

Memory hardness, resistance to both side-channel and time-memory tradeoff attacks, an RFC, and wide library support. Parameters are in Appendix B and target about one second on a mid-range 2024 phone. PBKDF2 is rejected as cheap on a GPU, scrypt because Argon2id supersedes it with clearer parameter guidance, and bcrypt for its password length limit and fixed small memory use.

3.7.8 Noise with BLAKE2b

The Noise framework names four hash functions: SHA-256, SHA-512, BLAKE2s, and BLAKE2b. BLAKE3 is not among them. SSP uses `Noise_IK_25519_ChaChaPoly_BLAKE2b` and defines no BLAKE3 variant. A new hash inside Noise would produce a pattern name no other implementation has, would discard the published Noise test vectors, and would move the handshake outside the analysis the Noise community has already done. The gain would be one fewer hash function in the binary; the loss would be the review lineage. BLAKE2b ships in every library that implements Noise, so the second hash costs no new dependency in practice.

3.7.9 QUIC with raw public keys

Binding A uses TLS 1.3 inside QUIC with the raw public key certificate type of RFC 7250. The relay authenticates with its Ed25519 relay key. There is no certificate chain, no certificate authority, and no name validation. A certificate authority is a trusted third party, and SSP has none. A relay identifier is a key, exactly as an identity is a key structure. The client learns the

relay key from the relay set, which arrives inside an authenticated identity log or a contact code.

SSP-NET-003 A client MUST authenticate a relay by its Ed25519 relay key, and MUST NOT accept a relay on the basis of a certificate chain, a hostname, or a DNS record.

Rationale. A hostname is a name a registrar controls and a certificate chain is a trust set a browser vendor controls. Neither belongs in the trust model.
Verification. Test: a relay presenting a valid certificate chain for the expected hostname but a different Ed25519 key is refused before any frame is sent.

3.7.10 Architectural alternatives rejected

Table 32. Cryptographic architectures considered and rejected.

Alternative	Reason
HPKE (RFC 9180) for messages	A clean single-shot public key encryption interface with no ratchet. SSP needs forward secrecy per message, so the ratchet is mandatory and HPKE would be an extra layer with no gain.
MLS as the whole protocol	Excellent, and it assumes a delivery service that totally orders handshake messages. That service is a central point. 3.9.7 keeps TreeKEM and removes the ordering service.
A triple ratchet or sender-key hybrid for two parties	The double ratchet suffices for two parties and has the longer analysis record.
Sender keys for groups	O(1) send cost, at the price of forward secrecy inside an epoch and a full redistribution on every membership change. TreeKEM gives logarithmic cost on membership change, which matters at 1024 members.
Onion routing inside SSP	Tor exists, has a decade of analysis, and has a network. 3.8.3 lets SSP run over it. A second onion network with a small user base would give worse anonymity than Tor with a large one.
Ring signatures or stealth addresses	These hide a sender in an anonymity set on a public ledger. SSP has no ledger, and its addressing layer already gives an opaque per-pair address, which is stronger here and far cheaper.
Post-quantum signatures now	3.7.12.

3.7.11 Key derivation

Every derivation has the same shape:

$$k = \mathsf{BLAKE3}_{\text{keyed}}\big(k_{\text{parent}},\ \texttt{SSP1/} \,\|\, \text{purpose} \,\|\, \texttt{0x00} \,\|\, \text{context}\big)$$

where the purpose comes from the registry in Appendix B and the context holds the values that make the derivation unique, canonically encoded.

Purpose	Context	Output
`mailbox-root`	sender SID, sender device, recipient SID, recipient device	32 B
`mailbox-addr`	epoch index	32 B
`deposit-seed`	epoch index	32 B
`ratchet-root`	DH output	64 B
`msg-key`	chain index	32 B
`blob-key`	blob root hash	32 B
`store-key`	device id	32 B

Table 33. Derivation examples.

SSP-CRY-006 An implementation MUST NOT use a label outside the registry of Appendix B, and MUST NOT reuse a label for a second purpose.

Rationale. Two derivations sharing a label produce related keys in unrelated contexts, which is the failure domain separation exists to prevent. Keeping the set closed also makes the registry a complete inventory of the key schedule.
Verification. Test: the label set reachable at runtime equals the registry in Appendix B, checked by enumerating the derivation call sites.

3.7.12 Post-quantum plan

Confidentiality and authenticity have different deadlines. Confidentiality has a deadline in the past: an adversary records traffic today and decrypts it when a quantum computer exists. Authenticity has a deadline in the future: a signature broken in 2040 is worthless, because the message is old and the key long rotated. An attacker must break a signature before the moment of use.

SSP-CRY-009 Every session key agreement MUST combine X25519 and ML-KEM-768, and MUST derive the session secret from both outputs.

Rationale. The hybrid keeps classical security if ML-KEM-768 falls to cryptanalysis, and quantum security if X25519 falls to a quantum computer. Lattice schemes are young, and a hybrid is the only responsible choice at a ten-year horizon.
Verification. Test: removing either component from the combination changes the derived secret, and a session established with only one component fails to decrypt traffic from a conformant peer.

The combination is a hash of both secrets with the transcript:

$$ss = \mathrm{KDF}\big(\texttt{SSP1/hybrid-combine},\ ss_{\text{X25519}} \,\|\, ss_{\text{ML-KEM}} \,\|\, H_{\text{transcript}},\ 64\big)$$

SSP-CRY-010 The hybrid combination MUST hash both secrets together with the handshake transcript, and MUST NOT use exclusive-or.

Rationale. A hash of both inputs is secure when either input is secure, and it binds the result to the transcript. Exclusive-or reaches the same security only under stronger assumptions about the independence of the two outputs.
Verification. Analysis of the key schedule, plus a test that altering one byte of either component secret or of the transcript changes the derived session secret.

Signature migration.

SSP-CRY-011 Every key record in an identity log MUST carry a two-byte algorithm identifier, and a verifier MUST reject an identifier it does not know.

Rationale. This one field is the entire migration mechanism. A future SSP/2 adds the ML-DSA-65 identifier, and an SSP/1 client meeting it fails closed rather than accepting an unverified key.
Verification. Test: a key record carrying an unassigned algorithm identifier is rejected, and the entry containing it does not apply.

The procedure, when the maintainers trigger it: SSP/2 defines the ML-DSA-65 identifier and a dual-signature entry type; a client writes a RootRotate carrying both an Ed25519 key and an ML-DSA-65 key, signed by both; contacts that support SSP/2 use the newer key and older contacts use Ed25519; after the deprecation date a verifier rejects an Ed25519-only entry.

ML-DSA-65 costs 1952 bytes per public key and 3309 bytes per signature today. In a log with fifty entries and eight devices that is roughly 200 KiB of growth, which 3.12 does not accept while the classical scheme is sound.

3.7.13 Randomness

SSP-CRY-012 A client MUST take all random bytes from the operating system generator.

Rationale. A user-space generator adds a failure mode with no benefit; the kernel holds the entropy sources and the reseed logic.
Verification. Inspection of every path that produces random bytes: each reaches the platform generator directly, with no intermediate buffer or user-space expansion.

SSP-CRY-013 A client MUST NOT generate a long-term key before the operating system reports that the generator is ready.

Rationale. An embedded device or a fresh virtual machine can boot with a predictable pool, and keys made in that window are guessable.
Verification. Test with a delayed entropy source: key generation blocks rather than proceeding, and no key is emitted before the platform reports readiness.

SSP-CRY-014 A client MUST NOT use a deterministic generator seeded only by a user passphrase, except for the recovery key derivation of 3.6.11.

Rationale. A passphrase-seeded generator produces the same key material on every device that knows the passphrase, which silently removes the independence the key hierarchy of 3.6.9 assumes. The recovery derivation is the one case where that reproducibility is the point.
Verification. Inspection of key generation call sites: only the recovery derivation of SSP-KEY-008 accepts a passphrase-derived seed.

3.8 Networking, transport, and relays

3.8.1 Network model

The network holds clients and relays and nothing else: no supernode, no bootstrap authority, no distributed hash table, no overlay between relays.

A device holds a relay set of one to four relays. It deposits into the relay set of the peer and fetches from its own. Two devices need no common relay, because the sender connects to the relay of the recipient.

SSP-NET-004 A device SHOULD maintain a relay set of at least two relays run by different operators.

Rationale. One relay is a single point of failure and of observation, and two relays under one operator give neither redundancy nor privacy. It is a recommendation because a user may have access to only one relay, and the protocol still works in that case with reduced availability and a wider view for that operator.
Verification. Demonstration: with two relays, one is stopped and delivery continues; with one relay configured, the client warns and continues.

SSP-NET-005 A sender MUST deposit a copy of the envelope at every relay in the relay set of the recipient device, unless it received an acknowledgement from one relay and the recipient acknowledged the message end-to-end within 30 seconds.

Rationale. A relay that drops data is inside T4. Duplicate deposits cost bandwidth and buy delivery under an unreliable or hostile relay. The recipient discards the duplicate by message identifier.
Verification. Test with one relay silently dropping: the message still arrives through the second relay, and the recipient deduplicates when both deliver.

3.8.2 Binding A: QUIC

Binding A is the default, and both clients and relays support it.

Property	Value
Transport	QUIC version 1
Security	TLS 1.3 inside QUIC
Certificate type	Raw public key, Ed25519
ALPN	`ssp/1`
Cipher suite	`TLS_CHACHA20_POLY1305_SHA256` only
Key exchange group	`x25519` only
Client authentication	None at the TLS layer
Datagrams	RFC 9221, for padding and keepalive
Default port	4404/UDP

Table 34. Transport binding A.

QUIC brings stream multiplexing without head-of-line blocking, so a client can fetch from two hundred mailboxes and upload a blob on one connection. Connection migration lets a phone move between mobile and wireless networks without a fresh handshake, which matters both for battery cost and for the metadata a new handshake reveals. Resumption puts a fetch in the first flight when a client wakes. Congestion control and loss recovery come free. The datagram

extension gives a cheap channel for padding.

SSP-NET-006 A client and a relay MUST offer exactly one cipher suite and one key exchange group in the TLS handshake of binding A.

Rationale. It removes the downgrade surface and makes the handshake fingerprint identical across every SSP client, which supports 3.11.4.
Verification. Test with a packet capture: the ClientHello offers exactly one cipher suite and one group, and is byte-identical across two independent clients.

SSP-NET-007 A client MUST NOT send an application payload in a 0-RTT flight that changes state at the relay.

Rationale. 0-RTT data is replayable by definition. A fetch is idempotent and safe; a deposit is not, and the correct rule is to keep replayable data out of that flight.
Verification. Test: a captured 0-RTT flight is replayed; it produces no additional stored object at the relay, and the connection carries no deposit before the handshake completes.

Candidate	Reason for rejection
TCP with custom framing and Noise, as the only binding	Multiplexing, flow control, and migration would all have to be written. Binding B does use this stack, and only where QUIC cannot go.
HTTPS and WebSocket	Adds an HTTP stack, an HTTP parser, and a header set that leaks a client fingerprint. It buys traversal of a strict proxy, which binding B with a pluggable transport also achieves.
Plain UDP with custom reliability	Rewriting QUIC, worse.
Tor as the only transport	Multi-second latency, bandwidth limits, and dependence on one network. 3.8.3 makes it an option instead.

Table 35. Transports considered.

3.8.3 Binding B: TCP with Noise IK

Binding B serves three cases: an onion service address, a network that blocks UDP, and a censored network that needs obfuscation.

Property	Value
Transport	TCP, or a pluggable transport presenting a stream
Handshake	`Noise_IK_25519_ChaChaPoly_BLAKE2b`
Initiator static	An ephemeral key per connection, never the device key
Responder static	The X25519 link key of the relay, from the relay set
Framing	A `u16` length prefix, then a Noise transport message
Maximum message	65 535 bytes, segmented by the frame layer
Default port	4404/TCP

Table 36. Transport binding B.

SSP-NET-008 A relay MUST support binding B, and MUST publish both an Ed25519 relay key and an X25519 link key in its relay record.

Rationale. A relay reachable only over QUIC is unreachable from a network that blocks UDP and from an onion service, and the user in that position is the user who needs SSP most.
Verification. Demonstration: a client on a network that blocks UDP completes a deposit and a fetch against the relay over binding B.

SSP-NET-009 In binding B a client MUST use a fresh ephemeral static key per connection, and MUST NOT present its device key or link key to the relay.

Rationale. The IK pattern authenticates the initiator to the responder, which SSP does not want. A stable client key would link every connection of that device across time and across networks. The client stays anonymous to the relay, and the mailbox deposit key supplies the authorisation instead.
Verification. Test: two connections from one client present different static keys, and neither matches any key in that client's identity log.

SSP-NET-010 A client running over an onion network MUST NOT use binding A.

Rationale. QUIC needs UDP and an onion network gives a TCP stream. A client that tries binding A there either fails or falls back in a way an observer can see.
Verification. Test: with an onion transport configured, the client makes no UDP socket call, verified by a system-call trace.

SSP-NET-011 A client MUST support a pluggable transport interface taking a stream and returning a stream, and MUST NOT define its own obfuscation protocol.

Rationale. Obfuscation is a fast-cycle arms race that the Tor project already runs with obfs4, Snowflake, and WebTunnel. A protocol that hard-codes one method carries a defeated method for years.
Verification. Demonstration with obfs4: the client completes a deposit through the pluggable transport without protocol changes.

3.8.4 Relay link

A link opens with a transport handshake, then HELLO and HELLO_ACK. The acknowledgement carries the policy the relay enforces, and the client adapts to it.

SSP-RLY-001 A relay MUST state its full policy in HELLO_ACK, and MUST NOT enforce a limit it has not stated.

Rationale. A hidden limit produces a failure the client cannot diagnose and makes the network behave differently at each relay. G-01 needs the policy on the wire.
Verification. Test: a client drives each advertised limit to its stated value and one step beyond; every refusal cites a limit the policy declared, and no refusal cites an undeclared one.

Field	Type	Meaning
max_mailboxes	u32	Mailboxes one link may register.
mailbox_ttl	u32	Seconds until an unrenewed mailbox expires.
object_ttl	u32	Seconds until a stored object expires.
max_object_bytes	u32	Largest envelope.
default_quota	u32	Objects per mailbox per 24 hours.
max_quota	u32	Largest quota a mailbox may request.
blob_max_bytes	u64	Largest blob.
blob_ttl	u32	Seconds until a blob expires.
pow_algorithm	u16	`0x0001` for Equi-X.
pow_effort_intro	u32	Current effort for an introduction deposit.

Table 37. Relay policy block.

SSP-RLY-002 A relay MUST NOT request an identifier, an account, an email address, a payment token, or any value that persists across links.

Rationale. Every such value is a stable identifier linking the sessions of one device.
Verification. Inspection of the operation registry: no request field is retained across links, and no response carries a value the client is expected to present again.

Mailbox operations. Registration carries the address, the deposit public key, the quota in objects and bytes, a class, a lifetime, and a signature by the deposit secret proving that the registrant holds the key being registered.

SSP-RLY-003 A relay MUST reject a deposit to an address that no registration created.

Rationale. This single rule ends open relaying. A relay stores data only for a mailbox a subscriber asked for, so an attacker cannot make a relay store data by inventing addresses, and the relay cannot become a free disk for the internet.
Verification. Test: a deposit to a well-formed but never-registered address is refused with `MAILBOX_UNKNOWN`, and the relay stores nothing for it.

SSP-RLY-004 A relay MUST require a valid signature by the deposit key on registration, renewal, and deletion.

Rationale. Otherwise any party learning a mailbox address could delete the mailbox of the victim, which is a cheap denial-of-service attack.
Verification. Test: registration, renewal, and deletion each fail with `BAD_SIGNATURE` when signed by a key other than the registered deposit key.

The asymmetry is the point. The recipient registers the mailbox and proves it holds the deposit secret, then hands that secret to exactly one sender and keeps only the public key. From then on the sender can deposit, the recipient has a per-sender revocation switch through deletion, and the relay has a cheap check.

Fetch and subscribe. A subscription or a fetch names a set of mailboxes and a cursor per mailbox; the relay returns envelopes with a sequence number and a storage time, and deletes on acknowledgement.

SSP-RLY-005 A relay MUST assign a strictly increasing sequence number per mailbox and MUST NOT reuse a value.

Rationale. The cursor model needs it, and a gap tells a client that the relay dropped or deleted an object.
Verification. Test: 1000 deposits into one mailbox yield a strictly increasing sequence with no repeats; after a relay restart the next value still exceeds every prior value.

SSP-RLY-006 A relay MUST delete an object on a valid acknowledgement and MUST NOT retain a copy.

Rationale. The value of deletion is that a later seizure of the disk finds nothing. A relay that ignores this is inside T4, and no confidentiality property depends on the rule.
Verification. Demonstration on the reference relay: after an acknowledgement the object is absent from the store and from a raw disk image. The protocol does not depend on this, since a relay in class T4 may ignore it.

SSP-RLY-007 A client MUST acknowledge only after writing the plaintext to durable local storage.

Rationale. An acknowledgement before a durable write loses the message on a crash.
Verification. Test with crash injection between decryption and the durable write across 100 trials: no message is lost, and every message not written is redelivered.

3.8.5 What a relay knows

Table 38 is normative. An implementation that lets a relay learn more than the left column is non-conformant. The right column states what the protocol does not send, not what a relay is unable to infer: 3.11.6 covers inference from timing and volume, which no column of this table constrains.

A relay observes	Not exposed to a relay by the protocol
The IP address of the connecting client, absent an onion network or VPN	Which identity that client belongs to
The set of mailbox addresses one link registers or reads	Which peer, group, or conversation an address serves
That an object at a mailbox is a group commit, from its type class	The group identifier, the membership list, which other mailboxes belong to the same group, or the group size
The size class of each object, from the bucket field	The plaintext length inside the bucket
The type class of each object	The content type of the message
Deposit time, fetch time, object count	The identity of the depositor
The deposit key of a mailbox	The identity holding the matching secret
Blob chunk hashes and sizes	Blob plaintext, and the message referencing a blob
The identity logs a client asks it to serve	Nothing further; a log is public to any contact

Table 38. The relay knowledge boundary.

SSP-RLY-008 A relay MUST NOT record a mapping between an IP address and a mailbox address in persistent storage.

Rationale. An operator can break this rule, and the rule still has value: it defines conformance and gives an honest operator a clear instruction.
Verification. Inspection of the reference relay storage schema and write paths, plus a disk image search for any structure pairing an address with a network address.

SSP-RLY-009 A relay MUST NOT place a mailbox address in a log file, an access log, or a metrics label.

Rationale. Access logs and metrics series are retained for months, are frequently shipped to a third-party aggregator, and are the artifact most often produced under legal compulsion.
Verification. Inspection of the reference relay logging and metrics paths, plus a run at the most verbose level whose output contains no mailbox address.

3.8.6 Mailboxes

The core privacy mechanism is this: the network address of a recipient is not a property of the recipient, but of the pair.

Signal sends to a phone number, Matrix to a user on a homeserver, Session to a public key. In each case one identifier receives all traffic for one user and every sender uses it, so the operator can count the senders of a user, measure their traffic, and hold a subpoena-ready record. SSP derives a distinct address for each sender-device and recipient pair and rotates it daily. No two senders of the same recipient use the same address, and no address survives a day.

Derivation. At session setup both parties compute

$$mailbox\_root = \mathrm{KDF}\big(ss,\ \texttt{SSP1/mailbox-root},\ \mathrm{SID}_s \| dev_s \| \mathrm{SID}_r \| dev_r,\ 32\big)$$

and thereafter, with no exchange of messages,

$$\begin{aligned} e &= \lfloor t_{\mathrm{ms}} / 86\,400\,000 \rfloor \\ addr_e &= \mathrm{KDF}(mailbox\_root,\ \texttt{SSP1/mailbox-addr},\ e,\ 32) \\ seed_e &= \mathrm{KDF}(mailbox\_root,\ \texttt{SSP1/deposit-seed},\ e,\ 32) \\ (dsk_e, dpk_e) &= \mathsf{Ed25519.KeyGen}(seed_e) \end{aligned}$$

The recipient registers the next address one epoch ahead; the sender computes the same values and deposits.

SSP-MDR-001 A mailbox address MUST change at every epoch boundary, and the new address MUST NOT be computable from the old one without the mailbox root.

Rationale. An observer watching a relay for a year must not be able to link day 1 to day 365 for the same pair.
Verification. Test against the planned mailbox vectors: addresses for 30 consecutive epochs of one pair are pairwise distinct, and a party holding only the epoch-e address cannot produce the epoch-$e{+}1$ address.

SSP-MDR-002 A recipient MUST register the mailbox of epoch $e+1$ before the end of epoch e, and MUST keep the mailbox of epoch $e-1$ for a further 24 hours.

Rationale. Clock skew of a few minutes must not lose a message; a 24-hour overlap covers assumption A-02 with a wide margin.
Verification. Test: with the sender clock ten minutes fast and the recipient clock ten minutes slow, delivery across an epoch boundary succeeds in both directions.

SSP-MDR-003 A recipient SHOULD register the mailboxes of an epoch in random order and spread the registrations across the epoch.

Rationale. A relay receiving two hundred registrations in one burst learns that one client holds two hundred mailboxes. Spreading hides the count and costs link time. A client with a short online period cannot spread them and has to accept the burst, so this is a recommendation; 3.8.7 covers that case.
Verification. Demonstration: over a 24-hour period with 200 mailboxes, no relay observes more than 20 registrations in any 10-minute window, and the ordering shows no correlation with the order in which the peers were added.

SSP-MDR-004 A recipient device MUST use one mailbox per peer *identity* for the receive direction, not one per peer device.

Rationale. The devices of one peer already share that identity and already know the SID, so a per-device receive mailbox adds no unlinkability against a relay and multiplies the count by eight. The send direction still uses a per-device session, because the ratchet is per device.
Verification. Inspection of the derivation inputs: the receive-side mailbox root omits the recipient device identifier, so all devices of one peer resolve to one address.

With that rule a user with two hundred contacts holds two hundred receive mailboxes per device, plus one per group and one introduction mailbox.

3.8.7 Offline recipients

A mailbox address is derivable by both parties without any exchange, but a relay refuses a deposit to an address that no registration created (SSP-RLY-003). A recipient that stops registering therefore stops receiving, however correctly the sender behaves. This subsection states how far that gap is bridged, and where it is not.

Pre-registration. A recipient registers mailboxes for future epochs while it is online. Let W be the number of future epochs it registers beyond the current one. SSP-MDR-002 sets the floor at $W \geq 1$. The ceiling is a policy choice with real costs on both sides, recorded as O-05 in 1.3.

SSP-MDR-017 A recipient MUST register the mailbox for epoch $e+1$ before epoch e ends, and MAY register up to W further future epochs in one online period, subject to the mailbox count and lifetime limits the relay states in its policy.

Rationale. Registration for the next epoch is what makes ordinary delivery continuous. Registration further ahead buys offline tolerance and costs unlinkability, because the relay observes W addresses arriving from one link for one peer.
Verification. Test: a client goes offline for W epochs and returns; every message deposited during that period is present, and a message deposited in epoch $W+1$ was refused at deposit time.

SSP-RLY-015 A relay MUST state a mailbox lifetime in its policy, MUST accept a registration whose requested lifetime is within it, and MUST keep a registered mailbox until that lifetime expires even when the registering client does not reconnect.

Rationale. A pre-registered mailbox is worthless if the relay discards it because the owner is absent, which is exactly the case it exists for.
Verification. Test: a mailbox registered for 30 days accepts a deposit at day 29 after the registering link has been closed throughout, and refuses one at day 31.

Past the window. Once a recipient has been offline for more than W epochs, its future mailboxes are exhausted and deposits fail. Delivery is not queued at the relay, because there is no address to queue it under.

SSP-MSG-024 A sender that receives an unknown-mailbox error for a peer MUST keep the message queued locally, MUST NOT treat the error as a delivery failure or as evidence that the peer has left, and MUST NOT deposit the message to any address it derives for a different epoch than the one in progress.

Rationale. An unknown-mailbox error is indistinguishable from a peer that is merely offline. Depositing to a past or future epoch address would either land where the recipient no longer looks or create an address the relay can link to the same pair outside its intended epoch.
Verification. Test: the sender is given an unknown-mailbox error for six consecutive epochs; it retains the message, and every deposit attempt in that period carries the address of the epoch then in progress.

SSP-MSG-025 A sender MUST recompute the mailbox address at each retry from the epoch in progress at the moment of that retry.

Rationale. This is what makes recovery from a long absence automatic. The moment the recipient returns and registers the current epoch, the next scheduled retry addresses a mailbox that exists.
Verification. Test: a message is queued during epoch e and delivered during epoch $e{+}9$ after the recipient returns; the accepted deposit carries the address for $e{+}9$.

Retry continues for the queue lifetime of SSP-SYN-005, which is seven days. The tolerated absence is therefore bounded by both numbers: a recipient absent for longer than W epochs receives nothing deposited after its window closed, and a sender that has waited longer than the queue lifetime discards the message and reports it as undelivered. The protocol does not offer indefinite offline delivery, and a client SHOULD NOT present it as though it does.

Interaction with the other mailbox rules. Pre-registration is in tension with two rules that exist for privacy, and the tension is not resolved by either one.

What the relay learns. Pre-registration gives a relay one extra fact: that $W{+}1$ specific addresses were registered together and belong to one client. It does not tell the relay which peer they serve, and it does not link them to any identity. It does mean that an observer who compromises the relay during the window sees a set of addresses that will be active on known future dates, which is a longer-lived correlation handle than the daily rotation of SSP-MDR-001 suggests in isolation. 3.11.6 records the limit.

Rule	Interaction
Randomised registration order and spreading (SSP-MDR-003)	Spreading assumes the client is online across the epoch. A client with a short online period must register a batch, which is the burst the rule exists to avoid. A client SHOULD spread pre-registration across whatever online period it has, and SHOULD order the addresses randomly even when it cannot spread them in time.
Relay sharding (SSP-MDR-010)	The 60 per cent ceiling applies to the whole registered set, future epochs included. Pre-registering W epochs for every peer multiplies the set by $W+1$ without changing the ratio, so the ceiling still holds, and the absolute number of addresses one relay can attribute to one client grows by the same factor.
Mailbox rotation on abuse (3.10.1)	Rotating away from a hostile sender means deleting the pre-registered future mailboxes for that pair as well, so the cost of a rotation rises with W.

Table 39. Pre-registration against the privacy rules.

Axis	Effect of a larger W
Availability	Longer tolerated absence, up to the sender queue lifetime.
Relay storage	Registration records scale with W per peer per device; stored objects do not, since an absent recipient receives at most one epoch of traffic per epoch.
Client bandwidth	One registration per epoch per peer, paid in advance rather than daily. The total is unchanged; the burstiness is not.
Privacy	A larger set of addresses attributable to one client at one relay, and a forward-dated correlation handle.
Rotation cost	More mailboxes to delete when a pair is abandoned or a sender is blocked.

Table 40. Trade-off in the pre-registration window.

3.8.8 Relay storage and expiry

Class	Max object	Default expiry	Max expiry
Message	64 KiB	14 days	30 days
Introduction	8 KiB	7 days	7 days
Group commit	256 KiB	30 days	30 days
Blob notice	4 KiB	14 days	30 days
Receipt	1 KiB	3 days	7 days
Blob chunk group	1 MiB	7 days	14 days

Table 41. Storage classes at a relay.

SSP-RLY-010 A relay MUST delete an object when its expiry passes, and MUST NOT keep a copy or a hash of it.

Rationale. Retention past expiry converts a store-and-forward node into an archive, and an archive is what makes seizing the disk worthwhile.
Verification. Demonstration: an object at expiry plus one minute is absent from the store and from any index, verified against a raw disk image.

SSP-RLY-011 A relay MUST hold a group commit object for at least 30 days.

Rationale. A member offline for three weeks must still be able to reach the current epoch. Shorter retention forces a full rejoin, which costs the group a commit and leaks the absence of that member.
Verification. Test: a member offline for 29 days retrieves the commit and reaches the current epoch without a rejoin.

SSP-RLY-012 A relay MUST enforce a total byte limit per mailbox and reject a deposit beyond it with the reason `MAILBOX_FULL`.

Rationale. A per-object limit alone lets a sender fill a disk with many objects.
Verification. Test: deposits continue until the byte bound is reached, and the next is refused with `MAILBOX_FULL` while other mailboxes are unaffected.

SSP-RLY-013 A relay MUST NOT delete an unacknowledged object before its expiry to make space, and MUST reject the new deposit instead.

Rationale. Dropping the oldest object gives an attacker a way to push a message out of the mailbox of a victim: a targeted deletion attack.
Verification. Test: a mailbox at its byte bound holding unacknowledged objects refuses a new deposit; every object present before the attempt is still present after it.

3.8.9 Relay discovery

A relay record holds the Ed25519 relay key, the X25519 link key, a list of endpoints (IPv4, IPv6, DNS name, or onion address), flags for whether it accepts new mailboxes and blobs, and a signature over the record and a validity window.

A client learns of a relay in four ways and no others: a bootstrap list embedded at build time; a record the user pastes or scans; up to four records inside a contact code; or a signed relay directory the user chooses to follow. A directory is a signed, hash-linked document that anyone

may publish. It is not a protocol service, and the signature of the publisher is its only authority.

SSP-NET-012 A client MUST let the user replace the entire bootstrap relay list.

Rationale. A fixed bootstrap list is a central point a state adversary can block and a client author can abuse.
Verification. Demonstration: the user replaces every bootstrap entry with a private relay record, and the client contacts no address from the shipped list afterwards.

SSP-NET-013 A client MUST NOT trust a relay directory for anything beyond the existence and the address of a relay.

Rationale. A directory publisher must not be able to affect confidentiality. It can affect availability, and 3.8.10 states the consequence.
Verification. Analysis of the directory handling path: a directory entry can change which address is dialled and cannot change a relay key already learned from a contact code or an identity log.

No distributed hash table. A DHT would give automatic discovery and user lookup, and is rejected for both.

Problem	Detail
Sybil attacks	With no admission control, an attacker places nodes near a target key and owns the lookups for it. Every known defence needs a trusted party, proof of work, or a stake.
Enumeration	A DHT storing user records is a queryable list of every user.
Metadata	A lookup tells the nodes on the path which key the client wants: the social graph, published to strangers.
Complexity	A production DHT runs to thousands of lines with a long defect history in routing, traversal, and churn.

Table 42. Reasons against a DHT.

SSP therefore has no lookup by identifier. A contact code carries the relay hints and the identity log carries updates. If every relay of a contact disappears at once, that contact must send a new code out of band. This is a real loss of convenience and an accepted one.

SSP-MDR-005 The protocol MUST NOT define any operation mapping a human-visible value, a SID, or a hash of either, to a network address, a relay, or a mailbox.

Rationale. Every such operation is a user enumeration oracle. Rate limits do not fix it, because an attacker with a botnet has unlimited query budget. Signal built a hardware enclave for this problem and the result still depends on the enclave vendor. SSP removes the problem instead of managing it.
Verification. Inspection of the operation registry in Appendix C: no operation accepts an identifier or a hash of one and returns a location.

3.8.10 Relay properties

SSP-RLY-014 A relay MUST hold no state that another relay cannot recreate from client actions.

Rationale. A user must be able to leave a relay in one minute, losing only the objects in flight. This is what keeps an operator honest and stops a relay becoming a platform.
Verification. Demonstration: a client migrates its whole mailbox set to a fresh relay and resumes delivery, losing only objects in flight at the moment of the change.

The useful question is not whether a relay is trustworthy but what it gains by being hostile. Table 43 answers it.

Compromise	Effect
The operator reads the disk	Ciphertext, opaque addresses, deposit times. No content and no identities.
The operator seizes a running process	The same, plus live IP addresses and the mailbox sets of current links.
The relay drops every object	Denial of service for its users. Two relays are required, so one relay dropping does not stop delivery.
The relay serves a forked identity log	Detected: clients cross-check two relays and gossip heads.
The relay lies about its policy	The client sees failures and moves.
The relay colludes with a network observer	The T5 and T6 combination; 3.11.6 states the limit.

Table 43. Effect of relay compromise.

No federation. A relay never talks to another relay.

Reason	Detail
No routing state	Federation needs a routing table, and a routing table is a map of where users are.
No amplification	A relay forwarding to another relay is a reflector and a flood vector.
No trust edges	Each relay would have to decide which relay to trust, recreating the reputation problem that 3.10 rejects.
Simplicity	The sender already knows the relay set of the recipient; a forward adds a hop for nothing.
Evidence	Matrix federation replicates room state to every participating server, multiplying metadata exposure by the server count.

Table 44. Reasons against federation.

The cost is that a sender must reach the relay of the recipient. Where a censor blocks that relay, the sender uses binding B with a pluggable transport.

3.9 Messaging

3.9.1 Asynchronous key agreement

SunSpot asynchronous key agreement (SAKA) establishes a session between two devices while the recipient is offline. It follows the shape of PQXDH and uses the SSP primitive set.

Prekey bundle. A device publishes a bundle to a prekey mailbox at each of its relays, named by a PrekeyPointer entry in its identity log.

Field	Type	Lifetime
device id	[8]	long term
identity key	Ed25519 [32]	long term
link key	X25519 [32]	180 days
KEM key	ML-KEM-768 [1184]	180 days
signed prekey	X25519 [32]	7 days
one-time prekey	X25519 [32]	single use
one-time KEM key	ML-KEM-768 [1184]	single use
signatures	Ed25519 [64]	over each element and over the whole bundle

Table 45. Prekey bundle.

SSP-MSG-004 A relay MUST give each one-time prekey to at most one requester, and MUST delete it once given.

Rationale. The one-time key is what makes the first message forward-secret. A relay serving the same key twice destroys that property for both senders.
Verification. Test: two concurrent requests for one prekey receive different keys, and the second receives none when the store holds one.

SSP-MSG-005 A sender MUST verify every signature in a bundle against the device key held in the identity log of the recipient, before using any value from it.

Rationale. The bundle arrives from an untrusted relay. Without the check, the relay chooses the keys and reads everything.
Verification. Test: a bundle with a valid outer signature and a substituted signed prekey is rejected.

The agreement. The initiator holds an ephemeral EK_i and its static link key L_i; the responder has published SPK_r, an optional one-time OPK_r, its static link key L_r, a static KEM key, and an optional one-time KEM key.

$$dh_1 = \mathsf{X25519}(EK_i, SPK_r) \qquad dh_2 = \mathsf{X25519}(L_i, SPK_r)$$
$$dh_3 = \mathsf{X25519}(EK_i, L_r) \qquad dh_4 = \mathsf{X25519}(EK_i, OPK_r) \quad \text{if present}$$
$$(c_1, k_1) = \mathsf{MLKEM768.Encaps}(KEM_r) \qquad (c_2, k_2) = \mathsf{MLKEM768.Encaps}(OKEM_r) \quad \text{if present}$$

$$T = H\big(\texttt{SSP1/saka-transcript} \,\|\, \mathrm{SID}_i \| dev_i \| \mathrm{SID}_r \| dev_r \,\| EK_i \| SPK_r \| OPK_r \| KEM_r \| OKEM_r \| c_1 \| c_2\big)$$

$$ss = \mathrm{KDF}\big(dh_1 \| dh_2 \| dh_3 \| dh_4 \| k_1 \| k_2,\ \texttt{SSP1/saka-root},\ T,\ 64\big)$$

The initial message carries the ephemeral public key, the KEM ciphertexts, the identifiers of the prekeys used, and the first ratchet message. It goes to the introduction mailbox of the recipient, or to an existing mailbox if the pair has met before.

SSP-MSG-006 SAKA MUST include dh_2, which uses the static link key of the initiator.

Rationale. dh_2 authenticates the initiator cryptographically. Without it, any party could open a session as any sender, and the recipient would need a signature over the whole message, which would destroy deniability.
Verification. Analysis of the agreement, plus a test that a session established without this component accepts an initial message from a party that does not hold the initiator link key.

SSP-MSG-007 SAKA MUST NOT sign the initial message with the device key of the sender.

Rationale. Deniability. Authentication comes from dh_2, which both parties can compute, so neither can prove authorship to a third party. A signature would be a transferable proof and would turn every message into evidence.
Verification. Inspection of the initial message: it carries no signature over its content by the sender device key, and the recipient authenticates it solely through the agreement.

SSP-MSG-008 The transcript hash MUST cover every public value of the handshake, and the session secret MUST bind to it.

Rationale. Downgrade protection and resistance to unknown key share.
Verification. Test: altering any public handshake value changes the derived secret and the session fails to establish.

X3DH alone has no post-quantum component. Noise has excellent interactive patterns, which SSP uses at the relay link and at enrolment, and it does not cover the asynchronous case where the responder is offline and a third party holds its prekeys. SAKA fills that gap and borrows the transcript discipline of Noise.

3.9.2 Double ratchet

Parameter	Value
DH function	X25519
KDF	BLAKE3 keyed mode
AEAD	ChaCha20-Poly1305
Nonce	The 12-byte big-endian chain index, with eight leading zero bytes
Root step	$\mathrm{KDF}(\text{root}, \texttt{SSP1/ratchet-root}, dh, 64)$ into a new root and chain
Chain step	$\mathrm{KDF}(\text{chain}, \texttt{SSP1/ratchet-chain}, i, 64)$ into the next chain and a message key
Header key	Derived per chain; headers are encrypted
Skipped keys	1000 per chain, 2000 per session
Chain length before a forced DH step	100

Table 46. Ratchet parameters.

SSP-MSG-009 A client MUST use the header encryption variant of the double ratchet.

Rationale. A cleartext header carries the ratchet public key and the counters. That key changes per step and would let a party reading two envelopes link them to one session. The header sits inside the mailbox ciphertext, so this defends against a compromised recipient store rather than the relay, at negligible cost.
Verification. Test: two envelopes from one session carry no common plaintext bytes in the header region, and the ratchet public key is not recoverable from the envelope.

SSP-MSG-010 A client MUST store at most 1000 skipped message keys per chain and drop the oldest beyond that.

Rationale. An attacker sending a header with a counter of 2^{31} would make a naive client derive two billion keys. The limit turns that attack into a lost message.
Verification. Test: a header claiming a counter of 2^{31} stores at most 1000 skipped keys and completes in bounded time.

SSP-MSG-011 A client MUST perform a DH ratchet step on the first message it sends after receiving one, and after 100 messages in one chain even with no reply.

Rationale. The DH step is what delivers post-compromise security. A long one-sided chain leaves a compromise open for its whole length.
Verification. Test: a step occurs on the first message after a receive, and after 100 messages in a one-sided chain; a compromise at message 50 does not read message 201.

SSP-MSG-012 A client MUST delete a message key immediately after use, and a skipped key after seven days.

Rationale. Forward secrecy is a property of deletion, not of the algorithm.
Verification. Test: a memory scan after a message is read finds no copy of its key; a skipped key is absent from the store after seven days.

3.9.3 Message object

The header carries a domain label, the version, the sender SID and device, a conversation identifier, the message identifier, a per-sender counter, a timestamp, one to eight parent identifiers, the current identity log head of the sender, and flags. The body carries a content type and a payload, followed by an extension list.

SSP-MSG-013 A message identifier MUST be the BLAKE3 hash of the canonical encoding of the message with the identifier field set to zero.

Rationale. Content addressing. The identifier becomes verifiable and no party can choose it; a random identifier would let a sender collide two messages deliberately.
Verification. Test: recomputing the hash over a received message with the identifier field zeroed reproduces the identifier, over the planned message corpus.

SSP-MSG-014 A recipient MUST reject a message whose sender SID and device do not match the session that decrypted it.

Rationale. The ratchet supplies authentication; this check stops a confused sender identity inside a valid session.
Verification. Test: a message whose header names a different sender than the session is rejected, even when the AEAD tag verifies.

SSP-MSG-015 A recipient MUST treat the timestamp as advisory, MUST NOT order by it, and MUST reject a message timed more than 24 hours in the future.

Rationale. The clock of the sender is not trusted, and the rejection window stops a message that would sort to the top of a list for ever.
Verification. Test: a message timestamped 25 hours ahead is rejected; one timestamped 23 hours ahead is accepted and does not sort above later messages with correct clocks.

3.9.4 Ordering and convergence

SSP gives causal consistency with eventual convergence, and no total order. No server can supply one, and a consensus mechanism is a non-goal.

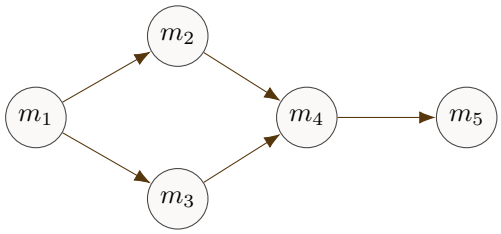

Figure 12. Messages form a directed acyclic graph. Each message names the heads its sender had seen.

SSP-MSG-016 A sender MUST name as parents every current head of its local view of the conversation, up to eight.

Rationale. The parent set proves what the sender had seen. It makes causality explicit and verifiable and lets a recipient detect a gap.
Verification. Test: over a simulated conversation, every sent message names exactly the head set the sender held, and a recipient detects an omitted parent as a gap.

SSP-MSG-017 Where a conversation has more than eight heads, a sender MUST choose the eight with the highest identifiers in byte order.

Rationale. A deterministic rule means two clients in the same state produce the same parent set, which helps deduplication and testing.
Verification. Test: two clients holding the same 12-head state independently select the same eight parents.

SSP-MSG-018 A client MUST display messages in a topological order of the graph, breaking ties between concurrent messages first by the lower value of $\lfloor sent\_at/60000 \rfloor$, then by the lower identifier in byte order.

Rationale. The minute bucket uses the sender clock where it is plausible, which matches user expectation, and the hash tie-break makes the result identical on every device, which is what convergence needs. A pure hash order would scramble a conversation; a pure timestamp order would let a sender with a wrong clock control the view.
Verification. Test: 1000 random delivery permutations of one graph produce one display order on every client.

SSP-MSG-019 A client MUST NOT hide a message because a parent is missing, and MUST show the gap.

Rationale. A relay inside T4 can drop a message. A client that waits for the parent for ever hands the attacker a silent censorship tool; a visible gap tells the user.
Verification. Demonstration: with a parent withheld by the relay, the message is displayed and the gap is visible to the user.

Mutable state. Different state needs different rules, fixed here so that two clients converge.

State	Rule	Reason
Message body	Immutable	It is content-addressed.
Edit	Last writer wins, among edits by the original sender only	An edit by another party is a forgery.
Delete for me	Local only, never on the wire	A display choice.
Delete for everyone	A request, not a command; the client hides the content and keeps the tombstone	A recipient device can ignore it and the protocol must not pretend otherwise.
Reaction	Add-wins set keyed by sender and emoji	Concurrent add and remove converge to add, matching what a user sees.
Profile	Last writer wins per identity	One value, one owner.
Group name and topic	Last writer wins per epoch; a commit resets it	The epoch is a natural ordering point.
Read state	Grow-only counter per conversation per device	Read state only moves forward.

Table 47. Convergence rules for mutable state.

SSP-MSG-020 A delete-for-everyone request MUST NOT remove the tombstone, and a client MUST show that a message existed and was withdrawn.

Rationale. An honest client cannot prove it deleted the content, and a protocol promising deletion makes a promise TX-03 excludes. A visible tombstone gives the user the truth.
Verification. Demonstration: after a delete-for-everyone request the body is hidden and a visible marker remains, and the marker survives a restart.

3.9.5 Delivery and receipts

Deposited means a relay stored the envelope; delivered means a recipient device decrypted the message; read means the user saw it.

SSP-MSG-021 A delivery receipt MUST travel end-to-end as a message, and MUST NOT come from a relay.

Rationale. A relay receipt would be a lie, because a relay cannot know that a device decrypted anything, and it would give the relay a reason to hold a mapping.
Verification. Inspection of the receipt path: receipts are produced only by a recipient device after decryption, and no relay operation emits one.

SSP-MSG-022 A client MUST NOT send a read receipt unless the user enabled it for that conversation.

Rationale. A read receipt is precise activity metadata sent to a party who may not deserve it.
Verification. Test: with read receipts disabled for a conversation, no receipt object of that type is deposited, verified by a capture over 100 read messages.

SSP-MSG-023 Delivery MUST be at-least-once, and a recipient MUST deduplicate by message identifier.

Rationale. Deposits at two relays are normal, so duplicates are normal.
Verification. Test: the same message delivered through two relays is displayed once, and the duplicate is discarded by identifier.

3.9.6 Groups

A group uses TreeKEM from RFC 9420 with the SSP primitive set, and without the MLS delivery service and authentication service.

MLS component	SSP replacement
Delivery service ordering handshake messages	None. 3.9.7 resolves concurrent commits deterministically.
Authentication service with credentials	The identity log. A leaf holds a SID and a device id, verified against that log.
Ciphersuite registry	One suite.
Public messages on a shared channel	Per-pair mailboxes, so a relay never sees a group.

Table 48. Divergence from MLS.

The suite is TreeKEM over X25519 with ML-KEM-768 combined as in 3.7.12, BLAKE3 as KDF and hash, ChaCha20-Poly1305 as AEAD, and Ed25519 for signatures.

SSP-GRP-001 A group MUST use exactly one ciphersuite, the one above.

Rationale. A group whose members disagree on the suite cannot derive a common epoch secret, so a second suite is a second, incompatible group protocol.
Verification. Inspection: the suite identifier is a constant, and no code path selects a group primitive from a negotiated value.

SSP-GRP-002 A leaf of the tree MUST be a device, not an identity.

Rationale. A device holds the private key, so the tree must reach the device. A leaf per identity would require the devices of that identity to share a key, which SSP-DEV-003 forbids.
Verification. Inspection of the leaf construction, plus a test that a commit naming an identity rather than a device is rejected.

SSP-GRP-003 A group MUST NOT hold more than 1024 identities or 4096 devices.

Rationale. Commit size grows with the logarithm of the tree and mailbox fan-out grows linearly. At 4096 leaves a commit costs about twelve encrypted path secrets per leaf and one message costs 4096 deposits, which is the upper bound 3.12 accepts. Larger audiences use a channel.
Verification. Test: a commit that would exceed 1024 identities or 4096 devices is rejected, and the group remains at its prior epoch.

SSP-GRP-004 A member MUST send an update proposal for its own leaf at least every seven days, and MUST commit at least every 30 days while the group is active.

Rationale. Post-compromise security in a group needs key refresh. Without a rule, a quiet group holds one secret for years and a single old compromise reads everything.
Verification. Test: a simulated group runs for 90 days; each member issues an update within every 7-day window and a commit occurs within every 30-day window.

SSP-GRP-005 A commit that removes a member MUST replace every path secret from that leaf to the root.

Rationale. This is the TreeKEM property that gives forward silence after removal.
Verification. Test: a removed member holding its former leaf secret cannot decrypt any message of the following epoch, over 1000 random membership sequences.

SSP-GRP-006 A member MUST check that the leaf credential of every other member matches a current device in the identity log of that member, and MUST reject a commit adding a leaf for a revoked device.

Rationale. The tree carries key material and the log carries the authority. A tree that disagrees with a log is a compromise.
Verification. Test: a commit adding a leaf whose credential names a device revoked in that member's identity log is rejected, and the group stays at its prior epoch.

What group traffic exposes.

SSP-GRP-007 A group message MUST travel to each recipient device through the pairwise mailbox of 3.8.6, and MUST NOT use a shared group mailbox.

Rationale. A shared mailbox would show a relay the exact membership set, the size, and the activity of the group, which is asset 3. The cost is fan-out: one message becomes N deposits.
Verification. Test: a group of eight members exchanges 100 messages against an instrumented relay; the relay log contains no address that receives traffic from more than one member pair, and no two addresses in the group correlate by deposit count.

Fan-out is the price of the property. A group of fifty members with two devices each costs one hundred deposits of equal size; at the 4 KiB bucket that is 400 KiB of upload for one message. This is decision O-01 in 1.3, and 6 names the research that would improve it.

The property this buys is bounded, and the bound is worth stating precisely. A relay does not learn the group identifier, the membership list, which of the mailboxes it holds belong to the same group, or the number of members. It does learn that a given mailbox carries objects of the group commit type, which places the owner of that mailbox in some group. An adversary that also observes deposit timing across many mailboxes may correlate the fan-out of a single group message into a candidate set, because those deposits are simultaneous and equally sized. Pairwise mailboxes hide the membership set; they do not provide membership indistinguishability against a relay that performs timing analysis. 3.11.6 records this as a residual limit.

3.9.7 Concurrent commits

Two members may commit at the same epoch. MLS solves this with a delivery service that orders commits; SSP has none and needs a deterministic rule.

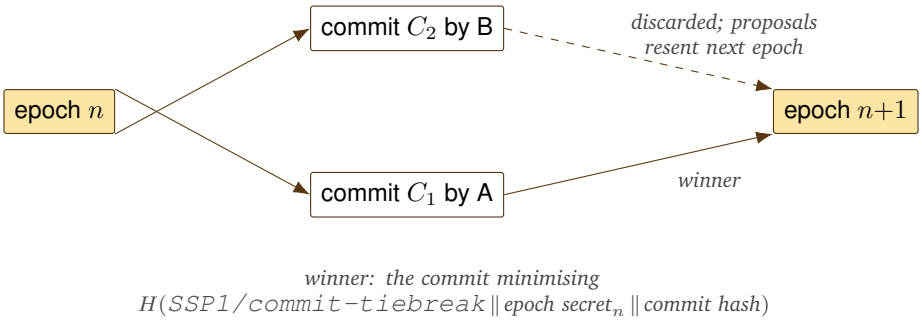

Figure 13. Deterministic resolution of concurrent commits.

SSP-GRP-008 A member MUST resolve concurrent commits at one epoch by the lowest value of $H(\texttt{SSP1/commit-tiebreak} \| epoch\_secret_n \| commit\_hash)$.

Rationale. The rule is deterministic, so every member converges. Including the epoch secret means a committer cannot grind a hash to win, because the value depends on a secret that is common to all members and controlled by none.
Verification. Test: 1000 random concurrency traces resolve to one winning commit on every member, and no member can bias the outcome by grinding its commit.

SSP-GRP-009 A member MUST retain epoch n state until it has seen no new commit for that epoch for 60 seconds, and MUST remain able to decrypt a message sent under epoch n for seven days.

Rationale. A member deleting the old key at once cannot read a message that crossed the epoch boundary. Seven days bounds the forward secrecy loss.
Verification. Test: a message sent under epoch n and delivered on day 6 decrypts; the same message delivered on day 8 does not, and the failure is reported rather than silently dropped.

SSP-GRP-010 A member whose commit lost MUST resend its proposals in the next epoch, and MUST NOT resend a proposal the winning commit already applied.

Rationale. A losing committer that stays silent loses its proposals altogether, and one that resends everything replays changes the winner already applied.
Verification. Test: two members commit concurrently; the loser resends exactly the proposals absent from the winning commit, and no proposal is applied twice.

The cost is one wasted round trip when two members commit at the same moment, which is rare outside a burst of membership change. The gain is that no party orders the group, and therefore no party can censor a commit by refusing to order it.

3.9.8 Channels

A channel is a one-to-many stream with one publisher and any number of subscribers. It exists because a group caps at 1024 identities and because a broadcast has different privacy needs.

Property	Group	Channel
Membership confidentiality	Hidden from relays and non-members	The publisher knows every subscriber
Who may send	Any member	The publisher only
Key structure	TreeKEM tree	A forward-ratcheted chain of epoch keys
Member limit	1024 identities	No protocol limit
Forward secrecy	Per message and per epoch	Per epoch, 24 hours by default
Removal	A commit rekeys the tree	The publisher stops issuing the next epoch key

Table 49. Groups compared with channels.

SSP-GRP-011 A channel epoch key MUST be derived as $k_{n+1} = \mathrm{KDF}(k_n, \texttt{SSP1/channel-epoch}, \mathit{channel\_id} \,\|\, n{+}1, 32)$, and the publisher MUST issue k_{n+1} only to the subscribers it keeps.

Rationale. A dropped subscriber holds k_n and the KDF is one-way, so it computes nothing forward. A joining subscriber receives only k_n and cannot read epoch $n-1$.
Verification. Test: a subscriber holding k_n cannot derive k_{n+1}; a subscriber joining at epoch n cannot decrypt an object from epoch $n-1$.

SSP-GRP-012 A client SHOULD show the user that a channel does not hide the subscriber list from the publisher.

Rationale. A channel and a group look alike in a user interface and differ in exactly this property. The obligation is on presentation, so it is a recommendation.
Verification. Demonstration: the subscription surface for a channel states that the publisher learns the subscriber list, and the equivalent surface for a group does not.

3.9.9 Attachments

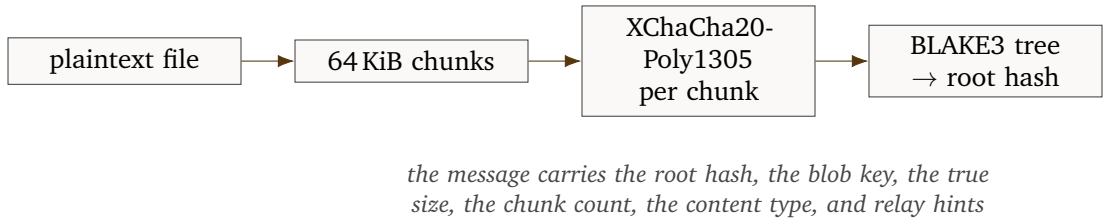

Figure 14. Attachment pipeline.

SSP-ATT-001 A blob MUST use a fresh random key.

Rationale. A shared key lets a party holding one key read others, and it breaks the deduplication privacy of SSP-ATT-005.
Verification. Test: 1000 blobs produced by one client have pairwise distinct keys, and no key is derived from the file content.

SSP-ATT-002 A recipient MUST verify each chunk against the root hash with the BLAKE3 tree proof, before decrypting it and before writing it to storage.

Rationale. This is the property BLAKE3 supplies and a flat hash cannot. A recipient downloading 100 MiB from a hostile relay detects a corrupt or malicious chunk at once rather than after the whole transfer.
Verification. Test: a corrupted chunk at the first, middle, and last position is each detected before decryption and before any byte reaches storage.

SSP-ATT-003 A client MUST pad the final chunk to 64 KiB with random bytes and carry the true size inside the encrypted message.

Rationale. The final chunk size would otherwise leak the exact file size modulo 64 KiB, and an exact size is a strong fingerprint of a known file.
Verification. Test: files of 1 byte and of 64 KiB minus 1 byte both produce a final chunk of exactly 64 KiB, and the recipient recovers the true length.

SSP-ATT-004 A blob MUST NOT exceed 100 MiB.

Rationale. A relay must bound its disk cost and a client its retry cost. A larger transfer belongs to a file transfer application, and a message protocol should not become one.
Verification. Test: a blob of 100 MiB is accepted and one byte larger is refused, at both sender and relay.

SSP-ATT-005 A client MUST NOT deduplicate a blob across recipients by reusing the root hash and key.

Rationale. Deduplication across recipients gives a relay a matching identifier in two mailboxes, linking two conversations. The bandwidth saving is not worth the link. Deduplication inside one conversation is permitted, where the relay already sees one mailbox.
Verification. Test: the same file sent to two recipients produces two distinct root hashes and two distinct keys.

SSP-ATT-006 A client MUST strip image, audio, and video metadata before sending, unless the user disables it for that file.

Rationale. EXIF holds location, device model, and time. This is a protocol requirement rather than a product choice, because a client that omits it harms the recipient as well as the sender.
Verification. Test: an image carrying GPS and device tags is transmitted with those tags absent, verified by parsing the received file.

3.9.10 Multi-device synchronisation

The devices of one identity form a self group: an ordinary group in which every device of the identity is a member and no other identity is. A DeviceAdd entry triggers an add proposal and a commit; a DeviceRemove triggers a remove proposal and an immediate commit.

SSP-SYN-001 A client MUST mirror every outgoing message, read state change, and contact change to the self group.

Rationale. Multi-device consistency with no server-side account. The self group offers the same forward secrecy and removal semantics as any group.
Verification. Test with three devices: a message sent from one appears on the other two, and read state converges within one synchronisation interval.

SSP-SYN-002 A client MUST NOT send plaintext state to a relay for synchronisation, and MUST NOT use a relay as a state store beyond the envelope model.

Rationale. A synchronisation store at a relay is an account, and an account is a central point holding long-lived data.
Verification. Inspection of every relay operation invoked during synchronisation: each carries only envelopes, and none stores an unencrypted state object.

SSP-SYN-003 History transfer to a new device MUST be device-to-device, over the enrolment channel or a fresh self-group session.

Rationale. N-04. A server-side history store needs a long-lived key, which destroys forward secrecy for the whole archive. The transfer is a one-time cost at enrolment and can move gigabytes over a local network.
Verification. Demonstration: a new device receives history over the local pairing channel, and no relay observes any history object during the transfer.

SSP-SYN-004 A user MUST be able to choose how much history to transfer, with at least none, the last 30 days, and everything.

Rationale. A new device is a new copy of the archive, and the user must be able to limit the exposure.
Verification. Demonstration: each of the three options transfers the stated range, and the device holds nothing outside it afterwards.

SSP-SYN-005 A client MUST queue an outgoing message when no relay of the recipient is reachable, retry with exponential backoff, and MUST NOT drop it before seven days pass.

Rationale. A message discarded on the first failure is lost to an ordinary offline period, and one retried for ever accumulates unbounded state on the sender.
Verification. Test: with every relay unreachable for six days, the message is still queued and is delivered on restoration; at eight days it is discarded and reported.

SSP-SYN-006 A client MUST NOT change the parent set of a queued message when it retries.

Rationale. The parent set records what the sender had seen at the moment of writing; updating it on retry would rewrite causality.
Verification. Test: a message queued for six days and delivered afterwards carries the parent set held at composition time, not the heads current at delivery.

3.9.11 Local storage

SSP-STO-001 A client MUST encrypt every persistent store with a key derived from the device master key.

Rationale. An unencrypted store places every past message outside the protection of the ratchet, and device seizure is the most common realistic threat to most users.
Verification. Test: a raw scan of the store finds no plaintext message body or key material.

SSP-STO-002 The device master key MUST be wrapped by the operating system key store, or by an Argon2id derivation from a user passphrase, and MUST NOT be stored in plaintext.

Rationale. A master key stored in the clear beside the data it protects provides no protection at rest at all.
Verification. Inspection of the key loading path on each platform, plus a scan of the application data directory for the unwrapped key.

SSP-STO-003 A client MUST implement crypto-erase, where deleting a conversation deletes its keys, and MUST NOT depend on the file system to erase data.

Rationale. A modern file system and a solid-state disk do not overwrite in place. The only reliable deletion is deletion of the key.
Verification. Demonstration against a raw disk image: after deleting a conversation, its keys are absent and its stored ciphertext is not decryptable by the client.

SSP-STO-004 The local store MUST hold an append-only event log, and every view MUST be a materialised projection of it.

Rationale. The message graph is the truth. A store holding only the current view cannot rerun the ordering rule when a late message arrives, and cannot converge.
Verification. Test: a message arriving after its successors is inserted and the view is recomputed to the same order two clients reach independently.

SSP-STO-005 A client SHOULD support a retention policy per conversation, offering at least: keep everything, keep one year, keep 30 days, and keep nothing after reading.

Rationale. The best defence against device seizure is to hold less data. The specific set of intervals is a product decision, so this is a recommendation; SSP-STO-003 carries the hard rule that deletion is real when it happens.
Verification. Demonstration: with the 30-day policy, messages older than 30 days and their keys are absent from a raw disk image.

3.10 Abuse prevention

SSP replaces reputation with capability. A relay accepts a deposit only into a mailbox a recipient created, and only under a signature by the deposit key that recipient chose. The recipient gives that key to one sender. Spam from an unknown party is refused by the relay before it reaches the recipient, except at the one place where a stranger has to be able to reach a user.

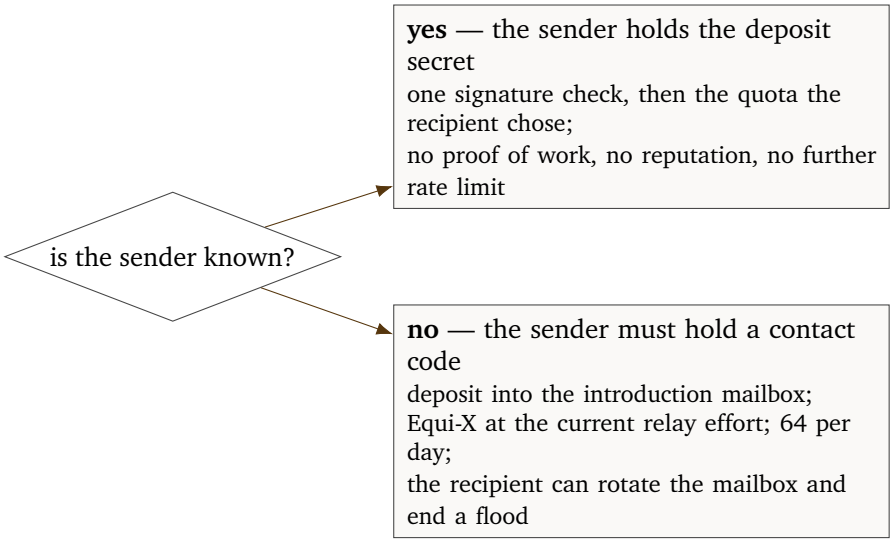

Figure 15. Admission decision at a relay.

3.10.1 Known senders

SSP-ABU-001 **Withdrawn.** This duplicated SSP-MSG-001, which states the same obligation at the point where the envelope is defined. The identifier is retained and not reused.

Rationale. Two identifiers for one obligation let an implementation satisfy one and cite the other, and make the traceability matrix overstate coverage.
Verification. Not applicable. SSP-MSG-001 carries the verification.

SSP-ABU-002 A recipient MUST set a quota on each mailbox at registration, in objects per 24 hours and in total bytes, and a relay MUST enforce both.

Rationale. The recipient knows how much traffic a contact should send; the relay enforces the number without knowing who either party is.
Verification. Test: a mailbox is registered with a quota of ten objects and 40 KiB. The eleventh object is refused on count, and a sequence of nine 8 KiB objects is refused on bytes before the count is reached.

SSP-ABU-003 A relay MUST answer a full quota with an explicit rejection carrying the reason and the time of the next reset, and MUST NOT drop silently.

Rationale. A silent drop looks like a network failure, so the sender retries for ever and wastes the resources of both parties.
Verification. Test: a sender exceeds a quota and receives a rejection carrying `QUOTA_EXCEEDED` and a reset time in the future; no deposit in that window is silently discarded.

SSP-ABU-004 A recipient that wants to stop a sender MUST delete that mailbox at every relay in its relay set.

Rationale. This is the block operation: instant, needing no server policy and no report to anyone. The blocked sender learns only that the mailbox is gone, which is indistinguishable from a relay change.
Verification. Test: after the recipient deletes the mailbox at both relays, a deposit by the blocked sender is refused with `MAILBOX_UNKNOWN` at each, and the recipient's other correspondents are unaffected.

This yields a property no reputation system can offer: a block enforced by an untrusted relay, with no knowledge of who is blocked, no appeal process, and no operator judgement.

3.10.2 Unknown senders

An introduction mailbox is the only place a party with no prior relationship can deposit. It is long-lived, it appears in the contact code, and it is the whole spam surface of the protocol. Three mechanisms protect it.

The contact code is a secret. An attacker cannot enumerate introduction mailboxes, because SSP-MDR-005 removes every lookup. The attacker must hold a code the user issued, which alone removes the mass-spam model of email and SMS.

Proof of work.

SSP-ABU-005 A relay MUST require a valid Equi-X solution for every introduction deposit, and MUST publish the current effort.

Rationale. The introduction mailbox is the only address a stranger can write to, so it is the only place where the cost of sending has to be raised deliberately.
Verification. Test: an introduction deposit with no solution, with a solution for a different challenge, and with a solution below the advertised effort are all refused; a correct solution is accepted once and refused on replay.

The challenge is $H(\texttt{SSP1/pow} \| relay\_key \| mailbox \| e \| relay\_seed)$, and the solver finds a 16-byte solution that Equi-X accepts and whose hash falls below the target for the current effort.

Property	Detail
Asymmetric cost	A solution takes roughly five milliseconds of CPU and a few megabytes; verification takes about fifty microseconds. A relay under attack does not spend more than the attacker.
Memory hardness	Resists a hardware advantage, so a botnet of commodity CPUs has no worse ratio than a specialist attacker.
Deployment record	Tor runs it against funded denial-of-service attacks on onion services.
Implementation	A reviewed Rust implementation exists inside the Arti project.

Table 50. Why Equi-X.

A hashcash-style SHA-256 puzzle is rejected because specialist hardware gives an attacker a factor of 10^5 over a phone. Argon2id as a puzzle is rejected because verification costs as much as solution, which makes the relay the victim. A verifiable delay function is rejected because it needs a setup and gives no memory hardness.

SSP-ABU-006 The effort MUST adapt to relay queue pressure, MUST NOT exceed a solution time of ten seconds on a 2020 phone, and MUST return to the base effort within ten minutes after the pressure ends.

Rationale. A fixed effort is either useless under attack or hostile in normal operation. The cap keeps a legitimate first contact possible under attack; without it an attacker raises the effort until real users cannot join.
Verification. Test with a simulated flood.

SSP-ABU-007 A relay MUST enforce a hard limit of 64 introduction objects per introduction mailbox per 24 hours, which the recipient MUST NOT be able to raise above 256.

Rationale. A user does not receive more than a few introductions a day. A ceiling the recipient cannot remove stops an attacker who compromises the client from opening the flood gate.
Verification. Test: the sixty-fifth introduction in 24 hours is refused; a registration requesting a cap of 512 is clamped to 256 by the relay.

SSP-ABU-008 A client MUST let the user rotate the introduction mailbox in one action, which invalidates every contact code issued earlier.

Rationale. The recovery action when a code becomes public. It costs the user the need to reissue codes to people who have not used theirs yet.
Verification. Demonstration: the user rotates the introduction mailbox; a deposit using a contact code issued before the rotation is refused, and a code issued after it succeeds.

3.10.3 Relay resource limits

SSP-ABU-009 A relay MUST apply a token bucket per link and per source prefix, at /64 granularity for IPv6 and /24 for IPv4.

Rationale. Per-address limits are useless against IPv6, where one host holds a large prefix.
Verification. Test: 64 connections from distinct addresses inside one IPv6 /64 are limited as one bucket, and connections from two different /64 prefixes are limited independently.

SSP-ABU-010 A relay MUST limit the mailboxes one link may register, MUST state the limit, and MUST NOT set it below 512.

Rationale. A normal user needs a few hundred. A lower limit breaks a normal client and no limit lets one link exhaust the address table.
Verification. Test: a relay advertising the minimum accepts 512 registrations on one link and refuses the 513th with `RELAY_FULL`; a relay advertising fewer than 512 fails conformance.

SSP-ABU-011 A relay MUST require an Equi-X solution for registration when one link exceeds four registrations per minute.

Rationale. Mailbox creation is the cheapest state-creating operation and needs a cost under pressure. The threshold does not affect a normal client, which registers a few mailboxes a day and spreads them under SSP-MDR-003.
Verification. Test: a link registering four mailboxes per minute is not challenged; at five per minute the relay issues a challenge and refuses further registrations until it is solved.

SSP-ABU-012 A relay MUST bound the disk one mailbox uses and the disk all mailboxes use, and MUST refuse new registrations as the second bound approaches.

Rationale. Without both bounds, one mailbox can fill a disk, and many mailboxes each inside their own bound can do the same collectively.
Verification. Test: a relay configured with a small total bound refuses new registrations once the bound is approached, while continuing to accept deposits into mailboxes that already exist and are inside their own quota.

SSP-ABU-013 A relay MUST NOT let an unauthenticated party cause more than one signature verification and one hash per received frame before that frame passes the length and rate checks.

Rationale. A signature check is the most expensive step, and an attacker must not be able to trigger many of them cheaply.
Verification. Analysis of the relay code path, and a load test.

3.10.4 Sybil resistance

SSP needs none, and that is a design outcome rather than an omission. A Sybil attack pays when identity count buys something: a vote, a reputation score, a share of a resource, or a place in a routing table. An SSP identity gets none of these. It gets no rights at a relay, because a relay does not know identities. It gets no access to a user, because access needs a contact code the user issued. It gets no position in a network structure, because there is no DHT and no federation.

SSP-ABU-014 The protocol MUST NOT grant any privilege, priority, or resource share on the basis of the number, age, or connectedness of an identity.

Rationale. Every such grant creates a Sybil incentive and a market for identities.
Verification. Inspection of every operation in Appendix C.

3.10.5 Strategies rejected

Table 51. Abuse strategies considered and rejected.

Strategy	Reason
Reputation scores	A score needs a global view, which needs a central service or a consensus system. A score is also a persistent label on a user: precisely the metadata the protocol works to remove.
Web-of-trust admission	Leaks the social graph to the participants and locks out a new user who knows nobody.
Payment or token deposit	Excludes users with no money or no payment rail, needs a ledger, and makes the network a target for financial regulation.
Phone number verification	A global identifier tying the identity to a state registry and a carrier, and it recreates the enumeration problem.
Central blocklists	A governance function needing an authority, which becomes the target of legal and commercial pressure.
Content filtering at a relay	A relay cannot read content. This is a feature.
CAPTCHA	Needs a vendor, excludes users with a disability, and machine solvers now cost less than the human ones.
Global rate limits per identity	Requires the relay to know the identity.
Adaptive proof of work on all traffic	Costs every honest user battery on every message, and adds nothing in the known-sender case the deposit key already solves.

3.10.6 Residual risk

Attack	Effect and residual risk
A contact turns hostile and floods a mailbox	The quota bounds it and the recipient deletes the mailbox. Residual: the user must act.
An attacker publishes the contact code of a victim	Introductions arrive at the effort cost, capped at 64 a day. Residual: the user must rotate and reissue.
A botnet floods a relay with connections	Token buckets and proof of work on registration. Residual: bandwidth exhaustion remains possible and affects every user of that relay; two relays are required.
An attacker fills a mailbox to block a real message	The quota is per mailbox and only the holder of the deposit secret can write. Residual: a hostile contact can block itself, which is not useful.
A relay operator floods its own users	The relay can forge nothing and can waste storage. Residual: the users move.

Table 52. Residual abuse risk.

3.11 Metadata resistance

3.11.1 Inventory

Table 53. What is exposed, to whom, and what remains.

Value	Seen by	Defence	Residual
Client IP address	Relay, transit	Onion or VPN, by user choice	Full exposure without one
Use of SSP	Transit	Binding B with a pluggable transport	Binding A is fingerprintable
Mailbox address	Relay	Per-pair derivation, daily rotation	Linkable within one day
Mailbox set of one client	Relay	Sharding, spread registration	One relay sees its own shard
Object size	Relay, network	Padding buckets	The bucket index leaks a range
Object count and time	Relay, network	Cover traffic in the high-risk mode	Full exposure by default
Object type class	Relay	None; in the clear	Five values
Deposit key	Relay	Unlinkable to any identity	Links deposits to one sender within an epoch
Blob size	Relay	64 KiB chunks, padded final chunk	Chunk count leaks an approximate size
Identity log	Contacts and serving relays	No name, no external identifier	Device count and change history
Group membership set	Not exposed as a set to any relay	Pairwise mailboxes, never a shared group mailbox	The commit type class marks a mailbox as belonging to a member of some group; a member sees the full membership
Message content	Nobody outside the session	The ratchet	An endpoint sees it

3.11.2 Padding buckets

SSP-MDR-006 Every envelope ciphertext MUST be padded to one of the sizes below, and the bucket field MUST name that size.

Rationale. Six buckets give a coarse size signal instead of an exact one. Finer would leak more; coarser would waste bandwidth. The 512-byte bucket keeps receipts cheap, and receipts dominate the object count.

Verification. Test: payloads of 1, 500, 4000, and 70 000 bytes are padded to buckets 0, 1, 2 and 4 respectively, and the ciphertext length equals the bucket size in each case.

Index	Size	Typical content
0	512 B	A receipt, a typing notice, a small control object
1	2 KiB	A short text message
2	8 KiB	A long text message, a small group commit
3	32 KiB	A group commit, a blob notice with a large key set
4	64 KiB	The maximum message
5	256 KiB	A large group commit only

Table 54. Padding buckets.

SSP-MDR-007 A client MUST pad with random bytes inside the AEAD, never outside it.

Rationale. Padding outside the AEAD is unauthenticated, and an attacker can strip or extend it to signal information.
Verification. Test: a receiver rejects an envelope whose padding was truncated or extended after encryption, because the AEAD tag fails.

SSP-MDR-008 A client MUST NOT choose a bucket larger than the smallest that fits, except in the high-risk mode.

Rationale. Free choice of bucket is a covert channel a compromised client can use to signal to a relay.
Verification. Test: a client encoding a 600-byte payload selects bucket 1 and not bucket 2 or above, over 1000 randomly sized payloads.

3.11.3 Deliberate leaks

Three values are visible on purpose.

Leak	Why it exists	What it costs
Object type, five values	The relay needs it for the expiry class and the quota class; without it an introduction and a message share a quota and the abuse model fails.	An observer learns whether an object is a message, an introduction, a commit, a blob notice, or a receipt. Commit traffic therefore reveals that a mailbox belongs to a group member.
Bucket, six values	The relay must check that the length matches, or a sender could signal in the difference.	A size range.
Deposit timing	A store-and-forward relay receives the object at some moment.	Full timing exposure by default.

Table 55. Deliberate plaintext in the envelope.

SSP-MDR-009 No further plaintext field MUST be added to an envelope in SSP/1.

Rationale. Each field looks small, and the set is what identifies a user. A version bump is the correct place to add one, with an explicit analysis.
Verification. Inspection of the envelope definition in 3.5.6 against the field list, at each revision.

3.11.4 Sharding and fingerprints

SSP-MDR-010 A client SHOULD distribute its mailboxes across its relay set so that no relay holds more than 60 per cent of them.

Rationale. A relay holding every mailbox of a client sees the full contact count and activity pattern. Sharding bounds what one relay sees; a coalition covering the whole relay set defeats it, and the user chooses that set. The exact ratio is a policy judgement rather than a wire rule, and a client that cannot reach two relays has to fall back to one.
Verification. Test: with 200 mailboxes and a relay set of three, no relay is given more than 120; with a relay set of one, the client reports the reduced protection rather than refusing to operate.

SSP-MDR-011 A client MUST use a separate transport connection per relay and MUST NOT reuse a source port in a way that correlates them.

Rationale. Two relays sharing a source port or a connection observe the same five-tuple, which links the two mailbox sets they each hold to one client.
Verification. Test with a packet capture: connections to two relays carry different source ports and share no transport session.

SSP-MDR-012 Every client MUST produce a byte-identical HELLO frame for the same version and capability set, and MUST NOT include a product name, a version string, a platform value, or a locale.

Rationale. A user agent string turns a client into a distinguisher and can identify one user out of a small population.
Verification. Test with a byte comparison across two implementations.

SSP-MDR-013 A client MUST NOT let a user-visible setting change a value the relay sees, beyond the mailbox set and the traffic the user causes.

Rationale. A rare setting combination is a fingerprint.
Verification. Inspection: no setting exposed in the user interface changes a byte the relay observes, other than through the mailbox set and the traffic the user generates.

SSP-MDR-014 A client MUST use the poll interval class for its power state from the fixed set in Appendix B.

Rationale. A custom poll interval identifies a client across relays and across addresses.
Verification. Test: over a one-hour capture in each power state, the interval between polls matches the class value and shows no client-specific jitter.

3.11.5 High-risk mode

SSP-MDR-015 A client SHOULD implement a high-risk mode, switchable per device. Where the mode is implemented and enabled, it MUST apply all of: constant-rate traffic of one object every 30 seconds, with padding frames filling empty slots; all traffic over an onion transport using binding B; every envelope padded to at least 8 KiB; and polling only on

the fixed schedule, never on a push notification.

Rationale. The mode raises the cost of the T6 attack within one relay view, at about 23 MiB of upload a day and up to 30 seconds of added latency. Whether to offer it is a product decision, and a client for a low-risk audience may reasonably omit it. Its parameters are not negotiable, because a mode that is only partly applied produces a distinctive traffic pattern of its own and is worse than not enabling it.
Verification. Test with a traffic capture over one hour: inter-deposit intervals are constant at 30 seconds, every object is at least 8 KiB, and no traffic leaves outside the schedule.

SSP-MDR-016 A client MUST state the bandwidth and latency cost before enabling the high-risk mode.

Rationale. The mode costs bandwidth and latency that matter on a metered or slow connection, and a user who does not expect that cost may disable the mode at the moment it is most needed.
Verification. Demonstration: enabling the mode presents the daily upload estimate and the added latency before it takes effect.

3.11.6 Limits

This subsection is normative. An implementation MUST NOT claim more than it states.

1. **A global passive adversary defeats the default mode.** An adversary watching both endpoints correlates a deposit at t with a fetch at $t + d$ for small d, and the bucket sizes match. The high-risk mode raises the cost within one relay view and does not defeat an adversary watching both endpoints over a long period.
2. **The relay set is a fingerprint.** A user with an unusual relay set is identifiable by it. Users SHOULD choose relays that many users choose, which conflicts with decentralisation. The conflict is stated rather than hidden.
3. **First contact is visible in time.** An introduction deposit tells the relay that some party met the owner of that mailbox. It does not say which party.
4. **Commit traffic is a signal.** A commit object at a mailbox tells the relay that the mailbox belongs to a member of some group. It does not say which group, how large it is, or which other mailboxes belong to it. A relay that correlates the simultaneous, equally sized deposits produced by one group message can nonetheless assemble a candidate membership set. Pairwise mailboxes hide the membership set; they do not give membership indistinguishability under timing analysis.
5. **Pre-registered mailboxes are forward-dated.** A client that registers W future epochs at once gives the relay a set of addresses known to activate on known dates (3.8.7).
6. **A mailbox links a pair within an epoch.** Rotation is daily, and an observer recording everything can chain the epochs of an active pair by activity correlation, without holding the derivation secret.
7. **Nothing protects a user against an endpoint.** TX-02 and TX-03.

3.12 Performance

These are preliminary engineering budgets. They were derived from the primitive costs and object sizes of 3.7 and 3.5, and none has been measured, because no implementation exists. They are recorded so that a design change can be seen to break one, and so that the first prototype has something to be compared against. A client SHOULD meet them; failing one is a signal to re-examine either the design or the budget, not a conformance failure.

Table 56. Preliminary performance budgets. Unmeasured; see the note above.

Reference	Statement	Target	Planned method
SSP-PRF-001	Session setup from a cached prekey bundle to first ciphertext, on a 2020 phone	50 ms	Test
SSP-PRF-002	Message encryption for one recipient device, 4 KiB payload	1 ms	Test
SSP-PRF-003	Identity log verification, fifty entries	20 ms	Test
SSP-PRF-004	Group commit object in a group of 4096 devices	256 KiB	Test
SSP-PRF-005	Relay deposit check and quota application, one core	200 μs	Test
SSP-PRF-006	Registered mailboxes accepted per link	512	Test
SSP-PRF-007	Mobile data per day, 200 contacts and 100 messages	5 MiB	Demonstration
SSP-PRF-008	Local state for 200 contacts and 20 groups, excluding bodies and blobs	50 MiB	Test
SSP-PRF-009	Heap allocations per message in the message path	2	Inspection
SSP-PRF-010	Cold start to first fetch, 2020 phone	2 s	Test
SSP-PRF-011	Equi-X verification at a relay	100 μs	Test
SSP-PRF-012	Blob encryption and hashing throughput, one core	20 MiB/s	Test

SSP-PRF-013 A client SHOULD batch its fetch operations across mailboxes rather than opening one request per mailbox.

Rationale. Two hundred mailboxes at one request each cost two hundred round trips and a great deal of radio wake time. This is a client efficiency choice: a client that ignores it interoperates correctly and drains its battery.

Verification. Test with a packet capture: a fetch across 200 subscribed mailboxes issues a single request and receives the objects on one stream.

3.13 Usability

Usability is a security property here, because a user who cannot understand a security event cannot act on it.

SSP-USE-002 A client SHOULD present a security event in plain language, and SHOULD NOT make a raw key, hash, or hexadecimal value the primary content of an alert.

Rationale. A user who cannot read an alert cannot act on it, and the alerts in this document exist to be acted on. The wording is a product decision, so this is a recommendation rather than a conformance rule.
Verification. Demonstration: a device-addition alert and a fork alert are shown to a reader unfamiliar with the protocol, who can state what happened and what to do.

SSP-USE-003 A client MUST support out-of-band authentication with a 60-bit short authentication string, and MUST derive that string from the SIDs of both parties in a canonical order.

Rationale. The canonical order is an interoperability rule: two clients that order the inputs differently show different words for the same pair, and the comparison fails for legitimate users. Sixty bits is enough against an online attack and fits in five words from a 4096-word list.
Verification. Test against the planned SAS vectors: both parties derive identical words regardless of which initiated, over 1000 random identity pairs.

SSP-USE-004 A client SHOULD NOT ask the user to make a cryptographic choice, and SHOULD NOT offer an option that weakens a default.

Rationale. Every such option is a way for a user to be argued into a worse configuration, and the protocol offers no algorithm agility for a user to select from in any case. This constrains product design rather than the wire.
Verification. Inspection of the settings surface against the primitive set of 3.7.

SSP-USE-005 A client SHOULD tell the user, at the first message to a new contact, that the contact is not yet authenticated out of band.

Rationale. The short authentication string is worth little if the user never learns that it is outstanding. The prompt is a presentation decision.
Verification. Demonstration: the first message to a contact added by code shows the unauthenticated state, and the indication clears after a successful comparison.

3.14 Interfaces

Interface	Parties	Binding
Relay link	Client and relay	3.8.2 or 3.8.3, frames of 3.5.5
Enrolment	Device and device	Noise IK over a rendezvous mailbox
Contact code	User and user	Bech32m text or QR code
Identity log	Any party	A signed byte string, transport independent
Blob	Client and relay	Chunk group put and get

Table 57. Protocol interfaces.

SSP-IFC-001 A client MUST support the `sunspot:` URI scheme with the forms `sunspot:code/...`, `sunspot:relay/...`, and `sunspot:id/...`, where the last form is display-only.

Rationale. One scheme with a fixed set of forms lets an operating system route a scan or a link. The identifier form is deliberately inert, because a SID alone gives no way to reach a user and SSP-MDR-005 requires that.
Verification. Test: each of the three forms is parsed and routed; a form with an unknown verb is rejected; the identifier form produces no network action.

SSP-IFC-002 A client MUST NOT act on a `sunspot:` URI without an explicit user action.

Rationale. A link in a web page must not be able to add a contact.
Verification. Demonstration: following a `sunspot:code/` link from a web page opens a confirmation surface and adds no contact until the user confirms.

SSP-IFC-003 An implementation SHOULD expose the protocol through an interface taking bytes and time and returning bytes and events, and SHOULD NOT require a network socket inside the protocol core.

Rationale. This is what makes the core testable against vectors and portable to a platform with a different network stack. It constrains implementation structure, not the wire, so a differently structured implementation can still be conformant.
Verification. Inspection: the core builds and its state machines run in a test that opens no socket and consults no system clock.

Requirement	Constraint and reason

3.15 Design constraints

Table 58. Design constraints.

Requirement	Constraint and reason
SSP-CON-004	The mandatory core SHOULD be implementable with no dynamic allocation in the parsing path. The hard rule that bounds allocation against hostile input is SSP-CON-003; this entry is the architectural goal that makes it easy to hold.
SSP-CON-005	The protocol MUST NOT need a wall clock accurate to better than five minutes. A protocol that needs an accurate clock needs a time source, and a time source is a trusted party.
SSP-CON-006	The protocol MUST NOT need a stable IP address, a public port, or an inbound connection at a client. Most clients sit behind address translation.
SSP-CON-007	The protocol MUST work with a relay having one core, 2 GiB of memory, and 100 GiB of disk, serving 10 000 users. A relay must be cheap to run or there will be few relays.
SSP-CON-008	A relay MUST NOT need state surviving a restart beyond the mailbox table and the object store.
SSP-CON-009	No protocol operation MAY need more than four round trips.
SSP-CON-010	The protocol MUST NOT contain a licence-encumbered algorithm or a patented mechanism. Every implementation must be free to write.

3.16 Quality attributes

3.16.1 Security

Three places carry most of the implementation risk. The *parser* reads attacker bytes; 3.5.4 fixes the grammar, 5.3 forbids allocation and requires a bounds check per field, and a fuzz campaign is mandatory. The *state machine* holds skipped keys and epochs; hard limits on both stop an attacker forcing unbounded state. The *key store* separates keys by purpose and encrypts at rest with crypto-erase.

SSP-SEC-002 An implementation MUST NOT log a key, a plaintext, a mailbox address, or a SID at any log level.

Rationale. Logs outlive the data they describe, are copied to crash reporters and log aggregators, and are read by people who are not the user. A key or a plaintext in a log defeats the storage protections of 3.9.11 entirely.

Verification. Test: a build-time check fails the build when a logging macro is reached from a type carrying secret material; and a full-session log at the most verbose level contains no SID, mailbox address, or key.

Property	Status	Source
Confidentiality of content	yes	3.7, 3.9.1
Integrity and authenticity	yes	AEAD and the ratchet
Forward secrecy	yes, per message	Ratchet plus key deletion
Post-compromise security	yes, after one round trip	The DH ratchet step
Deniability	yes, offline	No signature on messages
Post-quantum confidentiality	yes, for a recorded transcript	Hybrid ML-KEM-768
Post-quantum authenticity	no, in SSP/1	3.7.12
Identity binding	to a log, not a name	3.6
Transparency of key change	detectable by a contact	3.6.12
Replay resistance	yes, at ratchet and relay	3.9.2, 3.5.6
Downgrade resistance	yes, transcript binding	3.17.2

Table 59. Security properties.

SSP-SEC-003 An implementation MUST perform cryptographic comparisons in constant time.

Rationale. A data-dependent comparison over a MAC tag or a derived key leaks the comparison position through timing, which is enough to forge one byte at a time.
Verification. Inspection of every comparison over secret-derived values, plus a timing harness over 10^6 trials showing no correlation between the number of matching leading bytes and elapsed time.

SSP-SEC-004 An implementation MUST NOT continue after a cryptographic check fails, and MUST NOT reveal the reason to the peer beyond the defined error codes.

Rationale. A detailed error is an oracle.
Verification. Test: a bad AEAD tag, a bad signature, and an unknown mailbox each produce their defined code and no further processing; the peer cannot distinguish a bad tag from a bad signature by any other observable.

3.16.2 Privacy

SSP-PRV-001 A client MUST NOT send telemetry, a crash report, or a usage statistic carrying a mailbox address, a SID, a relay set, a contact count, or a group count.

Rationale. Telemetry leaves the device and is retained by whoever receives it. A contact count or a relay set is a fingerprint even without an identifier attached.
Verification. Inspection of every outbound payload other than protocol traffic, plus a capture confirming no such field is transmitted.

SSP-PRV-002 A client MUST NOT enable a push notification service that reveals a message or a sender to a third party, and MUST use a content-free wake signal if it uses a platform push service at all.

Rationale. A platform push service sits outside the trust model. A wake signal with no content tells it only that a device has traffic, which the network layer already reveals.
Verification. Test with a capture of the push payload: it carries no sender, no conversation identifier, and no ciphertext, and is identical for every message.

SSP-PRV-003 A client SHOULD make local storage of a conversation removable without a trace in an index, a search database, or a thumbnail cache.

Rationale. Crypto-erase of the conversation keys is defeated by a plaintext search index or a cached thumbnail beside it. The set of caches involved is platform-specific, so this is a recommendation rather than a testable wire rule.
Verification. Demonstration on each supported platform: after deleting a conversation, a raw scan of the application data directory finds no plaintext from it.

SSP-PRV-004 The protocol MUST NOT define a mechanism by which one party learns whether another is online, unless that party sends a message.

Rationale. Presence is high-resolution activity metadata, and it is the value users leak most freely. A presence service would also require a relay to know an identity.
Verification. Inspection of the operation registry: no operation returns the liveness, last-seen time, or connection state of another identity.

3.16.3 Reliability, maintainability, portability

SSP-SEC-005 A client SHOULD recover from a corrupt local store by re-derivation from the event log, and MUST NOT discard a key that it cannot re-derive.

Rationale. The event log of SSP-STO-004 is sufficient to rebuild every view, so corruption of a materialised view need not lose data. Losing an unre-derivable key, by contrast, permanently destroys the conversations it protects, which is why that half remains a hard rule.
Verification. Test: a materialised view file is truncated at a random offset and the client rebuilds it from the log with no message loss, over 100 trials; a separate trial confirms ratchet keys are never dropped during that rebuild.

SSP-SEC-006 A relay MUST survive power loss with no loss of an acknowledged deposit, and MUST flush an object to disk before acknowledging it.

Rationale. An acknowledgement is a promise that the object survives; a relay that acknowledges from a write buffer breaks that promise on power loss, and the sender has already deleted its queued copy.
Verification. Test: power is cut between the write and the acknowledgement across 100 trials; no acknowledged object is missing after restart.

SSP-GEN-008 Every wire object MUST have a test vector, and a change to an object MUST come with a change to the vector.

Rationale. A vector is the only artifact that makes an encoding rule checkable by a second implementation, and a rule whose vector is not updated with it silently stops describing the format.
Verification. Inspection during review: a change to a wire object is not merged without the corresponding change to the generation procedure of Appendix E.

SSP-GEN-009 The protocol core SHOULD be separable from transport, storage, and user interface at build level.

Rationale. Separation is what allows the core to be tested against vectors with no network and reused on a platform with a different stack. It is an architectural recommendation; a monolithic implementation can still be conformant on the wire.
Verification. Inspection of the dependency graph: the core builds with no dependency that opens a socket or a file.

SSP-GEN-010 The protocol core SHOULD build for a 32-bit target with no operating system, given an entropy source and a clock.

Rationale. A hardware token, a router, and a low-cost handset are all plausible clients, and a core that assumes a 64-bit host with an allocator excludes them for no protocol reason. This is a portability goal, not a conformance condition.
Verification. Test: a bare-metal build for a 32-bit target passes the encoding and session vectors.

3.17 Extensibility, versioning, and compatibility

3.17.1 Extension points

There are three, and no more: the content type of a message body, the extension list of a message, and the identity log entry type range 0x80 to 0xFF.

SSP-EXT-001 An extension MUST NOT change a cryptographic mechanism, a key schedule, a mailbox derivation, or a frame format.

Rationale. An extension touching the cryptography is a new protocol version under a false name. This rule is what stops SSP/1 becoming a family of incompatible dialects.
Verification. Inspection at registry review: a proposed extension that reads or writes key material, changes a derivation label, or alters a frame field is refused and directed to the version process.

SSP-EXT-002 A receiver MUST ignore an unknown non-critical extension and MUST reject the whole object on an unknown critical extension.

Rationale. The rule TLS and X.509 use. It lets a sender state whether the extension is needed for correct handling.
Verification. Test: an object carrying an unknown extension with the critical bit clear is processed and the extension preserved; the same object with the bit set is rejected whole.

SSP-EXT-003 A receiver MUST include an unknown extension in the bytes it hashes for the message identifier, and MUST NOT strip it before re-encoding.

Rationale. Stripping breaks the content address and the signature.
Verification. Test: a message with an unknown extension is re-encoded and rehashed; the identifier is unchanged and the signature still verifies.

SSP-EXT-004 A code point MUST come from the registry of Appendix C, and an experiment MUST use a private-use range.

Rationale. Two implementations choosing the same number for different meanings break the network in a way that is hard to diagnose.
Verification. Inspection of the assigned ranges at review, and a test that a code point in the private-use range is refused on a public relay link.

Registry policy: 0x0000–0x7FFF assigned by the maintainers with a specification; 0x8000–0xEFFF first come, first served with a public description; 0xF000–0xFFFF private use, never on a public network.

Fixed for the life of SSP/1. The primitive set, the frame header, the envelope layout, the mailbox derivation, the SAKA construction, the ratchet parameters, the group ciphersuite, and the padding bucket set. A change to any of them needs SSP/2.

3.17.2 Version negotiation

HELLO carries the supported versions, a capability bitmap, and a client nonce. HELLO_ACK carries the selection, the relay capability bitmap, the policy block, a relay nonce, and a transcript MAC:

$$T = H(\texttt{SSP1/hello} \| \text{HELLO} \| \text{HELLO\_ACK without the MAC}), \qquad mac = \mathrm{KH}(k_{\text{link}}, T)$$

SSP-EXT-005 A relay MUST bind the negotiated version and capability set to the transport key material with that MAC, and a client MUST check it.

Rationale. Downgrade protection. An attacker editing HELLO in flight cannot produce a valid MAC, because it does not hold the transport secret.
Verification. Test: an attacker rewrites the version list in a HELLO in flight; the client detects the MAC mismatch and closes the link before any operation.

SSP-EXT-006 A client MUST NOT offer a version below its highest supported version to avoid a failure, and MUST NOT retry at a lower version after a failure.

Rationale. An automatic downgrade retry is a downgrade oracle, and it defeats the MAC above at the application level. Several deployed protocols lost their downgrade protection exactly this way.
Verification. Test: a client offered only an unsupported version fails the link and does not reconnect at a lower version; a capture shows one attempt.

SSP-EXT-007 The end-to-end version MUST be negotiated inside the SAKA transcript, and MUST NOT be taken from the relay link.

Rationale. The relay is not trusted and cannot take part in an end-to-end negotiation.
Verification. Test: a relay that reports a different version in its link policy does not change the version the two endpoints derive inside the SAKA transcript.

A capability is a bit in a 128-bit map: it enables an optional behaviour and never changes a mandatory one. Bit 0 blob storage, bit 1 push wake, bit 2 datagram padding, bit 3 extended object lifetime, bit 4 proof of work currently not required, bits 5 to 127 reserved.

SSP-EXT-008 A party MUST NOT use a capability the other party did not offer, and MUST treat an unknown bit as not offered.

Rationale. A capability used without being offered is an undefined operation at the peer, and treating an unknown bit as offered would let a future revision silently enable behaviour an old peer cannot perform.
Verification. Test: a client invoking blob storage against a relay that did not offer it receives `NOT_SUPPORTED`; a reserved bit set by a peer is ignored.

3.17.3 Compatibility

SSP-EXT-009 A minor revision of SSP/1 MUST be additive, MUST NOT change an existing field, and MUST NOT change the meaning of an existing code point.

Rationale. A field whose meaning changes inside a version turns the version number into a lie, and every deployed peer that trusts it misparses.
Verification. Inspection at review, against the frozen list in 3.17.

SSP-EXT-010 An SSP/1 client MUST interoperate with any other SSP/1 client for every mandatory function, whatever the minor revision.

Rationale. This is the observable content of G-01: a minor revision that breaks a mandatory function has forked the network without changing the version.
Verification. Demonstration: implementations at the oldest and newest minor revisions complete the mandatory function set against each other.

The one break with the past. SSP/1 is not compatible with earlier SunSpot builds. The break happens once, and the reason is recorded here so that habit cannot repeat it: the old format has no canonical encoding, so no signature over it is sound; the old identity model has no key rotation, so every old identity must be reissued anyway; and the old addressing model uses a stable per-user identifier at the relay, which is exactly the property 3.8.6 removes, so a bridge would reintroduce the leak for every bridged conversation.

SSP-EXT-011 An SSP/1 client MUST NOT implement a bridge, a gateway, or a translation layer to a pre-SSP/1 build.

Rationale. A bridge would move plaintext, or the metadata leak, into the new network. Migration is a user action: create a new identity and send a contact code over the old system.
Verification. Inspection: no build target links a translation layer to a pre-SSP/1 format.

SSP-EXT-012 SSP/2 MUST run on a separate ALPN value and a separate frame version, and a relay MUST be able to serve both at once on one port.

Rationale. A flag day across a decentralised network is not possible. Two versions in parallel at one relay is the only way a network can move.
Verification. Demonstration: one relay serves `ssp/1` and a later ALPN value on one port, and a client of each version completes a deposit and a fetch.

4 Verification

This section states how a reviewer confirms that an implementation meets each requirement. None of the work described here has been carried out. There is no implementation, no test corpus, no fuzz campaign result, no benchmark, and no proof. Every artifact named below is a deliverable that conformance will depend on, and the plan is recorded now so that the requirements above are written against something checkable.

4.1 Strategy

Level	Object	Method
V1	A primitive and its parameters	Test against published vectors
V2	An object encoding	Test against the SSP vectors
V3	A state machine	Test against a scripted trace, plus model checking
V4	A protocol exchange	Demonstration between two implementations
V5	A property	Analysis, with a proof or a symbolic model
V6	A limit or a policy	Inspection of code and document

Table 60. Verification levels.

SSP-GEN-011 An implementation MUST pass every V1 and V2 case before any party calls it conformant.

Rationale. Encoding and primitives are where silent divergence starts, and a divergence there breaks every higher property.
Verification. Test: the implementation reproduces the encoding and primitive vectors of Appendix E in full, with no case skipped or marked expected-fail.

4.2 Selected cases

Table 61. Representative verification cases.

Requirement	Method	Case
SSP-IDN-001	Test	The SID of the reference genesis entry matches the published value.
SSP-IDN-004	Test	A log with a broken link at entry seven is rejected whole.
SSP-IDN-006	Test	A log with sequence 0, 1, 3 is rejected.
SSP-IDN-011	Test	A relay serving a fork triggers the alert path and stops sends.

Requirement	Method	Case
SSP-DEV-002	Test	A log with nine active devices is rejected.
SSP-DEV-004	Test	The enrolment handshake matches the Noise vector; a wrong pre-shared key fails.
SSP-KEY-009	Test	A claim naming an unknown `head_ref` is rejected; a claim valid under the policy at `head_ref` is accepted after a later policy replaces every key.
SSP-KEY-010	Test	A claim with a 71-hour pending window is malformed; an uncontested claim takes effect at 72 hours and not before.
SSP-KEY-011	Test	One contest, then five more; the deadline is identical in all six states and the claim takes effect at it.
SSP-KEY-013	Test	A rejection signed by $r - 1$ recovery keys leaves the claim live; one signed by r voids it.
SSP-KEY-014	Test	A compromised sole device removes its siblings and signs a rejection against three successive claims; every rejection is refused and every claim takes effect.
SSP-KEY-016	Test	A policy update between a claim and a rejection does not change which keys can reject that claim.
SSP-KEY-020	Test	The device introduced by a claim signs a contest against that claim; the contest is refused.
SSP-DEV-012	Test	With three devices, one removal is immediate; the removal that would leave one device is pending unless root-signed.
SSP-DEV-015	Test	Five successive removals of one target share the deadline of the first.
SSP-DEV-016	Test	Two devices each remove the other against the same `prev`; after ordering, one active device remains in every interleaving.
SSP-MDR-017	Test	A client offline for W epochs receives everything deposited in that window; a deposit at $W+1$ was refused at deposit time.
SSP-CRY-002	Test	Every hash output matches the BLAKE3 vector set.
SSP-CRY-003	Test	Each small-order X25519 input aborts the operation.
SSP-CRY-007	Test	The full ZIP 215 vector set gives the expected accept and reject sets.
SSP-CRY-009	Test	Removing either the X25519 or the ML-KEM output changes the session secret.
SSP-CRY-013	Test	Key generation blocks until the entropy source is ready.

4.3 Parser verification

SSP-GEN-012 An implementation SHOULD run a coverage-guided fuzz campaign against the envelope, message, identity log, and frame parsers, and MUST NOT be described as conformant while a known crash, hang, or memory error in those parsers is outstanding.

Rationale. The parsers read attacker-controlled bytes and are the highest-value target. Campaign length is an engineering judgement that depends on coverage saturation rather than on a fixed

count, so the number is not fixed here; an outstanding memory-safety defect in a parser is not a judgement call.
Verification. Test: the campaign log shows coverage saturation with no outstanding crash, hang, or sanitiser report against any of the four parsers.

SSP-GEN-013 An implementation MUST include a differential test against a second implementation for the accept or reject decision of every parser.

Rationale. Two parsers differing on one byte string give an attacker a way to split the network.
Verification. Test: the accept-or-reject decision of both implementations is compared over the full valid and malformed corpus; every disagreement is resolved against this document before either is described as conformant.

4.4 Property verification

No property below has been verified. The table records the intended method and the artifact that would constitute evidence.

Property	Planned method	Artifact that would satisfy it
Session confidentiality and authenticity	Analysis	A symbolic model of SAKA and the ratchet in ProVerif or Tamarin
Downgrade resistance	Analysis, Test	Transcript binding inside the model
Group forward silence after removal	Test	A removed member cannot decrypt the next epoch, over 1000 random membership sequences
Convergence of concurrent commits	Test	10^5 random concurrency traces reach one epoch state on every member
Convergence of display order	Test	10^5 random delivery orders of one graph give one order
Mailbox unlinkability	Analysis	The derivation is a PRF, so two epochs are indistinguishable without the root

Table 62. Property verification.

4.5 Interoperability suite

No interoperability suite exists yet. Conformance testing for SSP/1 depends on one, and this subsection specifies what it has to contain so that the requirement can be written now and satisfied later.

SSP-GEN-014 Before an implementation is described as conformant, an interoperability suite MUST exist that provides a reference relay, a scripted client, a hostile relay, and a traffic recorder, and the implementation MUST pass it.

Rationale. G-01 cannot be verified by reading. It needs a second implementation and a harness that a third party can run unaided.
Verification. Demonstration: two implementations, developed against this document without shared code, complete the mandatory function set against the suite.

The hostile relay has to be able to drop an object silently; reorder objects and reuse a sequence number; serve a stale or forked identity log; serve one prekey to two requesters; lie about its

policy limits; delay an object arbitrarily; and replay a deposit. A conformant client is tested against all seven.

4.6 Traceability

Appendix A links each requirement to a goal, a threat, and a verification method. A requirement tracing to neither a goal nor a threat is unnecessary and MUST be removed.

5 Implementation guidance

This section is informative. It records the implications of the design for a Rust implementation, because several rules above exist for reasons that only become visible at the code level.

5.1 Shape

The protocol core is a state machine taking bytes and time and returning bytes and events. It owns no socket, no thread, no timer, and no file, which is what makes every verification case in Section 4 a table of inputs and expected outputs with no network and no clock.

5.2 Crate boundaries

Crate	Contents
`ssp-wire`	Encoding and decoding, zero copy, no allocation in the parse path
`ssp-crypto`	Thin traits over the primitives, no algorithm choice
`ssp-identity`	Identity log, entries, verification, fork detection
`ssp-session`	SAKA and the double ratchet
`ssp-group`	TreeKEM, epochs, commit resolution
`ssp-mailbox`	Derivation, envelope construction, padding
`ssp-relay-proto`	Frames, operations, policy; no input or output
`ssp-transport`	QUIC and TCP-with-Noise bindings; async
`ssp-store`	Local encrypted store and event log; platform-specific
`ssp-client`	Orchestration
`ssp-relay`	The relay server
`ssp-testkit`	Vectors, hostile relay, fuzz targets

Table 63. Crate layout.

The rule that keeps this honest: the first six crates must not depend on an async runtime, on `std::net`, or on anything that opens a file. A dependency check in continuous integration enforces it.

5.3 Zero-copy parsing

The encoding is designed for a borrowed parse: a parsed object holds slices into the input buffer and copies nothing.

```
pub struct EnvelopeRef<'a> {
    pub etype:      u8,
    pub mailbox:    &'a [u8; 32],
    pub nonce:      &'a [u8; 16],
    pub bucket:     u8,
    pub ciphertext: &'a [u8],
    pub signature:  &'a [u8; 64],
}
// parse(): check total length against the bucket table first; take fixed
// fields by exact slice, never by index arithmetic; reject trailing bytes.
```

Three habits matter. Use checked splitting, never an index that a wire-controlled length field computes. Return a borrowed view and let the caller decide to own it, which is what keeps SSP-PRF-009 achievable. Make the parse function total: it takes bytes, returns a result, and panics on nothing.

Do not put `serde` on the wire. The protocol reason is in 3.5.4; the code reason is that a derive macro places the wire format under the control of a struct definition, so a field reorder during a refactor silently changes the network. `serde` is fine for a configuration file and a local store, where the format carries no cross-implementation meaning.

5.4 Type discipline

The protocol carries many 32-byte values with very different meanings. A raw `[u8; 32]` everywhere makes it possible to pass a mailbox address where a SID belongs and the compiler will not object. Wrap each in a distinct newtype with no dereference to the inner array. Apply the same discipline to keys: a signing key should not implement `Clone`, and a secret type should implement neither `Debug` nor `Display`, which turns SSP-SEC-002 into a compile-time property rather than a review item.

5.5 Zeroization

Rust makes SSP-KEY-003 harder than it looks. A growable buffer reallocates and leaves the old allocation holding the key. A clone of a secret makes a copy no destructor tracks.

- Hold secrets in a fixed-size array inside a wrapper whose destructor performs a volatile write. Use an established zeroization crate rather than a hand-written loop the optimiser may remove.
- Never place a secret in a growable container.
- Do not derive `Clone` on a secret type. Where a copy is genuinely needed, make it an explicit method with a name a reviewer notices.
- Lock the pages of the key store where the platform allows it, so a secret does not reach a swap file.
- Document the truth: a virtual machine that snapshots memory, and a core dump, both defeat this. Zeroization narrows the window and does not close it.

5.6 Async design

The transport layer is async and the core is not. The boundary is one task per relay connection with one channel into the core.

- Give the core exclusive ownership of protocol state behind one actor task. Shared mutable state behind a lock invites a deadlock between a fetch task and a send task.
- Put a deadline on every operation, and make the cancel path leave the ratchet whole. Ratchet state must advance atomically with the durable write, or a crash between the two loses a key.
- Use one connection per relay with one stream per logical operation, and do not spawn a task per mailbox.

5.7 Constant time

Use an established constant-time comparison crate for any secret-derived value. Comparing a MAC tag with slice equality is a timing oracle, and it is the defect reviewers find most often. The parser is not constant time and does not need to be, because it reads public bytes; do not confuse the two cases and do not slow the parser for nothing.

5.8 Build order

Each stage below is testable before the next begins, and every stage before the fifth is a pure function of bytes, so a defect is found by a vector rather than a packet capture.

1. `ssp-wire`, against the encoding vectors. No network.
2. `ssp-identity`, against the log vectors and the fork cases.
3. `ssp-session`, against the SAKA and ratchet vectors.
4. `ssp-mailbox`, against the derivation vectors.
5. `ssp-relay-proto` and a relay with an in-memory store.
6. `ssp-transport`, binding A first, binding B second.
7. `ssp-group`, against the concurrency traces.
8. Blobs, then the self group, then the high-risk mode.

6 Research agenda

Each item below has a decision in SSP/1. The entry states what would have to change for that decision to be revisited.

6.1 Private retrieval

Now. A client tells a relay which mailbox addresses it reads, and sharding bounds the view of any one relay.

What would change it. A practical private information retrieval scheme letting a client fetch from a set of N mailboxes without naming them. Current schemes cost either linear server work per query or large client state, and neither is acceptable at the scale of 3.12. A single-server scheme with sublinear amortised cost and a small hint would remove the strongest remaining relay-side leak.

6.2 Group fan-out

Now. One envelope per recipient device, buying full membership confidentiality against a relay.

What would change it. A construction letting a relay serve one ciphertext to N subscribers without learning that those N form a set. Broadcast encryption solves the key half and not the addressing half. An oblivious relay inside a trusted execution environment would work and would add a hardware vendor to the trust model, which the trust assumptions do not accept.

6.3 Post-quantum authenticity at an acceptable size

Now. Ed25519 with a defined migration.

What would change it. A post-quantum signature with a public key under 200 bytes and a signature under 1 KiB, with a comparable analysis record. Current lattice and hash-based schemes miss this by a large factor, and the identity log is where the size hurts most.

6.4 Anonymous credentials for first contact

Now. A contact code plus proof of work.

What would change it. A blind-signature or oblivious-PRF token that a recipient issues, a sender redeems once, and a relay verifies without linking redemption to issue. That would replace proof of work with an unlinkable capability and remove the battery cost of the introduction path. The construction is well understood; the open problem is distributing tokens to a party the recipient has never met, which is exactly the case the contact code covers.

6.5 Traffic analysis at an acceptable cost

Now. An opt-in constant-rate mode, with the default mode explicitly not resistant to a global passive adversary.

What would change it. A mix network design a phone can afford, with latency a messaging user accepts. The Loopix family is closest, and its cost model assumes a persistently connected client. A version surviving a mobile radio duty cycle would let the mode become the default.

6.6 Formal verification of the composition

Now. A symbolic model of SAKA and the ratchet.

What would change it. A machine-checked proof covering the identity log, the session layer, the group layer, and the addressing layer in one model. The parts have proofs; the composition does not, and the composition is where a protocol usually fails.

A Traceability matrix

A requirement tracing to neither a goal nor a threat is unnecessary. Withdrawn identifiers are retained here so that the numbering stays stable across revisions and so that a reference in an older review still resolves.

Table 64. Requirement traceability.

Requirement	Goal	Threat	Method
SSP-GEN-001	G-01	—	Inspection, Test
SSP-GEN-002	G-07	—	Inspection
SSP-GEN-003	G-02	TH-01	Test
SSP-GEN-004	G-01	—	Demonstration
SSP-GEN-005	G-12	—	Inspection
SSP-GEN-006	G-04	TH-11	Inspection, Test
SSP-GEN-007	G-02	TH-04	Test
SSP-GEN-008	G-01, G-09	—	Inspection
SSP-GEN-009	G-12	—	Inspection
SSP-GEN-010	G-12	—	Test
SSP-GEN-011	G-01	—	Test
SSP-GEN-012	G-12	TH-19	Test
SSP-GEN-013	G-01	TH-19	Test
SSP-GEN-014	G-01	—	Demonstration
SSP-IDN-001 to 003	G-05	TH-04	Test, Inspection
SSP-IDN-004 to 006	G-05	TH-04	Test
SSP-IDN-007	G-09	—	Test
SSP-IDN-008, 009	G-05	TH-04	Test
SSP-IDN-010 to 013	G-05	TH-09	Test, Demonstration
SSP-IDN-018	G-05	TH-09, TH-21	Test
SSP-IDN-014 to 017	G-04, G-08	TH-10, TH-13	Test, Demonstration
SSP-DEV-001 to 003	G-06	TH-05	Inspection, Test
SSP-DEV-004 to 007	G-06	TH-05	Test, Demonstration
SSP-DEV-008 to 011	G-03, G-06	TH-18	Test
SSP-DEV-012 to 016	G-05, G-06	TH-05, TH-21	Test, Demonstration
SSP-KEY-001 to 003	G-02, G-03	TH-01, TH-02	Inspection, Test
SSP-KEY-004 to 006	G-03, G-05	TH-02	Test
SSP-KEY-007, 008	G-05	TH-06	Test
SSP-KEY-009 to 011	G-05	TH-06, TH-21	Test
SSP-KEY-012	G-05, G-13	TH-06, TH-21	Demonstration
SSP-KEY-013 to 015	G-05	TH-06, TH-21	Test
SSP-KEY-016 to 018	G-05	TH-06	Test
SSP-KEY-019, 020	G-05	TH-21	Test
SSP-CRY-001 to 008	G-02, G-12	TH-01, TH-20	Test, Analysis

Requirement	Goal	Threat	Method
SSP-CRY-009 to 011	G-10	TH-03	Test, Analysis
SSP-CRY-012 to 014	G-02	TH-01	Test, Inspection
SSP-NET-001, 002	G-11	TH-15	Test
SSP-NET-003	G-07	TH-04	Test
SSP-NET-004, 005	G-07	TH-08	Test, Inspection
SSP-NET-006, 007	G-02	TH-20, TH-07	Test
SSP-NET-008 to 011	G-07	TH-15	Test, Demonstration
SSP-NET-012, 013	G-07	TH-10	Demonstration, Analysis
SSP-RLY-001 to 004	G-04, G-08	TH-14, TH-15	Test
SSP-RLY-005 to 007	G-11	TH-08	Test
SSP-RLY-008, 009	G-04	TH-11	Inspection
SSP-RLY-010 to 013	G-11	TH-15	Test
SSP-RLY-014	G-07	—	Demonstration
SSP-RLY-015	G-11	TH-15	Test
SSP-MSG-001 to 003	G-08	TH-07, TH-14	Test
SSP-MSG-004 to 008	G-02, G-03	TH-01, TH-04	Test, Analysis
SSP-MSG-009 to 012	G-03	TH-02	Test
SSP-MSG-013 to 015	G-02	TH-04	Test
SSP-MSG-016 to 020	G-01	TH-08	Test, Demonstration
SSP-MSG-021 to 023	G-04	TH-11	Test, Inspection
SSP-MSG-024, 025	G-01	TH-08	Test
SSP-GRP-001 to 006	G-02, G-03	TH-18	Test, Inspection
SSP-GRP-007	G-04	TH-17	Test, Analysis
SSP-GRP-008 to 010	G-01	TH-08	Test
SSP-GRP-011, 012	G-13	—	Test, Demonstration
SSP-ATT-001 to 006	G-02, G-04	TH-12	Test
SSP-SYN-001 to 006	G-06	TH-05	Test, Demonstration
SSP-STO-001 to 005	G-03	TH-02	Inspection, Demonstration
SSP-ABU-001	*Withdrawn. Superseded by SSP-MSG-001.*		
SSP-ABU-002 to 004	G-08	TH-14	Test
SSP-ABU-005 to 008	G-08	TH-13	Test, Demonstration
SSP-ABU-009 to 013	G-08, G-11	TH-15	Test, Analysis
SSP-ABU-014	G-08	TH-16	Inspection
SSP-MDR-001 to 005	G-04	TH-10, TH-11	Test, Analysis

Requirement	Goal	Threat	Method
SSP-MDR-017	G-04, G-11	TH-11	Test
SSP-MDR-006 to 009	G-04	TH-12	Test, Inspection
SSP-MDR-010 to 014	G-04	TH-12	Test, Analysis
SSP-MDR-015, 016	G-04, G-13	TX-01	Test, Demonstration
SSP-SEC-001 to 006	G-02	TH-01, TH-19	Inspection, Test
SSP-PRV-001 to 004	G-04	TH-11	Inspection, Test
SSP-PRF-001 to 013	G-11, G-12	—	Test, Demonstration
SSP-USE-001 to 005	G-13	TH-04	Demonstration
SSP-IFC-001 to 003	G-01	TH-10	Test, Inspection
SSP-CON-001 to 010	G-09, G-12	TH-19	Test, Inspection
SSP-EXT-001 to 012	G-09	TH-20	Test, Inspection

B Constants and registries

B.1 Sizes

Constant	Value	Constant	Value
SID	32 B	Maximum envelope	262 144 B
Mailbox address	32 B	Maximum blob	104 857 600 B
Device identifier	8 B	Maximum identity log	256 KiB, 1024 entries
Deposit nonce	16 B	Devices per identity	8
Ed25519 signature	64 B	Identities per group	1024
X25519 public key	32 B	Devices per group	4096
ML-KEM-768 key	1184 B	Parents per message	8
ML-KEM-768 ct	1088 B	Relays per relay set	4
Blob chunk	65 536 B	Mailboxes per link, min	512

Table 65. Size constants.

B.2 Times

Constant	Value	Constant	Value
Mailbox epoch	24 h	Old epoch retention in a group	7 days
Mailbox overlap window	24 h	Skipped key retention	7 days
Recovery pending period	72 h	Message queue lifetime	7 days
Recovery contested period	30 days	Default object expiry	14 days
Device removal pending period	72 h	Group commit expiry	30 days
Signed prekey rotation	7 days	Clock skew tolerance	5 min
Link and KEM rotation	180 days	Future timestamp rejection	24 h
Root key rotation	365 days	Group update proposal	7 days
Group commit interval	30 days	Mailbox pre-registration window	open, O-05

Table 66. Time constants.

B.3 Argon2id parameters

Use	Memory	Iterations	Parallelism	Output
Recovery key derivation	256 MiB	4	2	32 B
Local store key from a passphrase	64 MiB	3	4	32 B

Table 67. Argon2id parameters.

Recovery parameters are heavier deliberately: recovery happens once, in a crisis, and protects a key an attacker can grind offline for years. A store derivation happens at every unlock and must not annoy the user into a shorter passphrase.

B.4 Derivation labels

Label	Parent key	Output
`SSP1/saka-root`	concatenated DH and KEM secrets	64 B
`SSP1/saka-transcript`	plain hash	32 B
`SSP1/hybrid-combine`	plain hash	64 B
`SSP1/ratchet-root`	root key	64 B
`SSP1/ratchet-chain`	chain key	64 B
`SSP1/ratchet-header`	chain key	32 B
`SSP1/mailbox-root`	session secret	32 B
`SSP1/mailbox-addr`	mailbox root	32 B
`SSP1/deposit-seed`	mailbox root	32 B
`SSP1/blob-key`	random	32 B
`SSP1/store-key`	device master key	32 B
`SSP1/channel-epoch`	previous epoch key	32 B
`SSP1/commit-tiebreak`	plain hash	32 B
`SSP1/pow`	plain hash	32 B
`SSP1/rendezvous`	plain hash	32 B
`SSP1/recovery-salt`	plain hash	16 B
`SSP1/sas`	session secret	8 B
`SSP1/deposit`	signature domain	—
`SSP1/mbreg`	signature domain	—
`SSP1/idlog`	signature domain	—
`SSP1/msg`	hash domain	—
`SSP1/hello`	transcript domain	—

Table 68. Key derivation label registry.

B.5 Poll interval classes

Class	Power state	Interval
0	Foreground, mains power	Persistent subscription
1	Foreground, battery	Persistent subscription
2	Background, mains power	60 s
3	Background, battery	300 s
4	Low power	900 s
5	High-risk mode	30 s, fixed, with cover traffic

Table 69. Poll interval classes. A client uses one of these values and no other, so that the poll pattern is not a fingerprint.

C Code point registries

C.1 Error codes

Code	Name	Meaning
0x0001	PROTOCOL_ERROR	A frame broke the grammar.
0x0002	VERSION_UNSUPPORTED	No common version.
0x0003	MAILBOX_UNKNOWN	No registration for that address.
0x0004	MAILBOX_FULL	The byte limit is reached.
0x0005	QUOTA_EXCEEDED	The object count limit is reached.
0x0006	BAD_SIGNATURE	A signature did not verify.
0x0007	REPLAY	The deposit nonce is in the cache.
0x0008	POW_REQUIRED	A proof of work is needed.
0x0009	POW_INVALID	The solution failed.
0x000A	RATE_LIMITED	A token bucket is empty.
0x000B	TOO_LARGE	A length limit is exceeded.
0x000C	RELAY_FULL	The relay accepts no new mailboxes.
0x000D	NOT_SUPPORTED	The capability was not offered.
0x000E	INTERNAL	An unspecified relay failure.

Table 70. Error codes. A relay MUST NOT define a code that reveals a property of a mailbox or of another client.

C.2 Message content types

Code	Name	Code	Name
0x0001	Text, UTF-8	0x0100	Outgoing message mirror
0x0002	Delivery receipt	0x0101	Read state update
0x0003	Read receipt	0x0102	Contact list change
0x0004	Typing notice	0x0103	Mailbox state share
0x0005	Blob reference	0x0104	Profile change
0x0006	Reaction	0x0105	Protocol-affecting setting
0x0007	Edit	0x0106	History bundle chunk
0x0008	Delete request	0x0200	Group proposal
0x0009	Reply marker	0x0201	Group commit
0x000A	Contact code share	0x0202	Group welcome
0x000B	Log head gossip	0x0203	Group metadata change
0x000C	Group welcome pointer	0x0204	Channel epoch key
0x0010–0x001F	Reserved for media		

Table 71. Content types. Conversation types on the left, synchronisation and group control on the right.

D Object layouts

Three objects are given here in full, because an implementer has to reproduce them byte for byte before anything else will interoperate: the object a client registers at a relay, the identity log entry that carries a device, and the header every message begins with. Every value is unsigned and big-endian, per 3.5.4. Offsets are in bytes from the first byte of the object.

D.1 Mailbox registration

The payload of an `MB_REGISTER` operation (3.8.4). The recipient constructs it, signs it with the deposit secret it is about to bind to the mailbox, and thereby proves possession of the key it is registering (SSP-RLY-004). Every field is fixed width, so the object is 140 bytes.

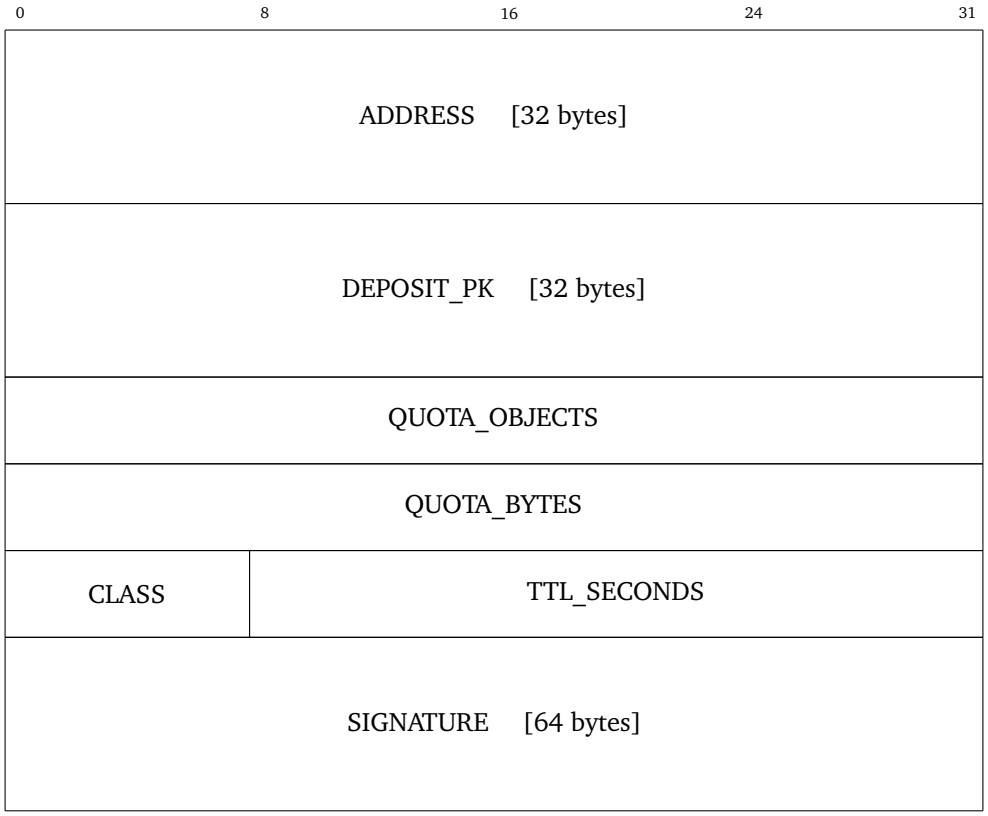

Figure 16. Mailbox registration payload.

Offset	Size	Field	Meaning
0	32	ADDRESS	The mailbox address, from the derivation of 3.8.6. Opaque to the relay.
32	32	DEPOSIT_PK	Ed25519 public key. The relay checks every later deposit against it.
64	4	QUOTA_OBJECTS	Objects accepted per 24 hours, bounded by the relay policy.
68	4	QUOTA_BYTES	Total bytes held for this mailbox, bounded by the relay policy.
72	1	CLASS	Storage class, selecting the expiry column of 3.8.8.
73	3	TTL_SECONDS	Requested mailbox lifetime, bounded by the relay policy (SSP-RLY-015).
76	64	SIGNATURE	Ed25519 over the byte sequence below.

Table 72. Mailbox registration fields. Total size 140 bytes.

The signature covers a domain label and every preceding field, in wire order and with no separator other than the label terminator:

$$\texttt{SSP1/mbreg} \parallel \texttt{0x00} \parallel \texttt{ADDRESS} \parallel \texttt{DEPOSIT\_PK} \parallel \texttt{QUOTA\_OBJECTS} \parallel \texttt{QUOTA\_BYTES} \parallel \texttt{CLASS} \parallel \texttt{TTL\_SECONDS}$$

The signature field itself is not covered. A verifier reconstructs the sequence from the received bytes rather than re-encoding a parsed structure, so that a non-canonical encoding cannot verify (SSP-CON-002).

D.2 Identity log entry, device addition

Entry type `0x02` of 3.6.2. Only the first three fields align to a 32-bit boundary, so the entry is given as offsets rather than as a bit diagram.

Offset	Size	Field	Value
0	12	LABEL	`SSP1/idlog` and two zero bytes
12	1	VERSION	`0x01`
13	1	TYPE	`0x02`, device addition
14	4	SEQ	One more than the previous entry
18	32	PREV	Hash of the previous entry
50	8	TIME	Milliseconds since the UNIX epoch
58	8	NOT_BEFORE	When the entry takes effect
66	8	DEVICE_ID	First eight bytes of H(DEVICE_PK)
74	32	DEVICE_PK	Ed25519 signature key
106	32	LINK_PK	X25519 static key
138	1184	KEM_PK	ML-KEM-768 encapsulation key
1322	4	CAPS	Capability bits, including the enrol capability
1326	8	ADDED_AT	Timestamp
1334	2	SIG_COUNT	Number of signature pairs that follow
1336	$72c$	SIGNATURES	c pairs of KEY_ID [8] and SIG [64]

Table 73. Device addition entry fields.

The fixed part is 1336 bytes and each signature pair adds 72, so an entry carrying one signature is 1408 bytes. SSP-DEV-002 caps an identity at eight devices, so the device records in a log contribute at most 11 264 bytes. Signatures cover the entry from offset 0 to offset 1333 inclusive, which is every field before SIG_COUNT.

D.3 Message header

The header of the end-to-end message object of 3.9, carried inside the envelope ciphertext. Offsets are fixed up to the parent list; the list is variable, so later fields are given as sizes.

Offset	Size	Field	Value
0	12	LABEL	`SSP1/msg` and four zero bytes
12	1	VERSION	`0x01`
13	32	SENDER_SID	Identifier of the sending identity
45	8	SENDER_DEV	Identifier of the sending device
53	32	CONV_ID	Pair hash, or group identifier
85	32	MSG_ID	Content address, zero while hashing (SSP-MSG-013)
117	8	COUNTER	Per sending device, per conversation
125	8	SENT_AT	Advisory sender clock (SSP-MSG-015)
133	2	PARENT_COUNT	p, from 1 to 8 (SSP-MSG-016)
135	$32p$	PARENTS	p parent message identifiers
var	32	LOG_HEAD	Identity log head of the sender (SSP-IDN-013)
var	2	FLAGS	Reserved, zero in SSP/1
var	2	CONTENT_TYPE	From the content type registry of C
var	4	PAYLOAD_LENGTH	Byte count of the payload
var	var	PAYLOAD	Body, interpreted per content type
var	2	EXT_COUNT	Number of extensions
var	var	EXTENSIONS	Triples of type, length, and value (SSP-EXT-002)

Table 74. Message header and body fields.

Everything from LOG_HEAD onward sits at $167 + 32(p - 1)$ and beyond, where p is the parent count. A header with one parent, an empty payload, and no extensions is 209 bytes; each additional parent adds 32. SSP-EXT-003 requires that unknown extensions stay inside the bytes hashed for MSG_ID, so a parser has to preserve the exact received extension bytes rather than re-encode them.

E Test vectors

The corpus described here does not exist. It is a planned conformance deliverable. This appendix fixes the generation procedure now, so that the verification method attached to each requirement names something reproducible, and so that the corpus two implementers generate is the same corpus.

Publishing a fixed set of vectors would give an implementer a small number of cases. Publishing a deterministic generator gives an unlimited set from a short definition, and removes the need to ship and version large binary fixtures.

Every value derives from one seed and a path:

$$value(path, n) = \mathsf{BLAKE3}_{\text{keyed}}\big(seed,\ \texttt{SSP1/vector/} \| path\big)[0\,..\,n]$$
$$seed = H(\texttt{sunspot-proposal-a-vector-seed-v1})$$

A key pair uses $value(path, 32)$ as the seed of the relevant key generation function. The generator needs no random source, so two implementations produce the same corpus byte for byte. No digest of the corpus is given here, because no corpus has been generated; the digest becomes part of the release that first publishes one.

Path prefix	Planned content
`identity/`	Identity logs covering rotation, device addition and removal, and the removal-to-single-device pending path, with invalid variants for each header and signature rule
`recovery/`	The claim state machine: uncontested expiry, single contest, repeated contests against one claim, root-key contest, contest by a device introduced by the claim, valid and invalid rejections, rejection thresholds differing from claim thresholds, a compromised sole device attempting denial, a stolen recovery key, and policy updates concurrent with a pending claim
`enrolment/`	Enrolment transcripts and failure cases
`wire/`	Valid encodings, and malformed or non-canonical variants each paired with its expected rejection reason
`labels/`	Every derivation label with a known input and output
`argon/`	Argon2id derivations across passphrases and normalisation forms
`crypto/`	The ZIP 215 cases and the small-order X25519 set
`mailbox/`	Mailbox derivations across epochs, including pre-registration windows and the unknown-mailbox retry path
`ordering/`	Message graphs, each with many random delivery orders and one expected display order
`group/`	Group traces with concurrent commits and the expected converged state
`sas/`	Short authentication string derivations

Table 75. Planned vector corpus. Counts are deliberately unstated: they are fixed by the release that generates the corpus, not by this document.

SSP-GEN-015 An implementation MUST reproduce the corpus from the published seed and generation procedure, and MUST agree with the released corpus digest, once both are published.

Rationale. A single digest makes the whole corpus verifiable in one comparison, so a silent divergence in one derivation cannot pass unnoticed.

Verification. Test, once the corpus is released: an independent regeneration from the seed matches

> the released digest.

Each invalid case carries the bytes, the expected error code, and the clause it tests. An implementation has to reject the bytes and return that code, which is what makes two parsers agree rather than merely both refusing.

F Comparison with existing systems

Table 76. SSP against comparable systems.

Property	Signal	Matrix	SimpleX	Session	SSP
Identifier	phone number	user on a home-server	none, per queue	public key	hash of a log genesis
Stable across key change	yes	yes	n/a	no	**yes**
Central service needed	yes	per home-server	no	node network	**no**
Directory or lookup	yes, enclave	yes	no	no	**no**
Relay learns the recipient	yes	yes	no	partly	**no**
Address changes over time	no	no	per queue	no	**daily**
Multi-device	with a primary	yes	limited	limited	**no primary**
Group protocol	sender keys	Megolm	pairwise queues	closed groups	**TreeKEM, no ordering service**
Membership set exposed to the server	yes	yes	no	partly	**no; commit traffic still marks a mailbox as a member of some group**
Post-quantum key agreement	yes	no	yes	no	**yes, hybrid**
Spam defence	phone number	server policy	out-of-band links	proof of work	**recipient capability, PoW at first contact only**
Value of seizing a server	some metadata	room state	little	little	**ciphertext and opaque addresses**
Federation	no	yes	no	n/a	**no, by decision**
Needs the reference client to interoperate	in practice	no	no	in practice	**no, by requirement**

Three differences deserve a note.

Against Signal. Signal is the reference for cryptographic quality and SSP reuses its ratchet. The difference is identity and infrastructure. Signal ties an account to a phone number and runs one service, which gives it a very good spam answer and a very good user experience, and gives a central point a state can pressure. SSP trades the user experience for the removal of that point.

Against SimpleX. SimpleX reaches a similar addressing property with unidirectional queues and no long-term identity at all. SSP keeps a long-term identity, because a stable identifier is what makes rotation, recovery, and a durable contact list possible, and then hides that identity from the relay with the pairwise derivation. The aim is to hold both properties at once.

Against Matrix. Matrix is the reference for an open specification with many implementations, and SSP copies that discipline. It rejects the replication model: sending room state to every participating server multiplies metadata exposure by the server count, and the homeserver holds the account.

G Acronyms

Term	Expansion	Term	Expansion
AEAD	authenticated encryption with associated data	PCS	post-compromise security
ALPN	application-layer protocol negotiation	PIR	private information retrieval
DAG	directed acyclic graph	PoW	proof of work
DH	Diffie-Hellman	PSK	pre-shared key
DHT	distributed hash table	QUIC	the transport of RFC 9000
KDF	key derivation function	RPK	raw public key
KEM	key encapsulation mechanism	SAKA	SunSpot asynchronous key agreement
MAC	message authentication code	SAS	short authentication string
MLS	messaging layer security	SID	SunSpot identifier
SSP	SunSpot protocol	SWE	SunSpot wire encoding
TTL	time to live	VOPRF	verifiable oblivious pseudorandom function
XOF	extendable output function		

Table 77. Acronyms.